UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

GRUPO #2

Plataforma de comercio electrónico para tienda de ropa con vestidores virtuales que utilice realidad aumentada

INTEGRANTES:

Barrios Lozano Juan Pablo Mateo	221043810
Contreras Fuentes Santiago	222008059

DOCENTE:	Ing. Angelica Garzon Cuellar
MATERIA:	Sistemas de Información II
SEMESTRE:	2/2026

Santa Cruz - Bolivia

CAPÍTULO I. ASPECTOS GENERALES....3
1.1. Introducción....3
1.2. Planteamiento del problema....4
1.3. Objetivos del proyecto....5
1.3.1. Objetivo general....5
1.3.2. Objetivos específicos....5
1.4. Justificación....6
1.5. Alcance....6
1.5.1. MÓDULO 1: GESTIÓN DE USUARIOS, ROLES Y AUTENTICACIÓN....6
1.5.2. MÓDULO 2: GESTIÓN DE SUCURSALES Y CIUDADES....7
1.5.3. MÓDULO 3: GESTIÓN DE CATÁLOGO DE PRODUCTOS....7
1.5.4. MÓDULO 4: GESTIÓN DE PROVEEDORES....7
1.5.5. MÓDULO 5: CONSULTA DE CATÁLOGO Y DISPONIBILIDAD....8
1.5.6. MÓDULO 6: VESTIDOR VIRTUAL (REALIDAD AUMENTADA)....8
1.5.7. MÓDULO 7: RESERVA DE PRENDAS....8
1.5.8. MÓDULO 8: CARRITO Y COMPRA DIGITAL....9
1.5.9. MÓDULO 9: COMPRA PRESENCIAL Y PUNTO DE VENTA....9
1.5.10. MÓDULO 10: PASARELA DE PAGOS ELECTRÓNICOS....10
1.5.11. MÓDULO 11: GESTIÓN DE INVENTARIO Y MOVIMIENTOS....10
1.5.12. MÓDULO 12: INTELIGENCIA ARTIFICIAL — RECOMENDACIÓN Y ASISTENCIA....10
1.5.13. MÓDULO 13: REPORTES Y DASHBOARDS....11
1.5.14. MÓDULO 14: PLATAFORMA SAAS Y GESTIÓN DE EMPRESAS (TENANTS)....11
1.5.15. MÓDULO 15: PLANES Y SUSCRIPCIONES....12
1.5.16. MÓDULO 16: OPERACIÓN OFFLINE (PWA Y MÓVIL)....12
1.5.17. MÓDULO 17: SINCRONIZACIÓN Y CONFLICTOS....12
1.6. Metodología de desarrollo ICONIX....13
CAPÍTULO II. ANÁLISIS DE REQUISITOS....15
2.1. Descripción del sistema propuesto....15
2.2. Identificación de actores....16
2.3. Requisitos funcionales y no funcionales....17
2.4. Reglas de negocio....20
2.5. Diagrama UML de casos de uso....21
2.6. Especificación de casos de uso....25
Casos de uso del primer parcial....25
Casos de uso modificados....48
CU2. Iniciar sesión (modificado)....48
CU4. Gestionar usuarios y roles (modificado)....48
CU5. Gestionar sucursales (modificado)....49
CU18. Registrar venta presencial (modificado)....49
CU20. Actualizar inventario automáticamente (modificado)....49
CU25. Consultar reportes y dashboards (modificado)....49
Casos de uso nuevos....50

CU27. Gestionar empresas (tenants)....50
CU28. Gestionar planes de suscripción....50
CU29. Gestionar suscripción de empresa....50
CU30. Verificar límites del plan....51
CU31. Preparar dispositivo para uso offline....51
CU32. Consultar catálogo sin conexión....52
CU33. Registrar reserva sin conexión....52
CU34. Registrar venta de contado sin conexión....53
CU35. Consultar operaciones pendientes....53
CU36. Sincronizar operaciones pendientes....53
CU37. Gestionar conflictos y operaciones rechazadas....54
CU38. Gestionar cambio de prendas....55
CU39. Solicitar reporte generativo (texto o voz)....55
CU40. Exportar reporte (PDF / Excel)....56
CU41. Consultar dashboard analítico....56
CU42. Entrenar y evaluar modelos de ML....57
2.7. Paquetes de casos de uso....57
2.7.1. Descripción de los paquetes del primer parcial....57
2.7.2. Paquetes nuevos del 2do parcial....59
2.8. Matriz de trazabilidad....60
CAPÍTULO III. ANÁLISIS Y DISEÑO CON ICONIX....61
3.1. Modelo de dominio....61
3.2. Diagramas de robustez....61
3.3. Diagramas de secuencia UML....63
3.4. Diagrama de clases UML....63
3.5. Diseño de la base de datos....64
3.5.1. Mapeo....64
3.5.2. Diseño de datos físico....65
3.5.2.1. Tabla de volumen....65
3.5.2.2. Script....68
3.5.3. Cambios del 2do parcial al script PostgreSQL....70
3.5.4. Base de datos local SQLite (app Flutter)....73
3.5.5. Almacenamiento local IndexedDB (PWA Angular)....74
3.5.6. Diagrama de estados de la operación offline....75
3.6. Diseño de interfaces de usuario....75
3.6.1. Prototipos del primer parcial....76
CICLO#1....76
CICLO#2....79
CICLO #3....82
CICLO #4....85
CAPÍTULO IV. DESARROLLO E IMPLEMENTACIÓN....0
4.1. Herramientas y tecnologías utilizadas....0
4.2. Diseño de la arquitectura del sistema....0
4.3. Implementación de la base de datos....0

4.4. Desarrollo de los módulos del sistema....0
4.5. Integración de componentes....0
CAPÍTULO V. PRUEBAS Y RESULTADOS....0
5.1. Plan de pruebas....0
5.2. Pruebas funcionales....0
5.3. Pruebas de Unidad....0
5.4. Pruebas de Integración....0
5.5 Pruebas de Aceptación....0
Proceso de Registro de Cliente....0
Proceso de Gestión de Catálogo....0
5.6. Casos de prueba nuevos (2do parcial)....0
5.7. Pruebas de unidad nuevas....0
5.8. Corrección de errores....0
5.9. Resultados de las pruebas....0
CAPÍTULO VI. CONCLUSIONES Y RECOMENDACIONES....0
6.1. Conclusiones....0
6.2. Recomendaciones....0
BIBLIOGRAFÍA....0
ANEXOS....0
Anexo A. Diagramas del primer parcial (PUDS)....0
A.1. Diagramas de paquetes....0
2.1.2. Relacionar paquetes y casos de uso....0
2.1.3. Vista de paquetes....0
A.2. Diagramas de comunicación....0
Ciclo #1....0
CICLO #2....0
CICLO #3....0
CICLO #4....0
A.3. Diagramas de análisis de una clase....0
Ciclo #1....0
Ciclo #2....0
CICLO #3....0
CICLO #4....0
A.4. Análisis de paquetes....0
Análisis Funcional de las Dependencias....0
A.5. Diseño de arquitectura....0
3.1.1. Diseño físico (Diagrama de despliegue)....0
3.1.2. Diseño Lógico (Diagrama de Paquete)....0
3.1.3. Diagrama de Red....0
A.6. Diagramas de secuencia....0
Ciclo #1....0
Ciclo #2....0
CICLO #3....0
CICLO #4....0

A.7. Diagramas de estado, de tiempo y de navegación....0
Diagrama de Estado....0
Proceso Catálogo....0
Proceso Venta....0
Proceso Compra....0
Proceso Ficha de Pago....0
Diagrama de Tiempo....0
Proceso Catálogo....0
Proceso Venta....0
Proceso Compra....0
Proceso Ficha de Pago....0
Diagrama de Navegación....0
Proceso Catálogo....0
Proceso Venta....0
Proceso Compra....0
Proceso Ficha de Pago....0
A.8. Diagrama de clases....0
A.9. Diagramas de componentes....0
4.2.1. Diagrama de Componente Principal del Sistema....0
4.3.1. Diagrama de Componente de cada Paquete....0
Paquete 1: Gestión de Identidad y Acceso....0
Paquete 2: Gestión de Catálogo y Sucursales....0
Paquete 3: Gestión de Operaciones de Compra....0
Paquete 4: Inteligencia de Recomendación y Vestidor Virtual....0
Paquete 5: Pagos y Notificaciones....0

CAPÍTULO I. ASPECTOS GENERALES

1.1. Introducción

El comercio minorista de indumentaria enfrenta una transformación acelerada en la que la experiencia de compra ya no puede entenderse como dos canales independientes tienda física y tienda digital sino como un único recorrido continuo del cliente. Las cadenas de tiendas de ropa que operan en varias ciudades y sucursales requieren, además, sincronizar información de inventario, temporadas, colecciones y proveedores en tiempo real para evitar decisiones comerciales basadas en datos desactualizados o fragmentados por local.

El presente documento la empresa responsable decidió comercializar FashionStore como un servicio en la nube bajo el modelo SaaS (Software as a Service). Diferentes empresas de prendas de vestir podrán contratar la plataforma mediante un plan de suscripción (Básico o Profesional). Cada empresa contratante (tenant) dispone de su propio entorno lógico para administrar sus sucursales, empleados, clientes, productos, inventarios, reservas y ventas, sin acceder a la información de otras empresas. El aislamiento se implementa sobre una base de datos PostgreSQL compartida, identificando cada registro con la empresa a la que pertenece y validando esa pertenencia en el backend.

Como muchas tiendas sufren cortes de Internet, la plataforma incorpora operación sin conexión. La aplicación web evoluciona a una Progressive Web App (PWA) con Service Worker e IndexedDB, y la aplicación móvil incorpora SQLite. Ambas permiten consultar el catálogo descargado y registrar reservas y ventas de contado sin conexión; luego sincronizan las operaciones pendientes con el servidor, evitan duplicados y detectan conflictos de inventario.

Además, se incorporan reportes generativos con inteligencia artificial: un usuario autorizado escribe o dicta en lenguaje natural lo que necesita (por ejemplo, "ventas de septiembre agrupadas por sucursal") y el sistema interpreta la solicitud, ejecuta consultas controladas sobre los datos reales de su empresa y presenta el resultado en tablas o gráficos exportables a PDF o Excel. Se complementa con un módulo de analítica basado en Machine Learning (modelos supervisados y no supervisados) y un dashboard analítico.

Por la magnitud de estos cambios, el desarrollo pasa del Proceso Unificado (PUDS) a la metodología ICONIX, dirigida por casos de uso, que mantiene una trazabilidad explícita entre requisitos, casos de uso, diagramas de robustez, diagramas de secuencia, clases, código y pruebas. Las capacidades de realidad aumentada y comercio electrónico del primer parcial se conservan.

1.2. Planteamiento del problema

FashionStore fue diseñada para una sola organización, por lo que presenta cuatro limitaciones frente al nuevo modelo de negocio:

- No admite varias empresas: el modelo de datos no distingue a qué empresa pertenece cada sucursal, producto o venta. Si dos empresas usaran el sistema, una podría ver o modificar la información de la otra.
- No hay forma de comercializar el servicio: no existen planes, suscripciones ni límites de uso (por ejemplo, número de sucursales) que regulen qué puede hacer cada empresa contratante.
- Depende totalmente de Internet: si una sucursal pierde la conexión, el cajero no puede registrar ventas ni el encargado puede registrar reservas, lo que detiene la atención al cliente. Tampoco existe un mecanismo para enviar después las operaciones hechas sin conexión ni para resolver diferencias de inventario.
- Los reportes son fijos: los reportes y dashboards existentes están predefinidos; un usuario que necesita un análisis distinto (otro periodo, otras sucursales, otra agrupación) depende de que se programe un reporte nuevo.

1.3. Objetivos del proyecto

1.3.1. Objetivo general

Evolucionar la plataforma FashionStore hacia una plataforma de comercio electrónico SaaS multitenant, incorporando planes de suscripción, aplicación web progresiva, operación offline en web y móvil, sincronización de datos y reportes generativos mediante inteligencia artificial, aplicando la metodología ICONIX y garantizando la coherencia entre los modelos UML 2.5+ y la implementación.

1.3.2. Objetivos específicos

- Aplicar ICONIX al análisis, diseño y evolución del sistema, con trazabilidad entre requisitos, casos de uso, modelos y pruebas.
- Transformar FashionStore en una plataforma multitenant que permita administrar múltiples empresas independientes.
- Implementar planes de suscripción que regulen el acceso a funcionalidades y recursos.
- Evolucionar la aplicación Angular hacia una Progressive Web App (PWA) instalable.
- Incorporar persistencia local con IndexedDB y operaciones offline en la aplicación web.
- Incorporar SQLite como mecanismo de persistencia local en la aplicación Flutter.
- Permitir el registro de reservas y ventas de contado sin conectividad a Internet.
- Implementar la sincronización de operaciones pendientes entre las aplicaciones cliente y el backend, sin duplicados.
- Detectar y gestionar los conflictos derivados de operaciones offline.

- Incorporar reportes generativos mediante inteligencia artificial y lenguaje natural escrito o por voz.
- Desarrollar un módulo de analítica con modelos de Machine Learning supervisados y no supervisados, y un dashboard analítico.
- Garantizar el aislamiento y la seguridad de la información entre empresas en el backend.
- Implementar pruebas funcionales, de integración y de reglas críticas de negocio, automatizadas.
- Actualizar la documentación técnica y los modelos UML 2.5+ para reflejar fielmente el software desarrollado.

1.4. Justificación

- Económica: el modelo SaaS permite vender la misma plataforma a muchas tiendas de ropa con un costo de operación compartido; las tiendas, a su vez, acceden a un sistema completo pagando una suscripción en lugar de financiar un desarrollo propio.
- Operativa: la operación offline evita que un corte de Internet detenga las ventas y reservas en sucursal, algo frecuente en el contexto local.
- De gestión: los reportes generativos reducen la dependencia del área técnica para obtener información; el gerente pide el reporte en lenguaje natural y lo obtiene con datos reales.
- Metodológica: ICONIX es más liviano que PUDS y se concentra en los requisitos nuevos y en los componentes que cambian, manteniendo la trazabilidad necesaria para verificar que cada requisito está implementado y probado.

1.5. Alcance

El sistema cubrirá la gestión integral del comercio electrónico de una cadena de tiendas de ropa con múltiples sucursales y ciudades, integrando catálogo digital, reservas de prendas, vestidores virtuales mediante realidad aumentada, compra presencial y digital, pasarela de pagos, gestión de inventario, proveedores, temporadas y colecciones, inteligencia artificial de recomendación, y reportes operativos. Queda explícitamente fuera de alcance el uso de frameworks de comercio electrónico preexistentes (PrestaShop, Shopify, Magento, WooCommerce o similares); toda la lógica de negocio se construye de manera propia sobre FastAPI, Angular y Flutter, con despliegue final en la nube.

1.5.1. MÓDULO 1: GESTIÓN DE USUARIOS, ROLES Y AUTENTICACIÓN

Objetivo: Permitir el registro, autenticación y control de acceso de los usuarios del sistema según su rol (cliente, administrador, encargado de sucursal, cajero o proveedor).

Casos de Uso:

- Registrar usuario

- Iniciar sesión
- Cerrar sesión
- Consultar y editar perfil
- Asignar y validar rol de acceso

1.5.2. MÓDULO 2: GESTIÓN DE SUCURSALES Y CIUDADES

Objetivo: Administrar la información de las sucursales físicas de la cadena y las ciudades en las que opera, como base para la disponibilidad y el inventario por local.

Casos de Uso:

- Registrar sucursal
- Editar sucursal
- Consultar sucursales por ciudad
- Asignar encargado a sucursal

1.5.3. MÓDULO 3: GESTIÓN DE CATÁLOGO DE PRODUCTOS

Objetivo: Mantener actualizado el catálogo de prendas de vestir, incluyendo categorías, tallas, colores, temporadas y colecciones.

Casos de Uso:

- Registrar producto
- Editar producto
- Gestionar categorías, tallas y colores
- Gestionar temporadas y colecciones
- Asociar producto a proveedor y temporada

1.5.4. MÓDULO 4: GESTIÓN DE PROVEEDORES

Objetivo: Administrar la información de los proveedores de la empresa y su relación con los productos, temporadas y colecciones que suministran.

Casos de Uso:

- Registrar proveedor
- Editar proveedor
- Asociar productos a proveedor
- Consultar histórico de suministro

1.5.5. MÓDULO 5: CONSULTA DE CATÁLOGO Y DISPONIBILIDAD

Objetivo: Permitir al cliente consultar el catálogo de prendas y su disponibilidad específica por sucursal desde la aplicación web y móvil.

Casos de Uso:

- Consultar catálogo (web y móvil)
- Buscar y filtrar prendas
- Consultar tallas y colores disponibles
- Consultar disponibilidad por sucursal

1.5.6. MÓDULO 6: VESTIDOR VIRTUAL (REALIDAD AUMENTADA)

Objetivo: Permitir al cliente visualizar mediante la cámara de su dispositivo móvil una aproximación visual de cómo luciría una prenda seleccionada del catálogo. Alcance delimitado a un Producto Mínimo Viable de superposición visual, sin simulación física de tela ni modelado 3D completo.

Casos de Uso:

- Activar vestidor virtual
- Seleccionar prenda del catálogo para probar virtualmente
- Visualizar superposición de la prenda en tiempo real

- Volver al catálogo o continuar hacia la reserva/compra

1.5.7. MÓDULO 7: RESERVA DE PRENDAS

Objetivo: Permitir al cliente reservar varias prendas indicando la sucursal y un horario aproximado para probarlas físicamente antes de decidir su compra.

Casos de Uso:

- Seleccionar prendas para reservar
- Elegir sucursal y horario de atención
- Confirmar reserva
- Consultar estado de la reserva
- Cancelar reserva
- Notificar la reserva a la sucursal
- Preparar prendas reservadas (encargado de sucursal)

1.5.8. MÓDULO 8: CARRITO Y COMPRA DIGITAL

Objetivo: Permitir al cliente añadir productos a un carrito de compras y finalizar la compra digital desde la aplicación web o móvil.

Casos de Uso:

- Agregar producto al carrito
- Editar o vaciar el carrito
- Confirmar compra desde la plataforma web
- Confirmar compra desde la aplicación móvil

1.5.9. MÓDULO 9: COMPRA PRESENCIAL Y PUNTO DE VENTA

Objetivo: Permitir que el cajero registre ventas presenciales una vez que el cliente decide qué prendas comprar tras probárselas en la sucursal.

Casos de Uso:

- Registrar venta presencial
- Procesar pago en el punto de caja
- Emitir comprobante de venta
- Actualizar inventario tras la venta presencial

1.5.10. MÓDULO 10: PASARELA DE PAGOS ELECTRÓNICOS

Objetivo: Permitir que los clientes realicen pagos digitales por sus compras en línea mediante una pasarela de pago en modo de prueba.

Casos de Uso:

- Seleccionar método de pago digital
- Iniciar transacción mediante pasarela
- Confirmar o rechazar la transacción
- Actualizar el estado del pago
- Generar comprobante de pago digital

1.5.11. MÓDULO 11: GESTIÓN DE INVENTARIO Y MOVIMIENTOS

Objetivo: Mantener el control de existencias de cada sucursal y registrar los movimientos derivados de reservas, ventas, devoluciones y recepción de productos.

Casos de Uso:

- Actualizar inventario tras reserva o venta
- Registrar ingreso de productos
- Consultar existencias consolidadas por sucursal
- Consultar productos agotados o próximos a ingresar

1.5.12. MÓDULO 12: INTELIGENCIA ARTIFICIAL — RECOMENDACIÓN Y ASISTENCIA

Objetivo: Proporcionar funcionalidades inteligentes que apoyen al cliente en la elección de productos y, de forma extendida según el tiempo disponible, a la administración en la generación de reportes.

Casos de Uso:

- Analizar preferencias e historial del cliente
- Recomendar prendas según temporada, categoría, talla y disponibilidad
- Consultar asistente virtual
- (Extensión opcional) Generar reportes mediante comando de voz

Cambios en el 2do parcial: se agregan los reportes generativos. El usuario escribe o dicta su solicitud en lenguaje natural; el sistema interpreta el tipo de reporte y sus filtros (fechas, sucursales, categorías, productos), ejecuta consultas controladas por el backend y presenta el resultado en tabla o gráfico, con exportación a PDF y Excel. Maneja solicitudes ambiguas o no soportadas. Disponible solo en el plan Profesional.

1.5.13. MÓDULO 13: REPORTES Y DASHBOARDS

Objetivo: Brindar a los administradores una vista analítica de ventas, reservas e inventario para apoyar la toma de decisiones.

Casos de Uso:

- Consultar reporte de ventas
- Consultar reporte de inventario
- Visualizar indicadores por sucursal y ciudad
- Filtrar información por temporada o categoría

Cambios en el 2do parcial: se agrega la analítica inteligente: generación de datos sintéticos, entrenamiento y evaluación de un modelo supervisado (predicción de ventas o demanda) y

uno no supervisado (segmentación de clientes o productos), y un dashboard analítico con indicadores, predicciones y métricas.

1.5.14. MÓDULO 14: PLATAFORMA SAAS Y GESTIÓN DE EMPRESAS (TENANTS).

Registro de empresas contratantes, administrador por empresa, asociación de usuarios y sucursales, aislamiento de datos por empresa en el backend. Mínimo dos empresas de prueba.

1.5.15. MÓDULO 15: PLANES Y SUSCRIPCIONES.

Planes Básico (hasta 2 sucursales, sin reportes con IA) y Profesional (hasta 10 sucursales, con reportes con IA), con límites configurables desde el backend; asignación y cambio de plan, fechas de inicio y vencimiento, estado de la suscripción y restricción de operaciones según el plan. La contratación y renovación se simulan administrativamente (sin cobro real).

1.5.16. MÓDULO 16: OPERACIÓN OFFLINE (PWA Y MÓVIL).

PWA instalable con Service Worker, Web App Manifest e IndexedDB; app Flutter con SQLite. Descarga de un subconjunto del catálogo, consulta de productos, precios, tallas, colores y existencias conocidas con aviso de información posiblemente desactualizada, registro local de reservas y ventas de contado, consulta de operaciones pendientes y conservación de los datos al cerrar la aplicación.

1.5.17. MÓDULO 17: SINCRONIZACIÓN Y CONFLICTOS.

Envío de operaciones pendientes al recuperar conexión, identificador único por operación (UUID), estados de sincronización (registrada localmente, enviada, confirmada, rechazada, en conflicto), reintentos, prevención de duplicados (idempotencia), validación de inventario

en el servidor y registro de resultados para diagnóstico. Incluye la gestión de cambios de prendas dentro del plazo definido por la empresa.

Limitaciones:

- Las ventas offline son solo de contado; el pago electrónico requiere conexión.
- No se implementa cobro recurrente real ni integración con plataformas de facturación SaaS.
- La IA no accede directamente a la base de datos: solo propone parámetros de consulta que el backend valida.
- No se desarrolla un nuevo vestidor virtual; se conserva el del primer parcial.
- Los modelos de Machine Learning se entrenan con datos sintéticos con fines académicos.

1.6. Metodología de desarrollo ICONIX

ICONIX es una metodología de desarrollo de software ubicada entre RUP (PUDS) y XP. Deriva de RUP, pero parte de que el 80% de los casos puede resolverse usando solo el 20% de UML, por lo que se limita a los diagramas necesarios sin perder documentación. Está dirigida por casos de uso, es iterativa e incremental, y su principal característica es la trazabilidad: cada requisito se asocia a un caso de uso, cada caso de uso a su diagrama de robustez y de secuencia, y estos a las clases, el código y las pruebas. Así puede verificarse en cualquier momento que cada requisito está satisfecho.

ICONIX se organiza en cuatro fases, precedidas por un storyboard de las pantallas principales:

1. **Análisis de requisitos:** modelo de dominio (diagrama de clases simplificado con los objetos del mundo real), prototipo de interfaces refinado con el cliente, diagramas de casos de uso agrupados en paquetes y asociación de cada requisito con un caso de uso.
2. **Análisis y diseño preliminar:** fichas de caso de uso (nombre, descripción, precondición, postcondición, flujo normal y flujo alternativo) y diagramas de

robustez, que usan objetos frontera (interfaces), control (lógica) y entidad (datos). Con ellos se actualiza el modelo de dominio.

3. **Diseño:** diagramas de secuencia derivados de cada ficha y de su diagrama de robustez, y refinamiento final del diagrama de clases con atributos y métodos.
4. **Implementación:** diagramas de componentes y despliegue cuando hacen falta, escritura del código según el diseño y pruebas planificadas a partir de los requisitos.

En el diagrama de robustez se respetan tres reglas: el actor solo se comunica con objetos frontera; los objetos frontera y entidad se comunican solo a través de controladores; y los controladores pueden comunicarse entre sí.

Aplicación en este proyecto: como el enunciado lo indica, ICONIX se aplica principalmente a los requisitos nuevos (SaaS, suscripciones, offline, sincronización, reportes generativos, analítica) y a los casos de uso existentes que se modifican. Los casos de uso sin cambios conservan su documentación del primer parcial.

UML en ICONIX: se usan diagramas de casos de uso y de paquetes (requisitos), diagramas de robustez (análisis; no son UML estándar pero se construyen con estereotipos UML), diagramas de secuencia y de clases (diseño), y diagramas de estados, componentes y despliegue como apoyo.

Marco teórico complementario:

- **SaaS (Software as a Service):** modelo en el que el software se aloja en la nube y se ofrece como servicio mediante suscripción; el cliente no instala ni mantiene servidores.
- **Multitenancy:** una misma instancia del sistema atiende a varios clientes (tenants). En este proyecto se usa una base de datos compartida con aislamiento lógico: cada tabla

de negocio tiene una columna `empresa_id` y el backend filtra toda consulta por la empresa del usuario autenticado.

- **Progressive Web App (PWA):** aplicación web que puede instalarse y funcionar sin conexión gracias al Service Worker (intercepta peticiones y guarda recursos en caché), al Web App Manifest (nombre, íconos, modo de pantalla) y a IndexedDB (base de datos del navegador).
- **Offline-first y sincronización:** la aplicación guarda primero cada operación localmente y luego la envía al servidor. Cada operación lleva un UUID generado en el dispositivo; si se reenvía, el servidor la reconoce y no la duplica (idempotencia).
- **Reportes generativos (lenguaje natural a consulta):** un modelo de lenguaje convierte la solicitud del usuario en una estructura de parámetros (tipo de reporte, métricas, filtros, agrupación). El backend valida esa estructura contra un catálogo de reportes permitidos y ejecuta consultas parametrizadas, nunca SQL libre generado por la IA.
- **Machine Learning:** aprendizaje supervisado (el modelo aprende de datos con una variable objetivo conocida, por ejemplo regresión para predecir ventas) y no supervisado (encuentra grupos o patrones sin etiquetas, por ejemplo K-Means para segmentar clientes).

CAPÍTULO II. ANÁLISIS DE REQUISITOS

2.1. Descripción del sistema propuesto

FashionStore SaaS es una plataforma en la nube que varias empresas de ropa contratan mediante un plan de suscripción. El Administrador de la plataforma registra cada empresa y le asigna un plan y un administrador. Cada empresa opera de forma aislada: gestiona sus

sucursales, empleados, catálogo, inventario, clientes, reservas y ventas, y solo ve sus propios datos.

En las sucursales, el cajero y el encargado pueden seguir trabajando sin Internet: la PWA (navegador) y la app Flutter guardan el catálogo y registran reservas y ventas de contado localmente. Al volver la conexión, las operaciones se envían al servidor, que las confirma, las rechaza o las marca en conflicto si el inventario real no alcanza. Los usuarios autorizados de empresas con plan Profesional pueden pedir reportes en lenguaje natural, y la plataforma ofrece un dashboard analítico con modelos de Machine Learning.

2.2. Identificación de actores

Actor	Tipo	Descripción	Cambio respecto al 1er parcial
Administrador de plataforma	Humano, interno	Personal de FashionStore SaaS. Registra empresas, gestiona planes y suscripciones, entrena y supervisa los modelos de ML. No opera datos comerciales de las empresas.	Nuevo
Administrador de empresa	Humano	Administrador de una empresa contratante (tenant). Gestiona usuarios, roles, sucursales, catálogo, proveedores, inventario y reportes solo de su empresa; consulta su suscripción.	Antes "Administrador"; ahora limitado a su empresa
Encargado de sucursal	Humano	Gestiona la operación de su sucursal: reservas, inventario local, reservas offline, sincronización y conflictos.	Agrega operación offline
Cajero	Humano	Atiende el punto de venta: ventas presenciales en línea, ventas de contado offline, cambios de prendas y sincronización.	Agrega operación offline y cambios
Cliente	Humano	Cliente de una empresa: navega su catálogo, usa el vestidor virtual, reserva y compra.	Pertenece a una empresa
Proveedor	Externo	Suministra información de productos a una empresa.	Pertenece a una empresa
Sistema de pagos	Sistema externo	Pasarela (Stripe sandbox) que procesa pagos electrónicos.	Sin cambio
Servicio de IA	Sistema externo	Modelo de lenguaje por API: recomienda prendas,	Agrega interpretación de reportes

		responde el asistente e interpreta solicitudes de reportes en lenguaje natural.	
Sistema (temporizador)	Interno	Dispara procesos automáticos: actualización de inventario, sincronización al detectar conexión y vencimiento de suscripciones.	Ampliado

2.3. Requisitos funcionales y no funcionales

Requisitos funcionales heredados del 1er parcial (se conservan; se listan para la trazabilidad):

Código	Requisito
RF01	Registrar clientes y autenticar usuarios con roles
RF02	Gestionar usuarios y roles de la empresa
RF03	Gestionar sucursales y consultarlas por ciudad
RF04	Gestionar productos, categorías, tallas, colores, temporadas y colecciones
RF05	Gestionar proveedores
RF06	Consultar el catálogo y la disponibilidad por sucursal
RF07	Probar prendas con el vestidor virtual (AR)
RF08	Reservar prendas y atender reservas en sucursal
RF09	Gestionar el carrito y confirmar compras digitales
RF10	Registrar ventas presenciales en línea
RF11	Procesar pagos electrónicos
RF12	Actualizar inventario y registrar movimientos
RF13	Recomendar prendas y responder el asistente virtual con IA
RF14	Consultar reportes convencionales, dashboards e historial de compras

Código	Requisito	Desafío
RF15	Registrar empresas contratantes con su administrador	VI
RF16	Asociar usuarios y sucursales a una empresa	VI
RF17	Limitar toda consulta y operación a la empresa del usuario autenticado, validado en el backend	VI
RF18	Gestionar planes con límites y funcionalidades configurables	VI
RF19	Asignar o cambiar el plan de una empresa con fecha de inicio y vencimiento	VI
RF20	Consultar el estado de la suscripción	VI
RF21	Restringir funcionalidades y límites según el plan y el estado de la suscripción	VI
RF22	Instalar la aplicación web como PWA y preparar el dispositivo para uso offline	II
RF23	Descargar y guardar localmente productos, variantes, sucursales autorizadas y clientes mínimos	II, III
RF24	Consultar productos, precios, tallas, colores y existencias sin conexión, avisando si la información puede estar desactualizada	II, III
RF25	Registrar reservas de prendas sin conexión	II, III
RF26	Registrar ventas de contado sin conexión con identificador, empresa, sucursal, usuario, cliente, fecha, productos, cantidades, precios, total, forma de pago y estado	II, III, IV
RF27	Consultar las reservas y ventas locales pendientes y su estado	II, III
RF28	Conservar los datos locales tras cerrar y reabrir la aplicación	II, III
RF29	Sincronizar las operaciones pendientes al recuperar la conexión, en forma automática o manual	II, III
RF30	Identificar cada operación con UUID, empresa y sucursal, y evitar duplicados al reenviarla	III
RF31	Reintentar el envío ante fallos sin borrar la operación local	III
RF32	Validar el inventario en el servidor y confirmar, rechazar o marcar en conflicto cada operación	IV
RF33	Consultar y resolver operaciones rechazadas o en	III, IV

	conflicto	
RF34	Registrar el resultado de cada sincronización para diagnóstico	III
RF35	Gestionar cambios de prendas dentro del plazo definido por la empresa	IV
RF36	Recibir solicitudes de reporte en lenguaje natural escrito o por voz	V
RF37	Interpretar el tipo de reporte y sus filtros (fechas, sucursales, categorías, productos)	V
RF38	Ejecutar consultas controladas sobre los datos reales de la empresa y presentarlas en tabla o gráfico	V
RF39	Manejar solicitudes ambiguas, incompletas o no soportadas	V
RF40	Exportar el reporte a PDF o Excel	V
RF41	Generar datos sintéticos y entrenar y evaluar un modelo supervisado y uno no supervisado	V
RF42	Mostrar un dashboard analítico con predicciones, patrones detectados y métricas	V

Requisitos no funcionales:

Código	Requisito	Cómo se cumple
RNF01	Seguridad	JWT con empresa, rol y sucursal; contraseñas con bcrypt; autorización por rol en cada endpoint
RNF02	Aislamiento de datos	Columna empresa_id en las tablas de negocio; dependencia de FastAPI que filtra toda consulta por la empresa del token; un recurso de otra empresa responde 404
RNF03	Persistencia offline	IndexedDB en la PWA y SQLite en Flutter; las operaciones se guardan antes de intentar enviarlas
RNF04	Integridad transaccional	La venta, su detalle y el movimiento de inventario se guardan en una sola transacción PostgreSQL
RNF05	Idempotencia	Restricción UNIQUE sobre uuid_cliente; un reenvío devuelve la respuesta original
RNF06	Recuperación ante fallos	La operación local solo cambia de estado con una respuesta del servidor; ante un error queda pendiente y se reintenta

RNF07	Mantenibilidad	Capas separadas: presentación, servicios de negocio, repositorios de persistencia y servicio de sincronización
RNF08	Compatibilidad	PWA en navegadores modernos (Chrome, Edge, Firefox, Safari); Flutter en Android
RNF09	Rendimiento	Índices por empresa_id; paginación del catálogo; descarga solo del catálogo de la sucursal
RNF10	Observabilidad	Tabla sync_log con UUID, resultado, código de error y fecha, sin datos sensibles
RNF11	Calidad	Pruebas automatizadas con pytest (backend), Jasmine/Karma (Angular) y flutter_test
RNF12	Coherencia documental	Matriz de trazabilidad RF, CU, robustez, secuencia y prueba

2.4. Reglas de negocio

Código	Regla
RN01	Toda entidad de negocio pertenece a una sola empresa. Un usuario solo consulta y modifica datos de su empresa; el Administrador de plataforma no opera datos comerciales.
RN02	Una sucursal pertenece a una sola empresa; una sucursal no es un tenant.
RN03	Cada empresa tiene al menos un Administrador de empresa activo.
RN04	Plan Básico: hasta 2 sucursales activas. Plan Profesional: hasta 10. Superar el límite se rechaza. Los valores se configuran en la tabla de planes.
RN05	Los reportes generativos con IA solo están disponibles en el plan Profesional.
RN06	Con la suscripción vencida o suspendida, la empresa solo puede iniciar sesión y consultar; no registra operaciones nuevas.
RN07	Las ventas offline son solo de contado (efectivo). La confirmación local del efectivo no implica que el servidor acepte la venta.
RN08	Cada operación offline lleva un UUID generado en el dispositivo, la empresa, la sucursal, el usuario y la fecha y hora local.
RN09	Una operación local nunca se elimina por haberse enviado; solo cambia de estado al recibir la respuesta del servidor.
RN10	Si al sincronizar una venta no hay stock suficiente en el servidor, la venta queda EN_CONFLICTO y no descuenta inventario hasta que el encargado la resuelva.
RN11	Si llega una operación con un UUID ya confirmado, el servidor devuelve la confirmación original y no crea registros nuevos.

RN12	La venta offline se registra con el precio local conocido al momento de la venta.
RN13	El catálogo local muestra la fecha de su última actualización; si tiene más de 24 horas, se advierte que puede estar desactualizado.
RN14	Un cambio de prenda se acepta dentro del plazo configurado por la empresa (7 días por defecto), con el comprobante de la venta y la prenda en buen estado; la prenda nueva debe tener stock.
RN15	La IA no ejecuta SQL ni accede a la base de datos: solo devuelve parámetros que el backend valida contra un catálogo de reportes permitidos, siempre filtrando por la empresa del usuario.

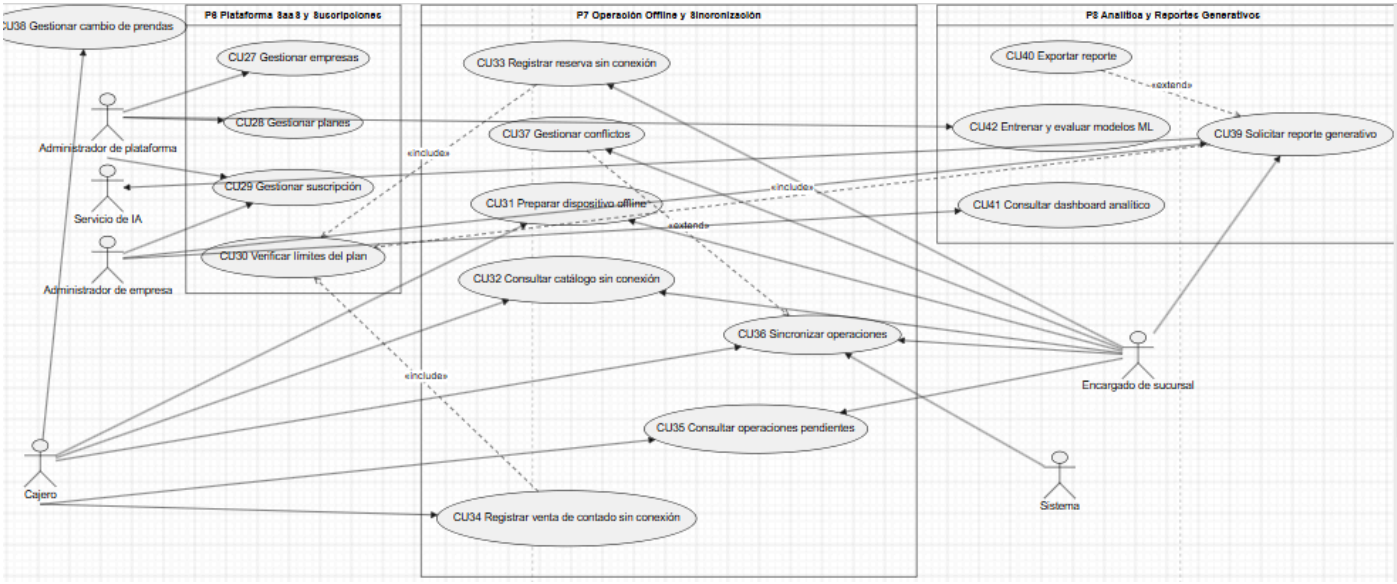
2.5. Diagrama UML de casos de uso

Código	Caso de uso	Paquete	Actor principal
CU1	Gestionar empresas (tenants)	P6 Plataforma SaaS	Administrador de plataforma
CU2	Gestionar planes de suscripción	P6 Plataforma SaaS	Administrador de plataforma
CU3	Gestionar suscripción de empresa	P6 Plataforma SaaS	Administrador de plataforma / Administrador de empresa (consulta)
CU4	Verificar límites del plan	P6 Plataforma SaaS	Sistema (incluido en CU5, CU33, CU34, CU39)
CU5	Preparar dispositivo para uso offline	P7 Offline y sincronización	Cajero / Encargado
CU6	Consultar catálogo sin conexión	P7 Offline y sincronización	Cajero / Encargado

CU7	Registrar reserva sin conexión	P7 Offline y sincronización	Encargado
CU8	Registrar venta de contado sin conexión	P7 Offline y sincronización	Cajero
CU9	Consultar operaciones pendientes	P7 Offline y sincronización	Cajero / Encargado
CU10	Sincronizar operaciones pendientes	P7 Offline y sincronización	Cajero / Encargado / Sistema
CU11	Gestionar conflictos y operaciones rechazadas	P7 Offline y sincronización	Encargado
CU12	Gestionar cambio de prendas	P3 Operaciones de compra	Cajero
CU13	Solicitar reporte generativo (texto o voz)	P8 Analítica e IA	Administrador de empresa / Encargado
CU14	Exportar reporte (PDF / Excel)	P8 Analítica e IA	Administrador de empresa / Encargado
CU15	Consultar dashboard analítico	P8 Analítica e IA	Administrador de empresa

CU16	Entrenar y evaluar modelos de ML	P8 Analítica e IA	Administrador de plataforma

Diagrama general de casos de uso



2.6. Especificación de casos de uso

CU1. Gestionar empresas (tenants)

- **Descripción:** el Administrador de plataforma registra una empresa contratante con su administrador y su plan inicial, y puede activarla o desactivarla.
- **Actores:** Administrador de plataforma.
- **Precondición:** sesión iniciada como Administrador de plataforma.
- **Flujo normal:**
  - El administrador elige "Nueva empresa".
  - El sistema muestra el formulario.
  - El administrador ingresa razón social, NIT, correo, teléfono, identificador (slug) y los datos del administrador de la empresa.
  - El administrador selecciona el plan inicial.
  - El sistema valida que el NIT y el slug no existan.
  - El sistema crea la empresa, su suscripción (CU29) y el usuario ADMIN_EMPRESA en una transacción.

    - El sistema muestra la empresa registrada.
- **Flujos alternativos:**
    - 5a. NIT o slug duplicado: el sistema lo informa y no registra.
    - 6a. Error al crear cualquiera de los tres registros: la transacción se revierte.
    - 1a. Desactivar empresa: el sistema cambia su estado a INACTIVA y sus usuarios ya no pueden iniciar sesión.
- **Postcondición:** la empresa existe con un administrador y una suscripción activa.

CU2. Gestionar planes de suscripción

- **Descripción:** el Administrador de plataforma define los planes y sus límites y funcionalidades.
- **Actores:** Administrador de plataforma.
- **Precondición:** sesión iniciada como Administrador de plataforma.
- **Flujo normal:**
    - El administrador abre la gestión de planes.
    - El sistema lista los planes (Básico, Profesional).
    - El administrador crea o edita un plan: nombre, precio mensual referencial, máximo de sucursales y funcionalidades habilitadas (reportes con IA, operación offline, etc.).
    - El sistema valida que los límites sean números positivos.
    - El sistema guarda el plan.
- **Flujos alternativos:**
    - 4a. Valor inválido: el sistema lo señala y no guarda.
    - 5a. Se reduce el límite por debajo de lo que usa una empresa: el sistema advierte; las sucursales existentes se mantienen pero no se pueden crear nuevas.
- **Postcondición:** los nuevos límites se aplican en la siguiente verificación (CU30).

CU3. Gestionar suscripción de empresa

- **Descripción:** permite asignar un plan, registrar fechas de inicio y vencimiento, cambiar de plan, renovar y consultar el estado de la suscripción.
- **Actores:** Administrador de plataforma (gestiona); Administrador de empresa (consulta).
- **Precondición:** la empresa está registrada.
- **Flujo normal:**
    - El administrador de plataforma selecciona una empresa.
    - El sistema muestra la suscripción vigente: plan, inicio, vencimiento, estado y uso actual (sucursales activas).
    - El administrador elige asignar, cambiar o renovar el plan e indica las fechas.
    - El sistema valida que el uso actual quepa en el nuevo plan.
    - El sistema cierra la suscripción anterior y registra la nueva como ACTIVA.
    - El sistema muestra el nuevo estado.
- **Flujos alternativos:**
    - 4a. El uso actual supera el nuevo plan (por ejemplo, 4 sucursales al bajar a Básico): el sistema rechaza el cambio y lo explica.

    - 2a. El Administrador de empresa solo consulta su suscripción, sin modificarla.
    - Sistema: al llegar la fecha de vencimiento, el estado pasa a VENCIDA (RN06).
- **Postcondición:** la empresa tiene una sola suscripción vigente.

CU4. Verificar límites del plan

- **Descripción:** caso de uso incluido que comprueba, antes de una operación, que la suscripción esté activa y que el plan permita la funcionalidad o el recurso.
- **Actores:** Sistema. Incluido en CU5, CU33, CU34 y CU39.
- **Precondición:** usuario autenticado con empresa.
- **Flujo normal:**
    - El sistema obtiene la empresa del token.
    - El sistema consulta la suscripción vigente y su plan.
    - El sistema verifica que la suscripción esté ACTIVA.
    - El sistema verifica el límite o la funcionalidad solicitada.
    - El sistema permite continuar.
- **Flujos alternativos:**
    - 3a. Suscripción vencida o suspendida: responde 403 "Suscripción no activa".
    - 4a. Límite alcanzado o función no incluida: responde 403 con el motivo.
- **Postcondición:** la operación continúa solo si el plan la permite.

CU5. Preparar dispositivo para uso offline

- **Descripción:** el empleado instala la PWA (o usa la app Flutter) y descarga a su dispositivo el catálogo y los datos necesarios de su sucursal.
- **Actores:** Cajero, Encargado.
- **Precondición:** sesión iniciada con conexión.
- **Flujo normal:**
    - El empleado elige "Instalar aplicación" (PWA) o abre la app móvil.
    - El sistema registra el Service Worker y guarda los recursos de la aplicación.
    - El empleado elige "Descargar catálogo".
    - El sistema solicita al backend los productos, variantes (talla y color), precios y existencias de su sucursal, las sucursales autorizadas y los clientes mínimos.
    - El sistema guarda los datos en IndexedDB (PWA) o SQLite (móvil) con la fecha de descarga.
    - El sistema muestra "Catálogo actualizado al dd/mm hh:mm".
- **Flujos alternativos:**
    - 1a. El navegador no admite instalación: la PWA funciona igual desde el navegador.
    - 4a. Se pierde la conexión durante la descarga: se conserva la versión anterior y se informa.
- **Postcondición:** el dispositivo puede operar sin conexión con datos de la empresa y sucursal del usuario.

CU6. Consultar catálogo sin conexión

- **Descripción:** el empleado consulta productos, precios, tallas, colores y existencias guardados localmente.
- **Actores:** Cajero, Encargado.
- **Precondición:** catálogo descargado (CU31).
- **Flujo normal:**
    - El sistema detecta que no hay conexión y muestra el indicador "Sin conexión".
    - El empleado busca por nombre o filtra por categoría, talla o color.
    - El sistema consulta la base local.
    - El sistema muestra los resultados con la fecha de la última actualización.
- **Flujos alternativos:**
    - 4a. Los datos tienen más de 24 horas: el sistema muestra "La información puede estar desactualizada" (RN13).
    - 3a. Sin catálogo local: el sistema indica que debe descargarse con conexión.
- **Postcondición:** ninguna (solo consulta).

CU7. Registrar reserva sin conexión

- **Descripción:** el encargado registra localmente una reserva de prendas para un cliente.
- **Actores:** Encargado. Incluye CU30 (con el plan guardado en la sesión).
- **Precondición:** catálogo local disponible.
- **Flujo normal:**
    - El encargado elige "Nueva reserva".
    - El encargado selecciona o registra el cliente (datos mínimos).
    - El encargado agrega prendas (producto, talla, color, cantidad).
    - El sistema verifica la existencia local conocida.
    - El sistema genera un UUID y guarda la reserva con empresa, sucursal, usuario, fecha y hora, estado PENDIENTE y su registro en la cola de sincronización.
    - El sistema muestra la reserva como "Pendiente de sincronizar".
- **Flujos alternativos:**
    - 4a. Existencia local insuficiente: el sistema advierte; el encargado puede reducir la cantidad.
    - 5a. Error al guardar localmente: el sistema no confirma la reserva e informa.
- **Postcondición:** la reserva queda guardada localmente y sobrevive al cierre de la aplicación.

CU8. Registrar venta de contado sin conexión

- **Descripción:** el cajero registra localmente una venta pagada en efectivo.
- **Actores:** Cajero. Incluye CU30.
- **Precondición:** catálogo local disponible; sin conexión.
- **Flujo normal:**
    - El cajero elige "Nueva venta".
    - El cajero agrega productos y variantes con sus cantidades.
    - El sistema calcula el total con los precios locales (RN12).
    - El cajero identifica al cliente si corresponde.

    - El cajero registra la recepción del efectivo (forma de pago: EFECTIVO).
    - El sistema genera un UUID y guarda la venta, su detalle y la operación pendiente con empresa, sucursal, usuario y fecha y hora.
    - El sistema descuenta la existencia local y emite un comprobante provisional.
    - El sistema muestra la venta como "Pendiente de confirmación".
- **Flujos alternativos:**
    - 2a. Existencia local insuficiente: el sistema advierte y permite continuar o quitar el producto.
    - 5a. Se elige un pago electrónico: el sistema lo impide sin conexión (RN07).
    - 6a. Error al guardar: la venta no se registra y se informa.
- **Postcondición:** la venta queda guardada localmente en estado PENDIENTE; no está aceptada por el servidor.

CU9. Consultar operaciones pendientes

- **Descripción:** el empleado consulta las reservas y ventas registradas localmente y su estado de sincronización.
- **Actores:** Cajero, Encargado.
- **Precondición:** existen operaciones locales.
- **Flujo normal:**
    - El empleado abre "Operaciones pendientes".
    - El sistema lee la cola local de operaciones.
    - El sistema muestra cada operación con tipo, fecha, total y estado: REGISTRADA_LOCAL, ENVIADA, CONFIRMADA, RECHAZADA o EN_CONFLICTO.
    - El empleado abre una operación para ver su detalle y el resultado de la sincronización.
- **Flujos alternativos:**
    - 2a. No hay operaciones: el sistema muestra "No hay operaciones pendientes".
- **Postcondición:** ninguna (solo consulta).

CU10. Sincronizar operaciones pendientes

- **Descripción:** el dispositivo envía al servidor las operaciones registradas sin conexión y actualiza su estado según la respuesta.
- **Actores:** Cajero, Encargado (manual); Sistema (automático al detectar conexión).
- **Precondición:** hay conexión y operaciones en estado REGISTRADA_LOCAL o con error de envío.
- **Flujo normal:**
    - El sistema detecta la conexión o el empleado pulsa "Sincronizar".
    - El sistema toma las operaciones pendientes en orden de registro y las marca como ENVIADA.
    - El sistema envía el lote al endpoint de sincronización con el token del usuario.
    - El servidor valida que la empresa y la sucursal de cada operación coincidan con el token.

    - El servidor busca el UUID: si no existe, registra la operación y, si es venta, valida y descuenta el inventario en una transacción (CU20).
    - El servidor guarda el resultado en el log de sincronización y responde por cada UUID: CONFIRMADA, RECHAZADA o EN_CONFLICTO con su motivo.
    - El dispositivo actualiza el estado de cada operación local y descarga existencias actualizadas.
    - El sistema muestra el resumen: confirmadas, rechazadas y en conflicto.
- **Flujos alternativos:**
    - 3a. Falla la comunicación o el servidor responde con error: las operaciones vuelven a REGISTRADA_LOCAL, se incrementa el contador de intentos y se reintenta después (RN09).
    - 5a. El UUID ya existe: el servidor devuelve la respuesta original sin crear registros (RN11).
    - 5b. Stock insuficiente: la venta se guarda EN_CONFLICTO sin descontar inventario (RN10).
    - 4a. Empresa o sucursal no coinciden, o suscripción no activa: la operación se RECHAZA con su motivo.
- **Postcondición:** cada operación enviada tiene un estado definitivo o sigue pendiente; ninguna se pierde ni se duplica.

CU11. Gestionar conflictos y operaciones rechazadas

- **Descripción:** el encargado revisa las operaciones en conflicto o rechazadas y decide cómo resolverlas.
- **Actores:** Encargado. Extiende CU36.
- **Precondición:** existe al menos una operación EN_CONFLICTO o RECHAZADA.
- **Flujo normal:**
    - El encargado abre "Conflictos".
    - El sistema lista las operaciones con su motivo (por ejemplo, "stock en servidor: 1, vendido: 2").
    - El encargado elige una acción: ajustar cantidades a lo disponible, reemplazar la variante, registrar un ajuste de inventario justificado o anular la operación con devolución al cliente.
    - El sistema aplica la acción en una transacción y registra quién resolvió, cuándo y por qué.
    - La operación pasa a CONFIRMADA o ANULADA.
- **Flujos alternativos:**
    - 3a. El encargado no tiene permisos sobre esa sucursal: el sistema lo impide.
    - 4a. El stock cambió de nuevo: el sistema vuelve a mostrar el conflicto actualizado.
- **Postcondición:** la operación queda resuelta y el inventario es coherente.

CU12. Gestionar cambio de prendas

- **Descripción:** el cajero registra el cambio de una prenda vendida por otra (otra talla, color o modelo) dentro del plazo permitido.
- **Actores:** Cajero.
- **Precondición:** la venta original existe y está CONFIRMADA.

- **Flujo normal:**
    - El cajero busca la venta por código de comprobante.
    - El sistema verifica que la fecha esté dentro del plazo de cambio de la empresa (RN14).
    - El cajero selecciona la prenda devuelta y la prenda nueva.
    - El sistema verifica el stock de la prenda nueva y calcula la diferencia de precio.
    - El cajero cobra la diferencia si corresponde.
    - El sistema registra el cambio y los movimientos de inventario (entrada de la devuelta, salida de la nueva) en una transacción.
    - El sistema emite el comprobante de cambio.
- **Flujos alternativos:**
    - 2a. Fuera de plazo: el sistema rechaza el cambio.
    - 4a. Sin stock de la prenda nueva: el sistema ofrece otra variante o reservarla.
    - 4b. La prenda nueva es más barata: el sistema registra un saldo a favor (no se devuelve efectivo).
- **Postcondición:** el inventario refleja el cambio y queda trazabilidad con la venta original.

CU13. Solicitar reporte generativo (texto o voz)

- **Descripción:** un usuario autorizado pide un reporte en lenguaje natural; el sistema lo interpreta, consulta los datos reales de su empresa y lo presenta en tabla o gráfico.
- **Actores:** Administrador de empresa, Encargado; Servicio de IA (secundario). Incluye CU30.
- **Precondición:** sesión iniciada; plan Profesional con suscripción activa.
- **Flujo normal:**
    - El usuario escribe o dicta su solicitud (por ejemplo, "ventas de septiembre agrupadas por sucursal").
    - Si es por voz, el sistema la convierte a texto.
    - El sistema verifica el plan (CU30).
    - El sistema envía el texto al Servicio de IA con el catálogo de reportes permitidos y sus parámetros.
    - El Servicio de IA devuelve una estructura JSON: tipo de reporte, métricas, filtros (fechas, sucursales, categorías, productos), agrupación y tipo de gráfico.
    - El backend valida la estructura contra el catálogo permitido y agrega el filtro de empresa (y de sucursal si es Encargado).
    - El backend ejecuta la consulta parametrizada correspondiente.
    - El sistema muestra el reporte en tabla y gráfico con título, filtros aplicados y una breve narrativa.
- **Flujos alternativos:**
    - 3a. Plan Básico: el sistema informa que la función requiere el plan Profesional.
    - 5a. Solicitud ambigua o incompleta (por ejemplo, sin periodo): el sistema pregunta el dato faltante o aplica un valor por defecto e indica cuál.

    - 6a. Solicitud no soportada o no permitida (modificar datos, datos de otra empresa): el sistema la rechaza y sugiere ejemplos válidos.
    - 7a. Sin datos para los filtros: el sistema muestra "No hay datos para el periodo seleccionado".
- **Postcondición:** el reporte se muestra y la solicitud queda registrada en el historial de reportes.

Tipos de reporte soportados (mínimo 4): ventas por periodo agrupadas (sucursal, categoría, semana, mes); productos más vendidos (top N); comparación de sucursales en un rango de meses; productos con menor existencia; ingresos por categoría.

CU14. Exportar reporte (PDF / Excel)

- **Descripción:** el usuario descarga el reporte generado con formato convencional.
- **Actores:** Administrador de empresa, Encargado. Extiende CU39 y CU25.
- **Precondición:** hay un reporte en pantalla.
- **Flujo normal:**
    - El usuario elige "Exportar" y el formato (PDF o Excel).
    - El sistema arma el documento con encabezado de la empresa, título, filtros, fecha de generación, tabla y gráfico.
    - El sistema descarga el archivo.
- **Flujos alternativos:**
    - 2a. Error al generar: el sistema informa y permite reintentar.
- **Postcondición:** el usuario tiene el archivo del reporte.

CU15. Consultar dashboard analítico

- **Descripción:** el administrador de empresa visualiza indicadores, predicciones y patrones obtenidos por los modelos de ML.
- **Actores:** Administrador de empresa.
- **Precondición:** hay modelos entrenados (CU42).
- **Flujo normal:**
    - El administrador abre el dashboard analítico.
    - El sistema muestra indicadores (ventas, ticket promedio, productos top).
    - El sistema muestra la predicción de ventas o demanda del modelo supervisado.
    - El sistema muestra los segmentos de clientes o productos del modelo no supervisado.
    - El sistema muestra las métricas de evaluación de cada modelo.
    - El administrador filtra por sucursal o periodo.
- **Flujos alternativos:**
    - 1a. Sin modelos entrenados: el sistema muestra solo los indicadores.
- **Postcondición:** ninguna (solo consulta).

CU15. Entrenar y evaluar modelos de ML

- **Descripción:** el Administrador de plataforma genera datos sintéticos, entrena los modelos y revisa sus métricas antes de publicarlos.
- **Actores:** Administrador de plataforma.

- **Precondición:** sesión de Administrador de plataforma.
- **Flujo normal:**
    - El administrador elige "Generar datos sintéticos" e indica el volumen.
    - El sistema genera el conjunto de datos y lo preprocesa (limpieza, selección de variables, codificación y escalado).
    - El administrador elige entrenar el modelo supervisado y el no supervisado.
    - El sistema divide los datos en entrenamiento y prueba, entrena y calcula métricas (por ejemplo, MAE y R^2 para regresión; coeficiente de silueta para clustering).
    - El administrador revisa las métricas y publica el modelo.
    - El sistema guarda el modelo con su versión y fecha.
- **Flujos alternativos:**
    - 5a. Métricas insuficientes: el administrador ajusta parámetros y vuelve al paso 3.
- **Postcondición:** hay un modelo publicado disponible para CU41.

2.7. Paquetes de casos de uso

2.7.1. Descripción de los paquetes del primer parcial

Paquete Gestión de Identidad y Acceso

Paquete	Descripción de los Escenarios	Resumen
Gestión de Identidad y Acceso (paquete principal)	Centraliza las operaciones relacionadas con la identidad de todos los actores del sistema: clientes y personal interno. Integra registro, autenticación, gestión de perfil y administración de roles.	Garantiza que solo usuarios válidos accedan a la plataforma y que cada rol tenga los privilegios correctos.
Autenticación (subpaquete)	Registro de nuevas cuentas de cliente, verificación de credenciales e inicio de sesión, con generación de sesión según el rol del usuario.	Asegura que solo usuarios registrados y activos puedan ingresar al sistema.
Perfil y Roles (subpaquete)	Consulta y actualización del perfil personal del cliente, y administración de usuarios internos y sus roles por parte del administrador.	Mantiene actualizada la información de cada usuario y controla qué puede hacer cada rol.

Paquete Gestión de Catálogo y Sucursales

Paquete	Descripción de los Escenarios	Resumen

Gestión de Catálogo y Sucursales (paquete principal)	Administra el catálogo de productos, sus proveedores, y la red de sucursales físicas de la cadena, junto con la consulta pública de ambos por parte del cliente.	Es la base de información comercial y geográfica que sostiene toda la operación de venta.
Catálogo Base (subpaquete)	Registro y mantenimiento de productos, categorías, tallas, colores, temporadas, colecciones y proveedores.	Mantiene actualizado qué se vende y quién lo suministra.
Sucursales y Disponibilidad (subpaquete)	Registro de sucursales y ciudades, y consulta pública del catálogo y su disponibilidad real por sucursal.	Conecta al cliente con la ubicación física y la existencia real de cada prenda.

Paquete Gestión de Operaciones de Compra

Paquete	Descripción de los Escenarios	Resumen
Gestión de Operaciones de Compra (paquete principal)	Orquesta el recorrido del cliente desde que prueba una prenda (física o virtualmente) hasta que confirma su compra, por cualquiera de los dos canales.	Es el núcleo transaccional de la plataforma, donde se concreta la venta.
Reserva y Prueba (subpaquete)	Vestidor virtual por realidad aumentada, reserva de prendas para probarlas en sucursal, y atención de esa reserva por parte del encargado.	Reduce la incertidumbre del cliente antes de comprar.
Checkout (subpaquete)	Carrito de compras, confirmación de compra digital y registro de venta presencial en caja.	Cierra el ciclo de venta, sin importar el canal elegido por el cliente.

Paquete Pagos e Inventario

Paquete	Descripción de los Escenarios	Resumen
Pagos e Inventario (paquete principal)	Procesa las transacciones electrónicas y mantiene sincronizado el inventario de productos entre todas las sucursales.	Garantiza que cada venta se cobre correctamente y que el stock mostrado al cliente sea siempre real.
Pasarela Transaccional (subpaquete)	Procesamiento de pagos electrónicos mediante pasarela en modo sandbox, confirmación o rechazo de la transacción.	Habilita el cobro digital de forma segura, sin exponer datos de tarjeta al comercio.
Control de Inventario (subpaquete)	Actualización automática del inventario tras reservas o ventas, y registro manual de movimientos (ingresos, mermas).	Mantiene el stock exacto por producto y sucursal en todo momento.

Paquete Inteligencia y Analítica

Paquete	Descripción de los Escenarios	Resumen
Inteligencia y Analítica (paquete principal)	Aplica inteligencia artificial para asistir al cliente en su elección de productos, y consolida indicadores operativos para la administración.	Aporta valor añadido sobre los datos que genera el resto del sistema, sin operar directamente el negocio.
IA de Recomendación (subpaquete)	Análisis de preferencias del cliente, recomendación de prendas y asistente conversacional; de forma extendida, generación de reportes por comando de voz.	Personaliza la experiencia de compra del cliente.
Reportes Operativos (subpaquete)	Consulta de reportes de ventas e inventario, indicadores por sucursal y ciudad, e historial de compras del cliente.	Da visibilidad a la administración sobre el desempeño real del negocio.

Insertar aquí los diagramas de paquetes actualizados

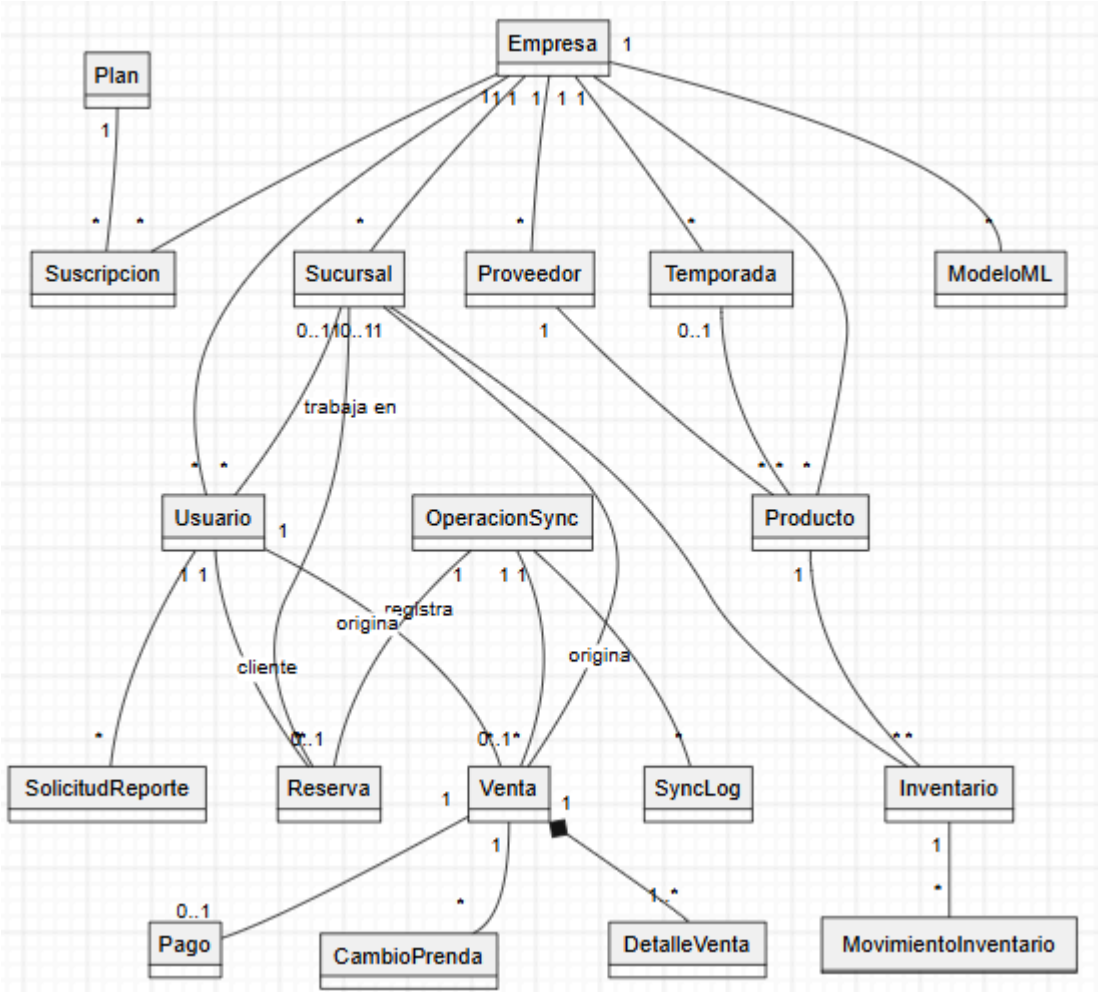
CAPÍTULO III. ANÁLISIS Y DISEÑO CON ICONIX

Los diagramas de comunicación, de análisis de una clase, de secuencia, de estado, de tiempo, de navegación, de arquitectura y de componentes del primer parcial se trasladaron al Anexo A. En este capítulo se presentan los modelos de ICONIX para los requisitos nuevos y los casos de uso modificados.

3.1. Modelo de dominio

El modelo de dominio muestra los objetos del mundo real que el sistema debe almacenar, sin métodos ni detalles de implementación. La empresa (tenant) es el centro: todo lo comercial depende de ella. Las clases nuevas respecto al 1er parcial son Plan, Empresa, Suscripción, OperacionSync, SyncLog, CambioPrenda, MovimientoInventario, SolicitudReporte y ModeloML.

Relaciones principales: un Plan tiene muchas Suscripciones; una Empresa tiene muchas Suscripciones, Sucursales, Usuarios, Proveedores, Temporadas y Productos; un Producto tiene inventario en varias Sucursales; una Venta se compone de uno o más DetalleVenta y puede tener un Pago y varios CambioPrenda; una OperacionSync origina una Venta o una Reserva y registra varios SyncLog.



3.2. Diagramas de robustez

3.3. Diagramas de secuencia UML

3.4. Diagrama de clases UML

3.5. Diseño de la base de datos

3.5.1. Mapeo

Para cada entidad se indica su clave primaria, sus claves foráneas y el resto de sus atributos, como base para el diseño físico de la sección 1.2.2.

usuarios

PK	FK	Otros campos
id	—	full_name, email, phone, password_hash, role, is_active, created_at, updated_at

sucursales

PK	FK	Otros campos
id	—	nombre, ciudad, direccion, telefono, latitud, longitud, is_active, created_at

temporadas

PK	FK	Otros campos
id	—	nombre, anio, descripcion, fecha_inicio, fecha_fin, is_active

proveedores

PK	FK	Otros campos

id	—	razon_social, nit, telefono, correo, direccion, is_active

productos

PK	FK	Otros campos
id	temporada_id, proveedor_id	nombre, descripcion, categoria, precio, is_active, created_at

inventario

PK	FK	Otros campos
id	producto_id, sucursal_id	talla, color, cantidad_disponible

carrito_items

PK	FK	Otros campos
id	cliente_id, producto_id	talla, color, cantidad, fecha_agregado

reservas

PK	FK	Otros campos
id	cliente_id, producto_id, sucursal_id	cantidad, estado, codigo, fecha_reserva, fecha_expiracion

pedidos

PK	FK	Otros campos
id	cliente_id, sucursal_id	tipo, estado, total, created_at

detalle_pedido

PK	FK	Otros campos
id	pedido_id, producto_id	talla, color, cantidad, precio_unitario

pagos

PK	FK	Otros campos
id	pedido_id	monto, metodo, estado, stripe_payment_id, created_at, paid_at

notificaciones

PK	FK	Otros campos
id	usuario_id	titulo, mensaje, is_read, created_at

3.5.2. Diseño de datos físico

3.5.2.1. Tabla de volumen

usuarios

Campo	PK	FK/UQ	Tipo	Descripción
id	PK		INTEGER	Identificador único del usuario
full_name			VARCHAR(150)	Nombre completo
email		UQ	VARCHAR(120)	Correo electrónico único
phone		UQ	VARCHAR(30)	Teléfono de contacto

password_hash			VARCHAR(255)	Contraseña cifrada con bcrypt
role			VARCHAR(30)	CLIENTE / ADMIN / ENCARGADO_SUCURSAL
is_active			BOOLEAN	Estado de la cuenta
created_at			TIMESTAMP	Fecha de creación
updated_at			TIMESTAMP	Fecha de actualización

Campo	PK	FK/UQ	Tipo	Descripción
id	PK		INTEGER	Identificador de la sucursal
nombre			VARCHAR(120)	Nombre comercial
ciudad			VARCHAR(80)	Ciudad donde opera
direccion			VARCHAR(180)	Dirección física
telefono			VARCHAR(30)	Teléfono de contacto
latitud			DECIMAL(10,6)	Latitud geográfica
longitud			DECIMAL(10,6)	Longitud geográfica
is_active			BOOLEAN	Estado de la sucursal
created_at			TIMESTAMP	Fecha de creación

Campo	PK	FK/UQ	Tipo	Descripción
id	PK		INTEGER	Identificador de la temporada
nombre			VARCHAR(100)	Nombre de la temporada/colección
anio			INTEGER	Año de la temporada
descripcion			TEXT	Descripción de la colección
fecha_inicio			DATE	Inicio de vigencia
fecha_fin			DATE	Fin de vigencia
is_active			BOOLEAN	Estado de la temporada

Campo	PK	FK/UQ	Tipo	Descripción
id	PK		INTEGER	Identificador del proveedor
razon_social			VARCHAR(150)	Nombre legal del proveedor
nit		UQ	VARCHAR(30)	Identificación tributaria
telefono			VARCHAR(30)	Teléfono de contacto
correo			VARCHAR(120)	Correo de contacto
direccion			VARCHAR(180)	Dirección del proveedor
is_active			BOOLEAN	Estado del proveedor

Campo	PK	FK/UQ	Tipo	Descripción
id	PK		INTEGER	Identificador del producto
temporada_id		FK	INTEGER	Temporada/colección a la que pertenece
proveedor_id		FK	INTEGER	Proveedor del producto
nombre			VARCHAR(150)	Nombre de la prenda
descripcion			TEXT	Descripción del producto
categoria			VARCHAR(60)	Categoría (camisas, pantalones, etc.)

precio			NUMERIC(10,2)	Precio de venta
is_active			BOOLEAN	Publicado / despublicado
created_at			TIMESTAMP	Fecha de creación

Campo	PK	FK/UQ	Tipo	Descripción
id	PK		INTEGER	Identificador del registro de stock
producto_id		FK	INTEGER	Producto asociado
sucursal_id		FK	INTEGER	Sucursal asociada
talla			VARCHAR(10)	Talla de la variante
color			VARCHAR(40)	Color de la variante
cantidad_disponible			INTEGER	Existencias disponibles

Campo	PK	FK/UQ	Tipo	Descripción
id	PK		INTEGER	Identificador del ítem de carrito
cliente_id		FK	INTEGER	Cliente propietario del carrito
producto_id		FK	INTEGER	Producto agregado
talla			VARCHAR(10)	Talla seleccionada
color			VARCHAR(40)	Color seleccionado
cantidad			INTEGER	Cantidad seleccionada
fecha_agregado			TIMESTAMP	Fecha en que se agregó

Campo	PK	FK/UQ	Tipo	Descripción
id	PK		INTEGER	Identificador de la reserva
cliente_id		FK	INTEGER	Cliente que reserva
producto_id		FK	INTEGER	Producto reservado
sucursal_id		FK	INTEGER	Sucursal donde se recoge
cantidad			INTEGER	Cantidad reservada
estado			VARCHAR(20)	PENDIENTE / ATENDIDA / CANCELADA / VENCIDA
codigo		UQ	VARCHAR(20)	Código de retiro en sucursal
fecha_reserva			TIMESTAMP	Fecha de creación de la reserva
fecha_expiracion			TIMESTAMP	Vencimiento de la reserva

Campo	PK	FK/UQ	Tipo	Descripción
id	PK		INTEGER	Identificador del pedido
cliente_id		FK	INTEGER	Cliente que compra
sucursal_id		FK (nullable)	INTEGER	Sucursal de recojo (null si es envío digital)
tipo			VARCHAR(20)	DIGITAL / PRESENCIAL
estado			VARCHAR(20)	PENDIENTE / PAGADO / ENVIADO / ENTREGADO / CANCELADO
total			NUMERIC(10,2)	Monto total del pedido
created_at			TIMESTAMP	Fecha de creación

Campo	PK	FK/UQ	Tipo	Descripción
id	PK		INTEGER	Identificador del detalle
pedido_id		FK	INTEGER	Pedido al que pertenece
producto_id		FK	INTEGER	Producto comprado
talla			VARCHAR(10)	Talla comprada
color			VARCHAR(40)	Color comprado
cantidad			INTEGER	Cantidad comprada
precio_unitario			NUMERIC(10,2)	Precio unitario al momento de la compra

Campo	PK	FK/UQ	Tipo	Descripción
id	PK		INTEGER	Identificador del pago
pedido_id		FK/UQ	INTEGER	Pedido asociado (un pago por pedido)
monto			NUMERIC(10,2)	Monto pagado
metodo			VARCHAR(30)	TARJETA / EFECTIVO / TRANSFERENCIA
estado			VARCHAR(20)	PENDIENTE / PROCESANDO / COMPLETADO / FALLIDO
stripe_payment_id			VARCHAR(80)	Referencia de la pasarela Stripe
created_at			TIMESTAMP	Fecha de creación
paid_at			TIMESTAMP	Fecha de confirmación del pago

Campo	PK	FK/UQ	Tipo	Descripción
id	PK		INTEGER	Identificador de la notificación
usuario_id		FK	INTEGER	Usuario destinatario
titulo			VARCHAR(120)	Título de la notificación
mensaje			TEXT	Contenido del mensaje
is_read			BOOLEAN	Indicador de lectura
created_at			TIMESTAMP	Fecha de creación

3.5.2.2. Script

A continuación se presenta el script base en PostgreSQL, coherente con la arquitectura y las 12 entidades definidas para la plataforma.

```
-- ============================================================
-- SCRIPT PostgreSQL
-- FashionStore — Plataforma de E-commerce Multi-Sucursal
-- ============================================================
BEGIN;

CREATE TABLE usuarios (
       id             INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
       full_name      VARCHAR(150) NOT NULL,
       email          VARCHAR(120) NOT NULL UNIQUE,
       phone          VARCHAR(30)  NOT NULL UNIQUE,
       password_hash   VARCHAR(255) NOT NULL,
       role           VARCHAR(30)  NOT NULL CHECK (role IN
('CLIENTE','ADMIN','ENCARGADO_SUCURSAL')),
       is_active      BOOLEAN        NOT NULL DEFAULT TRUE,
       created_at     TIMESTAMP      NOT NULL DEFAULT NOW(),
```

```
       updated_at     TIMESTAMP      NOT NULL DEFAULT NOW()
);

CREATE TABLE sucursales (
       id             INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
       nombre         VARCHAR(120) NOT NULL,
       ciudad         VARCHAR(80)  NOT NULL,
       direccion      VARCHAR(180) NOT NULL,
       telefono       VARCHAR(30),
       latitud        DECIMAL(10,6),
       longitud       DECIMAL(10,6),
       is_active      BOOLEAN        NOT NULL DEFAULT TRUE,
       created_at     TIMESTAMP      NOT NULL DEFAULT NOW()
);

CREATE TABLE temporadas (
       id             INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
       nombre         VARCHAR(100) NOT NULL,
       anio           INTEGER NOT NULL,
       descripcion    TEXT,
       fecha_inicio   DATE,
       fecha_fin      DATE,
       is_active      BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE proveedores (
       id             INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
       razon_social   VARCHAR(150) NOT NULL,
       nit            VARCHAR(30)  NOT NULL UNIQUE,
       telefono       VARCHAR(30),
       correo         VARCHAR(120),
       direccion      VARCHAR(180),
       is_active      BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE productos (
       id             INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
       temporada_id   INTEGER REFERENCES temporadas(id) ON UPDATE CASCADE ON DELETE SET
NULL,
    proveedor_id      INTEGER REFERENCES proveedores(id) ON UPDATE CASCADE ON DELETE
RESTRICT,
       nombre         VARCHAR(150) NOT NULL,
       descripcion    TEXT,
       categoria      VARCHAR(60),
       precio         NUMERIC(10,2) NOT NULL CHECK (precio >= 0),
       is_active      BOOLEAN NOT NULL DEFAULT TRUE,
       created_at     TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE inventario (
       id                    INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
       producto_id           INTEGER NOT NULL REFERENCES productos(id) ON UPDATE CASCADE ON
DELETE CASCADE,
       sucursal_id           INTEGER NOT NULL REFERENCES sucursales(id) ON UPDATE CASCADE
ON DELETE CASCADE,
       talla                 VARCHAR(10) NOT NULL,
       color                 VARCHAR(40) NOT NULL,
       cantidad_disponible  INTEGER NOT NULL DEFAULT 0 CHECK (cantidad_disponible >= 0),
       UNIQUE (producto_id, sucursal_id, talla, color)
);

CREATE TABLE carrito_items (
       id             INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
       cliente_id     INTEGER NOT NULL REFERENCES usuarios(id) ON UPDATE CASCADE ON DELETE
CASCADE,
       producto_id    INTEGER NOT NULL REFERENCES productos(id) ON UPDATE CASCADE ON DELETE
CASCADE,
```

```
        talla          VARCHAR(10) NOT NULL,
        color          VARCHAR(40) NOT NULL,
        cantidad       INTEGER NOT NULL CHECK (cantidad > 0),
        fecha_agregado  TIMESTAMP NOT NULL DEFAULT NOW(),
        UNIQUE (cliente_id, producto_id, talla, color)
);

CREATE TABLE reservas (
        id             INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        cliente_id     INTEGER NOT NULL REFERENCES usuarios(id) ON UPDATE CASCADE ON DELETE
RESTRICT,
        producto_id    INTEGER NOT NULL REFERENCES productos(id) ON UPDATE CASCADE ON DELETE
RESTRICT,
        sucursal_id    INTEGER NOT NULL REFERENCES sucursales(id) ON UPDATE CASCADE ON
DELETE RESTRICT,
        cantidad       INTEGER NOT NULL CHECK (cantidad > 0),
        estado         VARCHAR(20) NOT NULL CHECK (estado IN
('PENDIENTE','ATENDIDA','CANCELADA','VENCIDA')),
        codigo              VARCHAR(20) NOT NULL UNIQUE,
        fecha_reserva  TIMESTAMP NOT NULL DEFAULT NOW(),
        fecha_expiracion  TIMESTAMP NOT NULL
);

CREATE TABLE pedidos (
        id             INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        cliente_id     INTEGER NOT NULL REFERENCES usuarios(id) ON UPDATE CASCADE ON DELETE
RESTRICT,
        sucursal_id    INTEGER REFERENCES sucursales(id) ON UPDATE CASCADE ON DELETE SET
NULL,
        tipo           VARCHAR(20) NOT NULL CHECK (tipo IN ('DIGITAL','PRESENCIAL')),
        estado         VARCHAR(20) NOT NULL CHECK (estado IN
('PENDIENTE','PAGADO','ENVIADO','ENTREGADO','CANCELADO')),
        total          NUMERIC(10,2) NOT NULL CHECK (total >= 0),
        created_at     TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE detalle_pedido (
        id             INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        pedido_id      INTEGER NOT NULL REFERENCES pedidos(id) ON UPDATE CASCADE ON DELETE
CASCADE,
        producto_id    INTEGER NOT NULL REFERENCES productos(id) ON UPDATE CASCADE ON DELETE
RESTRICT,
        talla          VARCHAR(10) NOT NULL,
        color          VARCHAR(40) NOT NULL,
        cantidad       INTEGER NOT NULL CHECK (cantidad > 0),
        precio_unitario   NUMERIC(10,2) NOT NULL CHECK (precio_unitario >= 0)
);

CREATE TABLE pagos (
        id                    INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        pedido_id             INTEGER NOT NULL UNIQUE REFERENCES pedidos(id) ON UPDATE
CASCADE ON DELETE RESTRICT,
        monto                 NUMERIC(10,2) NOT NULL CHECK (monto >= 0),
        metodo                VARCHAR(30) NOT NULL CHECK (metodo IN
('TARJETA','EFECTIVO','TRANSFERENCIA')),
        estado                VARCHAR(20) NOT NULL CHECK (estado IN
('PENDIENTE','PROCESANDO','COMPLETADO','FALLIDO')),
        stripe_payment_id   VARCHAR(80),
        created_at            TIMESTAMP NOT NULL DEFAULT NOW(),
        paid_at               TIMESTAMP
);

CREATE TABLE notificaciones (
        id             INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        usuario_id     INTEGER NOT NULL REFERENCES usuarios(id) ON UPDATE CASCADE ON DELETE
CASCADE,
        titulo         VARCHAR(120) NOT NULL,
```

```
        mensaje        TEXT NOT NULL,
        is_read        BOOLEAN NOT NULL DEFAULT FALSE,
        created_at     TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_productos_temporada_id   ON productos(temporada_id);
CREATE INDEX idx_productos_proveedor_id   ON productos(proveedor_id);
CREATE INDEX idx_inventario_producto_id   ON inventario(producto_id);
CREATE INDEX idx_inventario_sucursal_id   ON inventario(sucursal_id);
CREATE INDEX idx_carrito_cliente_id       ON carrito_items(cliente_id);
CREATE INDEX idx_reservas_cliente_id      ON reservas(cliente_id);
CREATE INDEX idx_reservas_sucursal_id     ON reservas(sucursal_id);
CREATE INDEX idx_pedidos_cliente_id       ON pedidos(cliente_id);
CREATE INDEX idx_detalle_pedido_pedido_id ON detalle_pedido(pedido_id);
CREATE INDEX idx_notificaciones_usuario   ON notificaciones(usuario_id);

COMMIT;
```

3.5.3. Cambios del 2do parcial al script PostgreSQL

Se agrega al final del script del primer parcial. El aislamiento es lógico: una sola base de datos, columna empresa_id en cada tabla de negocio y filtro obligatorio en el backend. La tabla de volumen se amplía con las tablas nuevas (planes, empresas, suscripciones, detalle_reserva, movimientos_inventario, cambios_prenda, operaciones_sync, sync_log, solicitudes_reporte y modelos_ml) y con la columna empresa_id en las existentes.

```
-- FashionStore SaaS: cambios del 2do parcial (multitenant, offline e IA)
BEGIN;
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- 1. Plataforma SaaS
CREATE TABLE planes (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(40) NOT NULL UNIQUE,
    precio_mensual NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (precio_mensual >= 0),
    max_sucursales INTEGER NOT NULL CHECK (max_sucursales > 0),
    reportes_ia BOOLEAN NOT NULL DEFAULT FALSE,
    operacion_offline BOOLEAN NOT NULL DEFAULT TRUE,
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);
CREATE TABLE empresas (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    razon_social VARCHAR(150) NOT NULL,
    nit VARCHAR(30) NOT NULL UNIQUE,
    slug VARCHAR(60) NOT NULL UNIQUE,
    correo VARCHAR(120) NOT NULL,
    telefono VARCHAR(30),
    plazo_cambio_dias INTEGER NOT NULL DEFAULT 7 CHECK (plazo_cambio_dias >= 0),
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVA' CHECK (estado IN
('ACTIVA','INACTIVA')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE TABLE suscripciones (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    empresa_id INTEGER NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    plan_id INTEGER NOT NULL REFERENCES planes(id),
    fecha_inicio DATE NOT NULL,
    fecha_vencimiento DATE NOT NULL,
    estado VARCHAR(20) NOT NULL CHECK (estado IN
('ACTIVA','VENCIDA','SUSPENDIDA','REEMPLAZADA')),
```

```
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    CHECK (fecha_vencimiento > fecha_inicio)
);
CREATE UNIQUE INDEX ux_suscripcion_activa ON suscripciones(empresa_id) WHERE estado
= 'ACTIVA';

-- 2. Planes y dos empresas de prueba
INSERT INTO planes (nombre, precio_mensual, max_sucursales, reportes_ia)
VALUES ('BASICO', 0, 2, FALSE), ('PROFESIONAL', 0, 10, TRUE);
INSERT INTO empresas (razon_social, nit, slug, correo)
VALUES ('FashionStore Demo S.R.L.', '1000001', 'fashionstore-demo',
'admin@demo.com'),
       ('Moda Andina S.R.L.', '1000002', 'moda-andina', 'admin@modaandina.com');
INSERT INTO suscripciones (empresa_id, plan_id, fecha_inicio, fecha_vencimiento,
estado)
VALUES (1, 2, CURRENT_DATE, CURRENT_DATE + 365, 'ACTIVA'),
       (2, 1, CURRENT_DATE, CURRENT_DATE + 365, 'ACTIVA');

-- 3. empresa_id en las tablas de negocio existentes
ALTER TABLE usuarios ADD COLUMN empresa_id INTEGER REFERENCES empresas(id);
ALTER TABLE usuarios ADD COLUMN sucursal_id INTEGER REFERENCES sucursales(id);
ALTER TABLE sucursales ADD COLUMN empresa_id INTEGER REFERENCES empresas(id);
ALTER TABLE temporadas ADD COLUMN empresa_id INTEGER REFERENCES empresas(id);
ALTER TABLE proveedores ADD COLUMN empresa_id INTEGER REFERENCES empresas(id);
ALTER TABLE productos ADD COLUMN empresa_id INTEGER REFERENCES empresas(id);
ALTER TABLE inventario ADD COLUMN empresa_id INTEGER REFERENCES empresas(id);
ALTER TABLE carrito_items ADD COLUMN empresa_id INTEGER REFERENCES empresas(id);
ALTER TABLE reservas ADD COLUMN empresa_id INTEGER REFERENCES empresas(id);
ALTER TABLE pedidos ADD COLUMN empresa_id INTEGER REFERENCES empresas(id);
ALTER TABLE pagos ADD COLUMN empresa_id INTEGER REFERENCES empresas(id);
ALTER TABLE notificaciones ADD COLUMN empresa_id INTEGER REFERENCES empresas(id);
-- Los datos del 1er parcial pasan a la empresa 1; luego se aplica NOT NULL
UPDATE sucursales SET empresa_id = 1; -- repetir para cada tabla anterior
ALTER TABLE sucursales ALTER COLUMN empresa_id SET NOT NULL; -- repetir para cada
tabla, excepto usuarios

-- Roles nuevos; solo el administrador de plataforma no tiene empresa
UPDATE usuarios SET role = 'ADMIN_EMPRESA' WHERE role = 'ADMIN';
ALTER TABLE usuarios DROP CONSTRAINT usuarios_role_check;
ALTER TABLE usuarios ADD CONSTRAINT usuarios_role_check
    CHECK (role IN
('ADMIN_PLATAFORMA','ADMIN_EMPRESA','ENCARGADO_SUCURSAL','CAJERO','CLIENTE'));
ALTER TABLE usuarios ADD CONSTRAINT usuarios_empresa_check
    CHECK (role = 'ADMIN_PLATAFORMA' OR empresa_id IS NOT NULL);
ALTER TABLE proveedores DROP CONSTRAINT proveedores_nit_key;
ALTER TABLE proveedores ADD CONSTRAINT ux_proveedor_nit UNIQUE (empresa_id, nit);

-- 4. Campos de operación offline en ventas y reservas
ALTER TABLE pedidos ADD COLUMN usuario_id INTEGER REFERENCES usuarios(id);
ALTER TABLE pedidos ADD COLUMN uuid_cliente UUID UNIQUE;
ALTER TABLE pedidos ADD COLUMN origen VARCHAR(10) NOT NULL DEFAULT 'ONLINE' CHECK
(origen IN ('ONLINE','OFFLINE'));
ALTER TABLE pedidos ADD COLUMN fecha_local TIMESTAMP;
ALTER TABLE reservas ADD COLUMN uuid_cliente UUID UNIQUE;
ALTER TABLE reservas ADD COLUMN origen VARCHAR(10) NOT NULL DEFAULT 'ONLINE' CHECK
(origen IN ('ONLINE','OFFLINE'));
CREATE TABLE detalle_reserva (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    empresa_id INTEGER NOT NULL REFERENCES empresas(id),
    reserva_id INTEGER NOT NULL REFERENCES reservas(id) ON DELETE CASCADE,
    producto_id INTEGER NOT NULL REFERENCES productos(id),
    talla VARCHAR(10) NOT NULL,
    color VARCHAR(40) NOT NULL,
    cantidad INTEGER NOT NULL CHECK (cantidad > 0)
);
ALTER TABLE reservas ALTER COLUMN producto_id DROP NOT NULL;
ALTER TABLE reservas ALTER COLUMN cantidad DROP NOT NULL;
```

```
-- 5. Movimientos de inventario y cambios de prenda
CREATE TABLE movimientos_inventario (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    empresa_id INTEGER NOT NULL REFERENCES empresas(id),
    inventario_id INTEGER NOT NULL REFERENCES inventario(id),
    tipo VARCHAR(20) NOT NULL CHECK (tipo IN
('INGRESO','SALIDA_VENTA','AJUSTE','CAMBIO_ENTRADA','CAMBIO_SALIDA')),
    cantidad INTEGER NOT NULL CHECK (cantidad > 0),
    referencia VARCHAR(60),
    uuid_origen UUID,
    usuario_id INTEGER REFERENCES usuarios(id),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE TABLE cambios_prenda (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    empresa_id INTEGER NOT NULL REFERENCES empresas(id),
    pedido_id INTEGER NOT NULL REFERENCES pedidos(id),
    producto_devuelto_id INTEGER NOT NULL REFERENCES productos(id),
    talla_devuelta VARCHAR(10) NOT NULL,
    color_devuelto VARCHAR(40) NOT NULL,
    producto_nuevo_id INTEGER NOT NULL REFERENCES productos(id),
    talla_nueva VARCHAR(10) NOT NULL,
    color_nuevo VARCHAR(40) NOT NULL,
    diferencia NUMERIC(10,2) NOT NULL DEFAULT 0,
    usuario_id INTEGER NOT NULL REFERENCES usuarios(id),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

-- 6. Sincronización
CREATE TABLE operaciones_sync (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    uuid_cliente UUID NOT NULL UNIQUE,
    empresa_id INTEGER NOT NULL REFERENCES empresas(id),
    sucursal_id INTEGER NOT NULL REFERENCES sucursales(id),
    usuario_id INTEGER NOT NULL REFERENCES usuarios(id),
    tipo VARCHAR(20) NOT NULL CHECK (tipo IN ('VENTA','RESERVA')),
    payload JSONB NOT NULL,
    estado VARCHAR(20) NOT NULL CHECK (estado IN
('CONFIRMADA','RECHAZADA','EN_CONFLICTO','ANULADA')),
    motivo TEXT,
    pedido_id INTEGER REFERENCES pedidos(id),
    reserva_id INTEGER REFERENCES reservas(id),
    fecha_local TIMESTAMP NOT NULL,
    recibida_at TIMESTAMP NOT NULL DEFAULT NOW(),
    resuelta_por INTEGER REFERENCES usuarios(id),
    resuelta_at TIMESTAMP
);
CREATE TABLE sync_log (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    empresa_id INTEGER NOT NULL REFERENCES empresas(id),
    uuid_cliente UUID NOT NULL,
    dispositivo VARCHAR(60),
    resultado VARCHAR(20) NOT NULL,
    codigo_error VARCHAR(40),
    detalle TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

-- 7. Reportes generativos y analítica
CREATE TABLE solicitudes_reporte (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    empresa_id INTEGER NOT NULL REFERENCES empresas(id),
    usuario_id INTEGER NOT NULL REFERENCES usuarios(id),
    texto_original TEXT NOT NULL,
    canal VARCHAR(10) NOT NULL CHECK (canal IN ('TEXTO','VOZ')),
    parametros JSONB,
    tipo_reporte VARCHAR(40),
```

```
    estado VARCHAR(20) NOT NULL CHECK (estado IN
('GENERADO','AMBIGUO','NO_SOPORTADO','RECHAZADO')),
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE TABLE modelos_ml (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    empresa_id INTEGER REFERENCES empresas(id),
    tipo VARCHAR(20) NOT NULL CHECK (tipo IN ('SUPERVISADO','NO_SUPERVISADO')),
    algoritmo VARCHAR(60) NOT NULL,
    version VARCHAR(20) NOT NULL,
    metricas JSONB NOT NULL,
    ruta_archivo VARCHAR(200) NOT NULL,
    publicado BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

-- 8. Índices por tenant
CREATE INDEX ix_sucursales_empresa ON sucursales(empresa_id);
CREATE INDEX ix_productos_empresa ON productos(empresa_id);
CREATE INDEX ix_inventario_empresa ON inventario(empresa_id, sucursal_id);
CREATE INDEX ix_pedidos_empresa ON pedidos(empresa_id, created_at);
CREATE INDEX ix_reservas_empresa ON reservas(empresa_id, sucursal_id);
CREATE INDEX ix_opsync_estado ON operaciones_sync(empresa_id, estado);
CREATE INDEX ix_synclog_uuid ON sync_log(uuid_cliente);
COMMIT;
```

3.5.4. Base de datos local SQLite (app Flutter)

Guarda solo lo necesario para operar en la sucursal del usuario. Las claves locales de ventas y reservas son el UUID, para que coincidan con uuid_cliente del servidor.

```
PRAGMA foreign_keys = ON;
CREATE TABLE metadatos (clave TEXT PRIMARY KEY, valor TEXT NOT NULL);
CREATE TABLE sucursales (id INTEGER PRIMARY KEY, nombre TEXT NOT NULL, ciudad TEXT
NOT NULL);
CREATE TABLE productos (id INTEGER PRIMARY KEY, nombre TEXT NOT NULL, categoria
TEXT,
    precio REAL NOT NULL, imagen_url TEXT, actualizado TEXT NOT NULL);
CREATE TABLE inventario (id INTEGER PRIMARY KEY,
    producto_id INTEGER NOT NULL REFERENCES productos(id),
    sucursal_id INTEGER NOT NULL REFERENCES sucursales(id),
    talla TEXT NOT NULL, color TEXT NOT NULL, cantidad_conocida INTEGER NOT NULL,
    UNIQUE (producto_id, sucursal_id, talla, color));
CREATE TABLE clientes (id INTEGER PRIMARY KEY, uuid_local TEXT UNIQUE,
    nombre TEXT NOT NULL, telefono TEXT, email TEXT);
CREATE TABLE ventas (uuid TEXT PRIMARY KEY,
    sucursal_id INTEGER NOT NULL REFERENCES sucursales(id), usuario_id INTEGER NOT
NULL,
    cliente_ref TEXT, fecha_local TEXT NOT NULL, total REAL NOT NULL,
    forma_pago TEXT NOT NULL DEFAULT 'EFECTIVO', estado_sync TEXT NOT NULL DEFAULT
'REGISTRADA_LOCAL');
CREATE TABLE detalle_ventas (id INTEGER PRIMARY KEY AUTOINCREMENT,
    venta_uuid TEXT NOT NULL REFERENCES ventas(uuid) ON DELETE CASCADE,
    producto_id INTEGER NOT NULL REFERENCES productos(id),
    talla TEXT NOT NULL, color TEXT NOT NULL,
    cantidad INTEGER NOT NULL CHECK (cantidad > 0), precio_unitario REAL NOT NULL);
CREATE TABLE reservas (uuid TEXT PRIMARY KEY,
    sucursal_id INTEGER NOT NULL REFERENCES sucursales(id), usuario_id INTEGER NOT
NULL,
    cliente_ref TEXT NOT NULL, fecha_local TEXT NOT NULL, fecha_expiracion TEXT NOT
NULL,
    estado_sync TEXT NOT NULL DEFAULT 'REGISTRADA_LOCAL');
CREATE TABLE detalle_reservas (id INTEGER PRIMARY KEY AUTOINCREMENT,
    reserva_uuid TEXT NOT NULL REFERENCES reservas(uuid) ON DELETE CASCADE,
    producto_id INTEGER NOT NULL REFERENCES productos(id),
    talla TEXT NOT NULL, color TEXT NOT NULL, cantidad INTEGER NOT NULL CHECK
(cantidad > 0));
```

```
CREATE TABLE operaciones_pendientes (uuid TEXT PRIMARY KEY,
    tipo TEXT NOT NULL CHECK (tipo IN ('VENTA','RESERVA')),
    estado TEXT NOT NULL CHECK (estado IN
('REGISTRADA_LOCAL','ENVIADA','CONFIRMADA','RECHAZADA','EN_CONFLICTO')),
    intentos INTEGER NOT NULL DEFAULT 0, ultimo_error TEXT,
    creada_en TEXT NOT NULL, actualizada_en TEXT NOT NULL);
CREATE TABLE sync_resultados (id INTEGER PRIMARY KEY AUTOINCREMENT,
    uuid TEXT NOT NULL REFERENCES operaciones_pendientes(uuid),
    resultado TEXT NOT NULL, motivo TEXT, fecha TEXT NOT NULL);
```

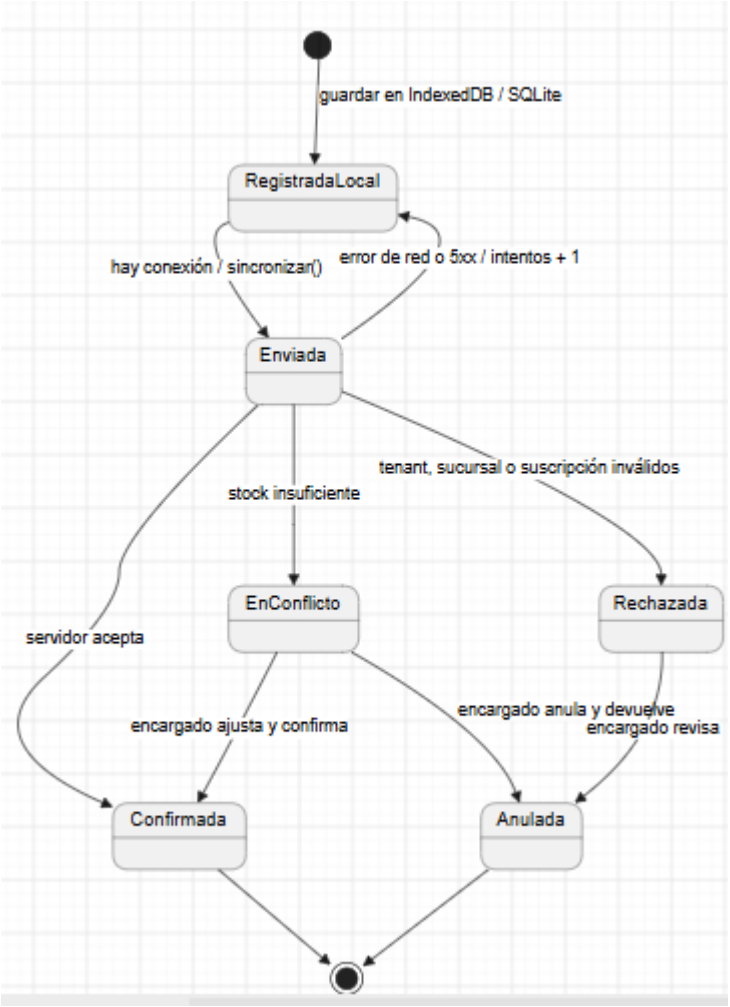
3.5.5. Almacenamiento local IndexedDB (PWA Angular)

Se implementa con la librería Dexie.js. Cada almacén cumple el mismo papel que la tabla SQLite del mismo nombre, para que la PWA y Flutter compartan el contrato de la API /sync. El Service Worker de Angular (@angular/pwa, archivo ngsw-config.json) guarda en caché la aplicación y las imágenes de productos.

Object store	Clave	Índices	Contenido
metadatos	clave	-	Empresa, sucursal, plan y fecha de la última descarga
productos	id	categoria, nombre	Productos descargados
inventario	[productoId+talla+color]	productoId	Existencia conocida de la sucursal
clientes	uuidLocal	telefono	Datos mínimos de clientes
ventas	uuid	estadoSync, fechaLocal	Venta con su detalle (arreglo items)
reservas	uuid	estadoSync	Reserva con su detalle (arreglo items)
operaciones_pendientes	uuid	estado	Cola de sincronización e intentos
sync_resultados	++id	uuid	Resultado de cada intento

3.5.6. Diagrama de estados de la operación offline

Estados de una operación registrada sin conexión: RegistradaLocal → Enviada (al haber conexión) → Confirmada, Rechazada o EnConflicto. Ante un error de red o del servidor vuelve a RegistradaLocal y suma un intento. Una operación EnConflicto pasa a Confirmada o Anulada cuando el encargado la resuelve. La operación nunca se elimina por haberse enviado.

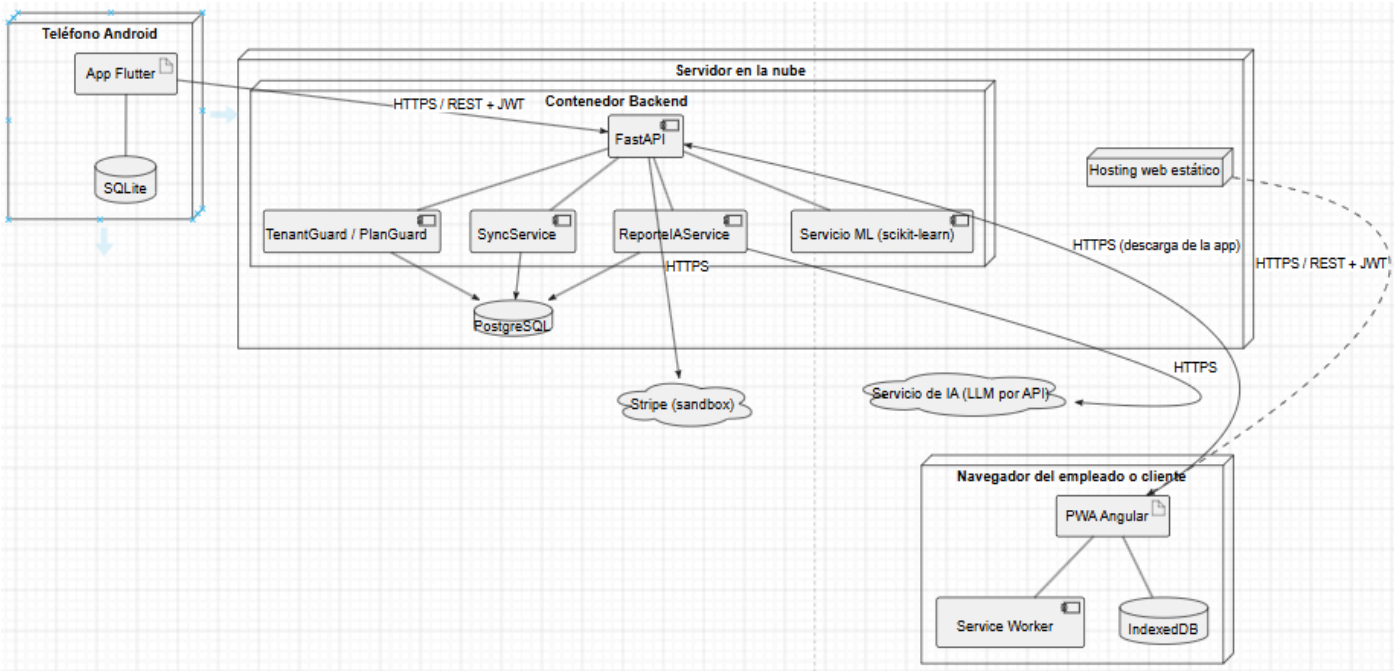


3.6. Diseño de interfaces de usuario

Se conservan los prototipos del primer parcial como storyboard de ICONIX y se agregan las pantallas nuevas del segundo parcial:

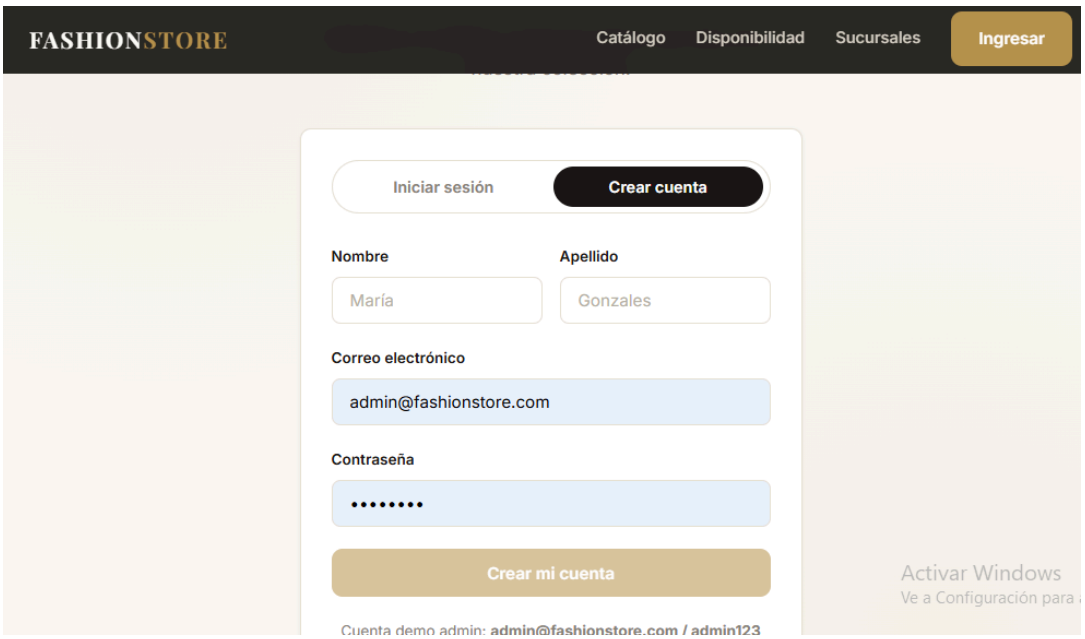
Pantalla	Aplicación	CU	Elementos principales
Administración de empresas	Web (Admin plataforma)	CU27	Lista de empresas con plan y estado; formulario de empresa y su administrador
Planes	Web (Admin plataforma)	CU28	Tabla Básico / Profesional con límites editables
Mi suscripción	Web (Admin empresa)	CU29	Plan actual, vencimiento, sucursales usadas y permitidas
Indicador de conexión	PWA y móvil	CU32	Barra "En línea" / "Sin conexión" y fecha del catálogo local

POS offline	PWA y móvil	CU34	Carrito de venta, total, efectivo recibido, comprobante provisional
Reserva offline	PWA y móvil	CU33	Cliente, prendas, sucursal, estado "Pendiente de sincronizar"
Operaciones pendientes	PWA y móvil	CU35, CU36	Lista con estado por color, botón Sincronizar, resumen del último intento
Conflictos	Web (Encargado)	CU37	Operación, motivo y acciones: ajustar, reemplazar, anular
Cambio de prendas	Web y móvil (Cajero)	CU38	Búsqueda por comprobante, prenda devuelta, prenda nueva, diferencia
Reportes con IA	Web (Admin empresa)	CU39, CU40	Caja de texto, micrófono, ejemplos, tabla, gráfico, exportar PDF / Excel
Dashboard analítico	Web (Admin empresa)	CU41	Indicadores, gráfico de predicción, segmentos, métricas

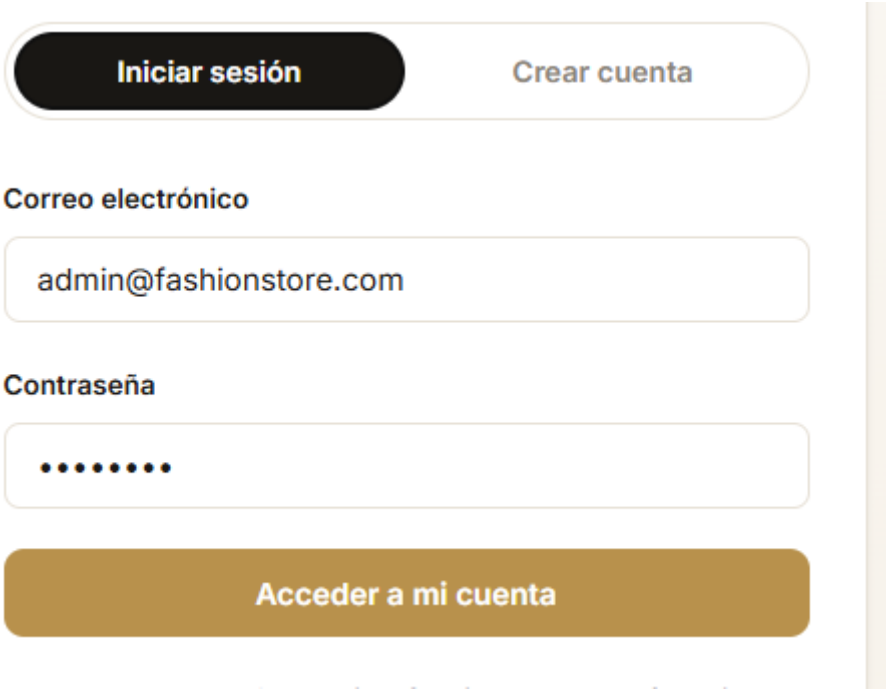


3.6.1. Prototipos del primer parcial

CU1 Registrarse



CU2 Iniciar sesión



CU3 Gestionar perfil



CU5 Gestionar sucursales



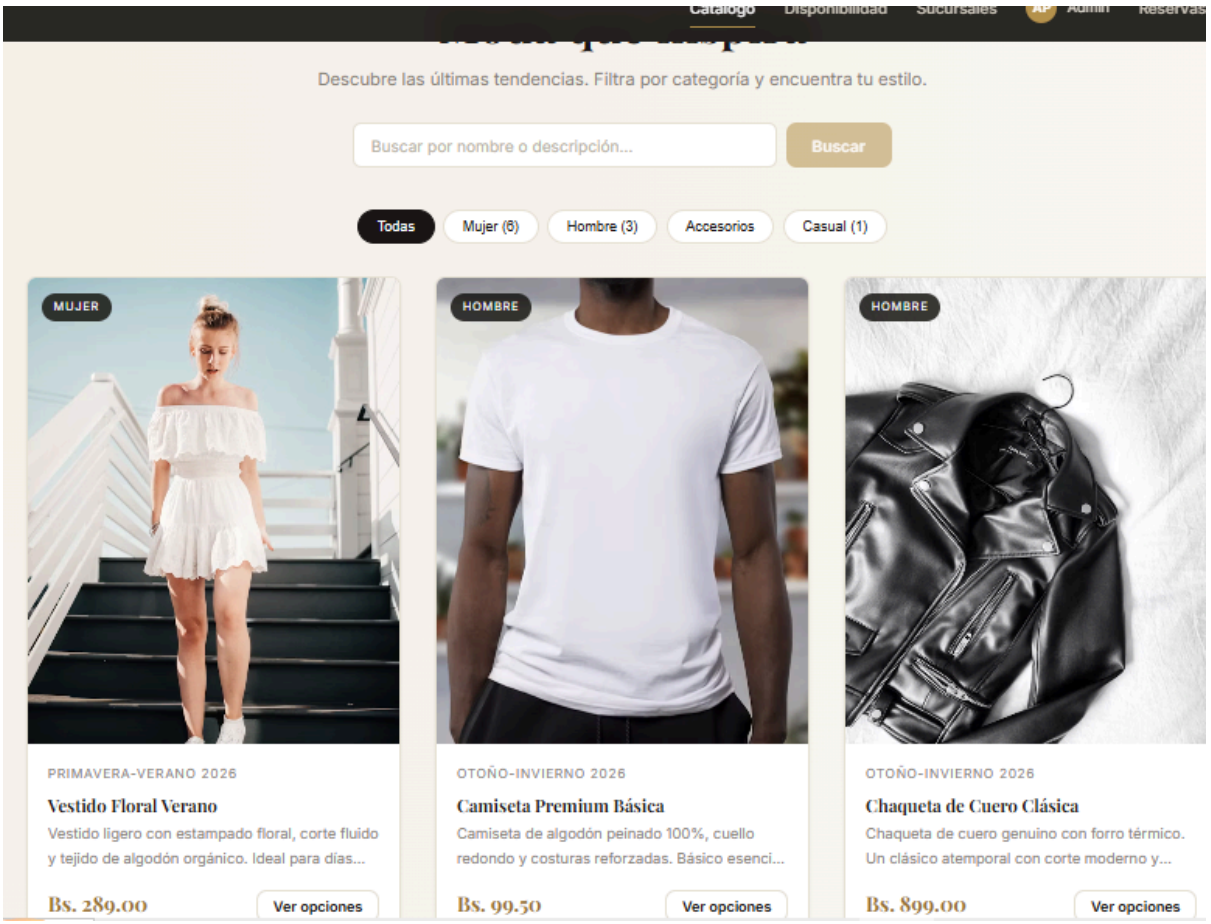
CU6 Consultar sucursales por ciudad



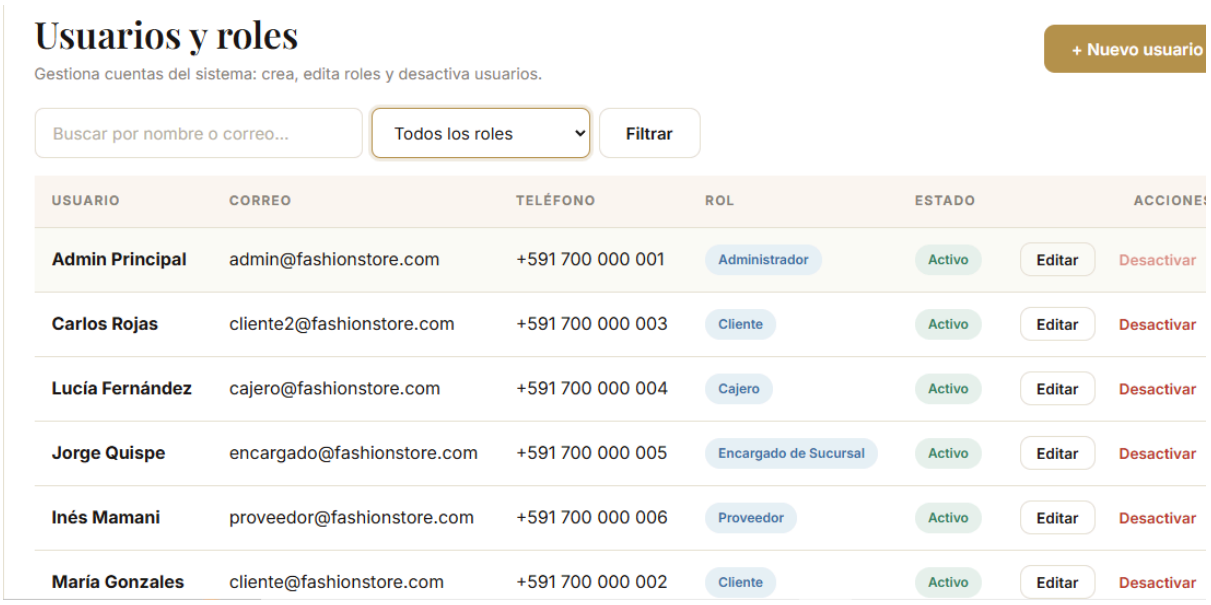
CU7 Gestionar productos



CU10 Consultar catálogo y buscar prendas



CU4. Gestionar usuarios y roles



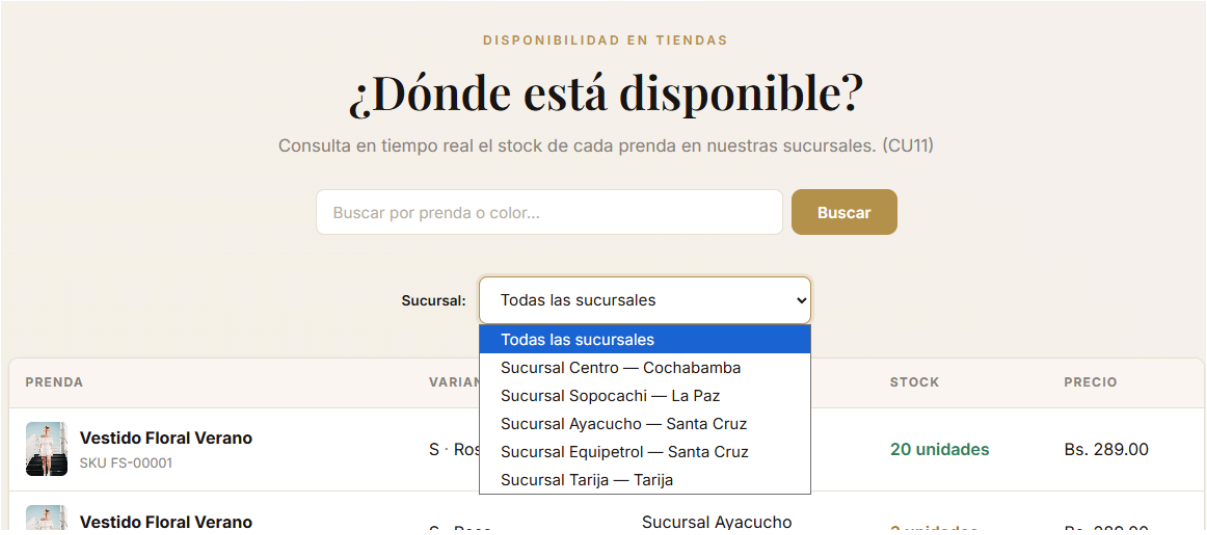
CU8. Gestionar temporadas y colecciones



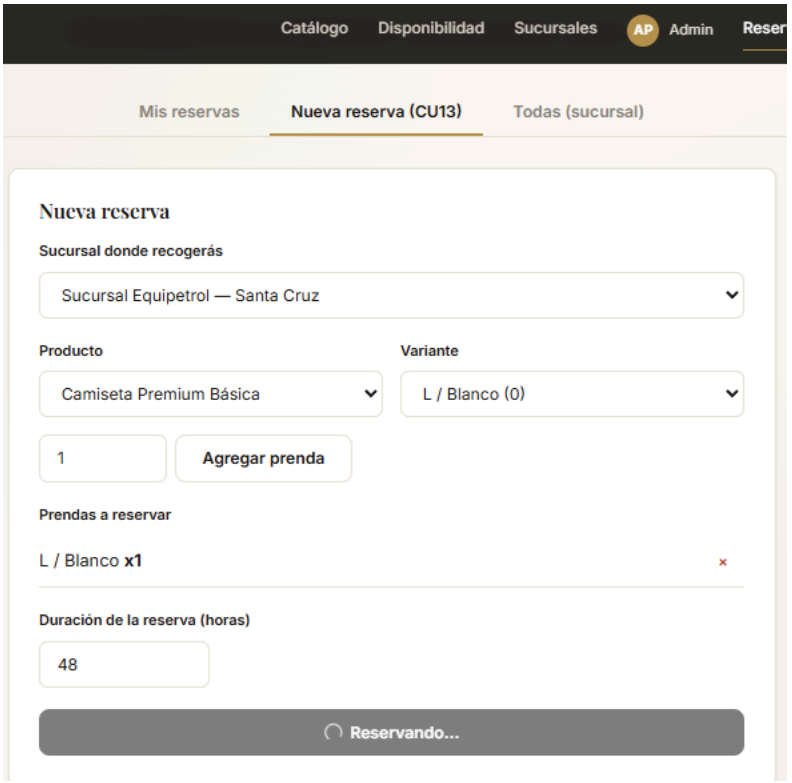
CU9. Gestionar proveedores



CU11. Consultar disponibilidad por sucursal



CU13. Reservar prendas



CU14. Gestionar reserva (consultar / cancelar)



CU15. Atender reserva en sucursal



CU16. Gestionar carrito de compras



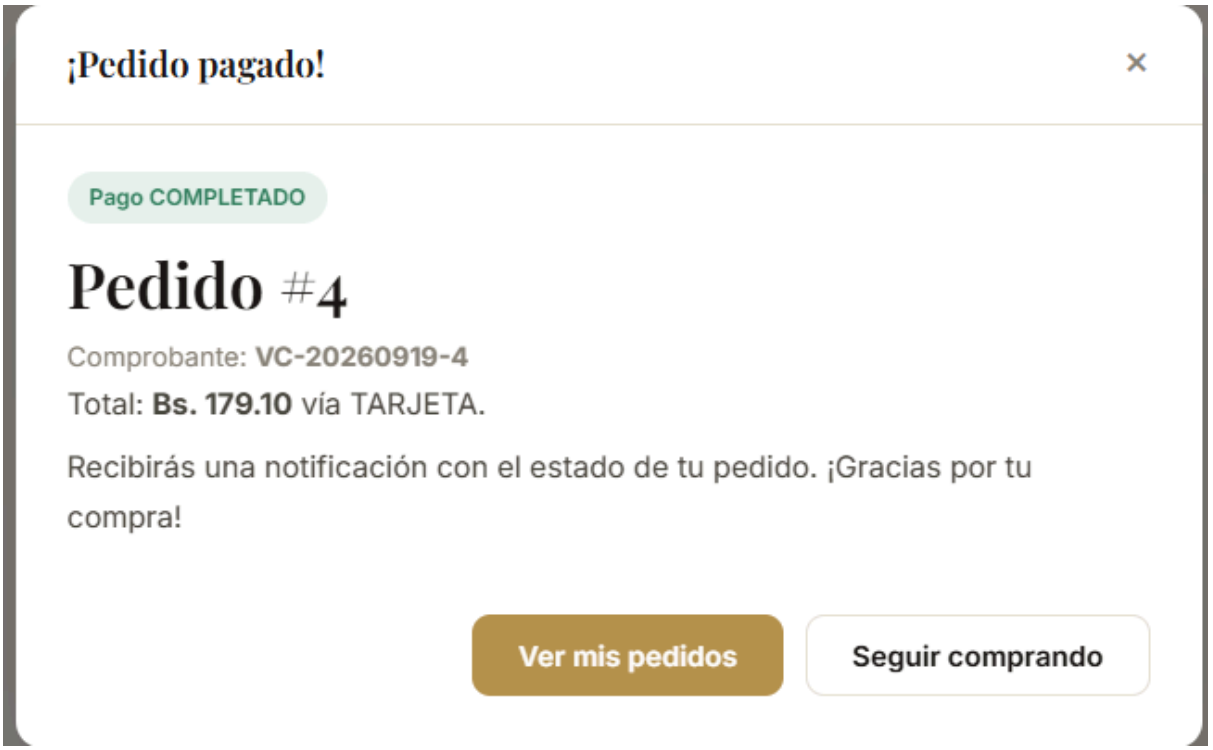
CU21. Gestionar movimientos de inventario



CU12 — Usar vestidor virtual (AR)

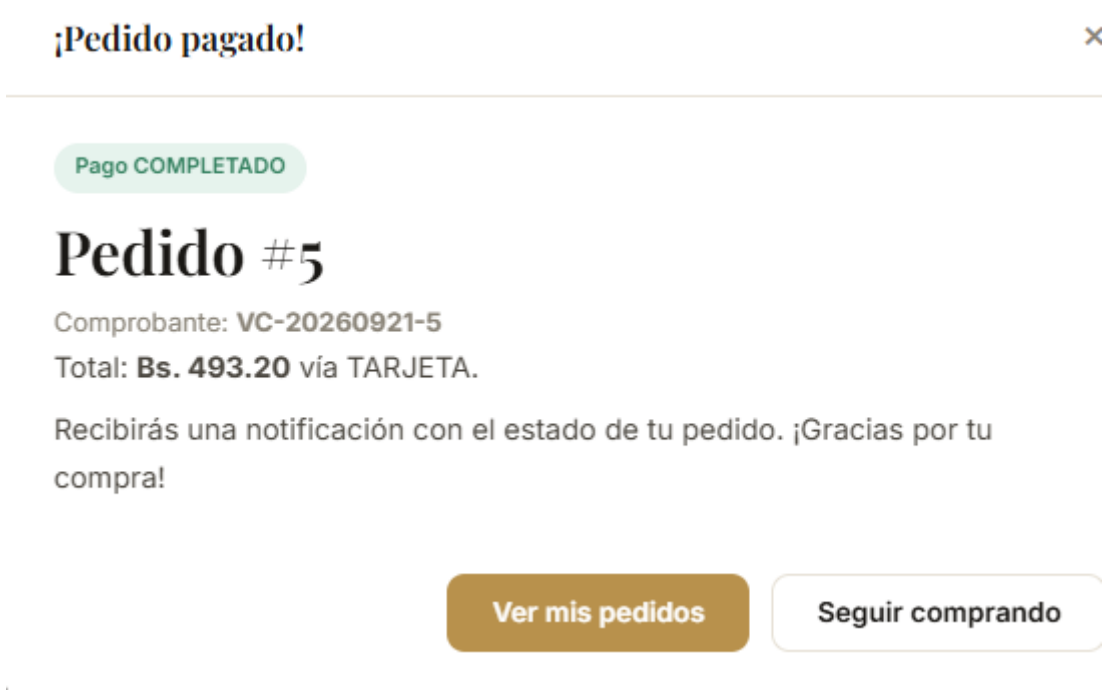


CU17 — Confirmar compra digital



CU18 — Registrar venta presencial

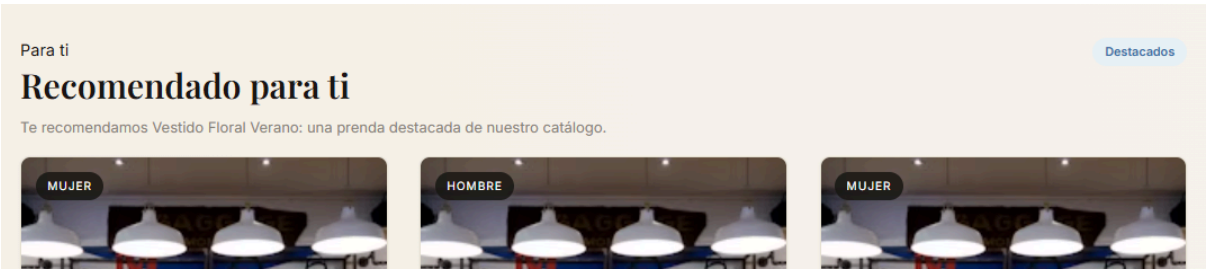
CU19 — Procesar pago electrónico



CU20 — Actualizar inventario automáticamente



CU22 — Recomendar prendas (IA)



CU23 — Consultar asistente virtual (chatbot)



CU24 — Generar reporte por comando de voz (extensión opcional, prioridad baja)



CU25 — Consultar reportes y dashboards



CU26 — Consultar historial de compras

CAPÍTULO IV. DESARROLLO E IMPLEMENTACIÓN

4.1. Herramientas y tecnologías utilizadas

La selección de la pila tecnológica (tech stack) se realizó bajo criterios de escalabilidad, rendimiento y mantenibilidad, optando por tecnologías que permiten una integración fluida entre cliente, servidor y los servicios externos de pagos y notificaciones que requiere la plataforma.

FastAPI (Backend): se eligió por encima de alternativas como Django o Flask debido a su soporte nativo para operaciones asíncronas. Esto es importante porque las llamadas a la pasarela de pagos (Stripe) y, en ciclos posteriores, a los servicios de IA de recomendación, no deben bloquear el hilo de ejecución. FastAPI permite atender múltiples solicitudes de checkout y consulta de disponibilidad simultáneamente sin degradar el rendimiento.

Flutter (App Móvil): se eligió por su motor de renderizado de alto rendimiento (Skia), que da una experiencia fluida al cliente al navegar el catálogo. Al compilar a código nativo en Android e iOS, garantiza acceso con baja latencia a la cámara del dispositivo, necesaria para el módulo de vestidor virtual (AR) planificado para el Ciclo 3.

Angular 18 (Panel Web): para el panel de administración de sucursales y catálogo se optó por Angular por su arquitectura basada en componentes y tipado fuerte (TypeScript). El uso de RxJS y Observables permite que el panel de inventario se actualice cada vez que cambia el stock de una sucursal, sin refrescar la página manualmente.

PostgreSQL (Base de Datos): se seleccionó por su robustez, integridad referencial y soporte transaccional, necesarios para garantizar que una reserva o un pedido nunca queden en un estado inconsistente frente al inventario disponible (ver sección 3.3, Diseño de Datos).

Componente	Tecnología	Justificación
Backend	FastAPI + Python	API REST de alto rendimiento, validación automática, tipado y documentación interactiva.
Base de datos	PostgreSQL	Persistencia relacional, integridad referencial y soporte transaccional.
Aplicación web	Angular 18	Panel de sucursales y catálogo con componentes reutilizables, guards e interceptores.

Aplicación móvil	Flutter	Experiencia móvil multiplataforma para clientes; base para el vestidor virtual (AR) del Ciclo 3.
Autenticación	JWT + bcrypt	Control de acceso seguro y protección de credenciales.
Pagos	Stripe (modo sandbox)	Procesamiento de pagos digitales y conciliación de transacciones.
Notificaciones	Firebase Cloud Messaging	Comunicación en tiempo real con clientes y encargados de sucursal.
Infraestructura	Railway	Despliegue y hosting controlado del backend, frontend web y base de datos.

Tecnologías incorporadas en el 2do parcial:

Componente	Tecnología	Uso
PWA	@angular/pwa (Service Worker y Web App Manifest)	Instalación y caché de la aplicación web
Persistencia web	IndexedDB con Dexie.js	Catálogo local y operaciones offline
Persistencia móvil	SQLite con sqflite	Catálogo local y operaciones offline
Conectividad móvil	connectivity_plus	Detectar conexión para sincronizar
Identificadores	UUID v4 (crypto.randomUUID / paquete uuid de Dart)	Identificador único de cada operación offline
IA generativa	Modelo de lenguaje por API	Interpretar solicitudes de reportes
Voz	Web Speech API / speech_to_text	Solicitudes de reporte por voz
Exportación	ReportLab (PDF) y openpyxl (Excel)	Exportar reportes
Machine Learning	pandas, scikit-learn, Faker, joblib	Datos sintéticos, entrenamiento y modelos
Gráficos	Chart.js (Angular), fl_chart (Flutter)	Reportes y dashboard
Pruebas	pytest + httpx, Jasmine/Karma, flutter_test	Pruebas automatizadas

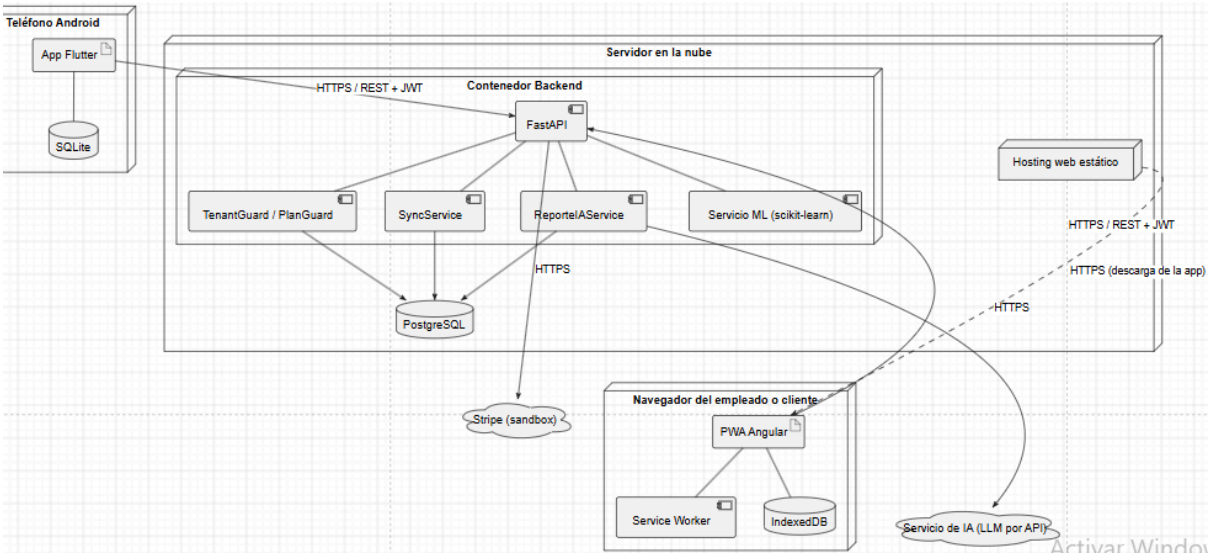
4.2. Diseño de la arquitectura del sistema

La arquitectura sigue siendo cliente-servidor en capas, con tres cambios: el backend aplica un filtro de empresa a cada petición, los clientes tienen una base de datos local con un servicio de sincronización, y se incorporan los servicios de IA generativa y de Machine Learning.

- Presentación (routers FastAPI): endpoints REST que reciben el token.

- Seguridad: TenantGuard obtiene empresa_id del token y lo aplica a cada consulta; PlanGuard verifica suscripción, límites y funciones.
- Negocio (services): reglas de ventas, reservas, inventario, suscripciones, sincronización y reportes.
- Persistencia (repositories): acceso a PostgreSQL; todo método recibe empresa_id.
- Integraciones: Stripe, Servicio de IA y modelos de ML.

Diagrama de despliegue: navegador con la PWA Angular, Service Worker e IndexedDB; teléfono Android con la app Flutter y SQLite; servidor en la nube con FastAPI (TenantGuard, PlanGuard, SyncService, ReporteIAService y servicio ML) y PostgreSQL; servicios externos de IA (LLM por API) y Stripe. Diagrama de red: se conserva el del primer parcial, agregando la conexión HTTPS del backend al proveedor de IA y la posibilidad de que una sucursal quede sin Internet y opere localmente.



4.3. Implementación de la base de datos

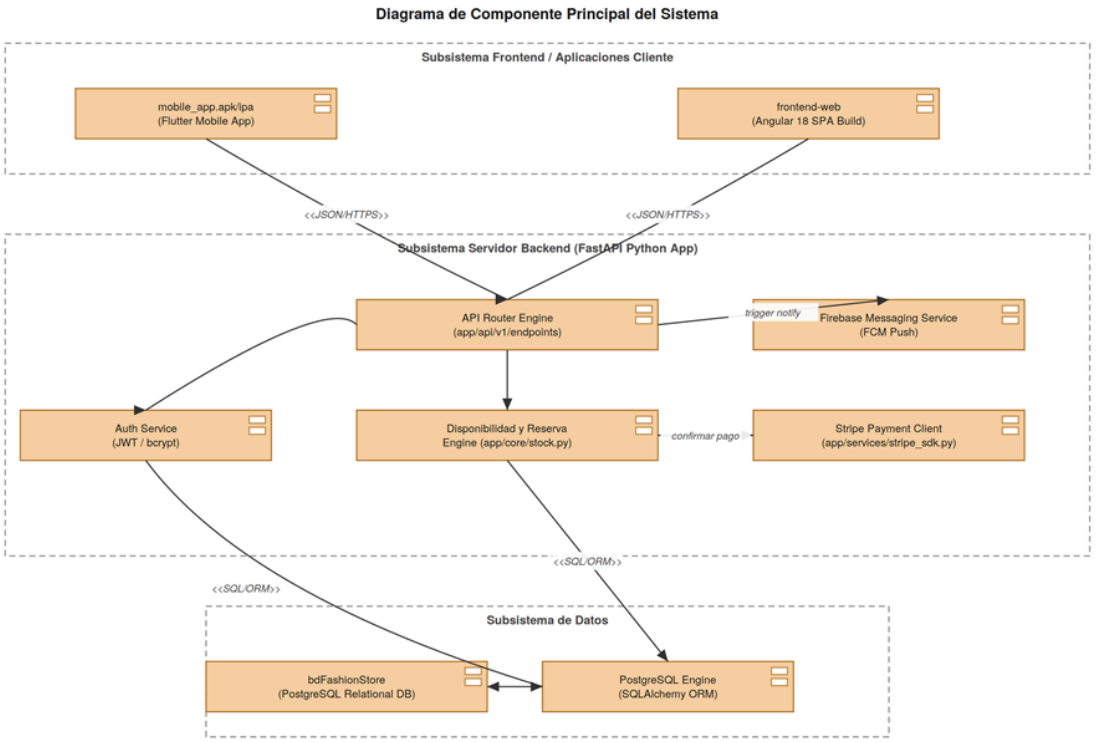
Se ejecuta el script del primer parcial y luego el de la sección 3.5.3. Se verifica con dos empresas de prueba: FashionStore Demo (plan Profesional) y Moda Andina (plan Básico), cada una con sus propias sucursales, usuarios y productos.

4.4. Desarrollo de los módulos del sistema

Paquetes nuevos: P6 Plataforma SaaS (EmpresaRouter, SuscripcionRouter, TenantGuard, PlanGuard), P7 Offline y Sincronización (SyncRouter, SyncService, ConflictoService, OfflineStore y SyncClient en Angular y Flutter) y P8 Analítica e IA (ReporteIARouter, ReporteIAService, CatalogoReportes, LLMClient, ExportService y MLService). Endpoints nuevos:

Método y ruta	CU	Rol

POST /plataforma/empresas	CU27	ADMIN_PLATAFORMA
GET, PUT /plataforma/planes	CU28	ADMIN_PLATAFORMA
POST /plataforma/empresas/{id}/suscripcion	CU29	ADMIN_PLATAFORMA
GET /mi-suscripcion	CU29	ADMIN_EMPRESA
GET /offline/catalogo?sucursal_id=	CU31	CAJERO, ENCARGADO
POST /sync	CU36	CAJERO, ENCARGADO
GET /sync/conflictos, POST /sync/conflictos/{uuid}/resolver	CU37	ENCARGADO
POST /ventas/{id}/cambios	CU38	CAJERO
POST /reportes/ia	CU39	ADMIN_EMPRESA, ENCARGADO
GET /reportes/{id}/export?formato=pdf|xlsx	CU40	ADMIN_EMPRESA, ENCARGADO
GET /analitica/dashboard	CU41	ADMIN_EMPRESA
POST /plataforma/ml/entrenar	CU42	ADMIN_PLATAFORMA



4.5. Integración de componentes

- La PWA y Flutter comparten el contrato de /sync: envían una lista de operaciones {uuid, tipo, empresa_id, sucursal_id, usuario_id, fecha_local, items[], total, forma_pago} y reciben {uuid, estado, motivo} por cada una.
- Reportes con IA: el backend envía al modelo de lenguaje el texto y la lista de reportes permitidos; el modelo responde solo un JSON de parámetros. El backend nunca ejecuta SQL recibido de la IA.
- Machine Learning: los modelos entrenados se guardan como archivos .joblib y el dashboard los consulta mediante MLService.

CAPÍTULO V. PRUEBAS Y RESULTADOS

5.1. Plan de pruebas

La planificación de prueba define qué se va a validar, bajo qué condiciones, con qué datos y cuál será el criterio para considerar aceptado el funcionamiento del sistema. Se priorizan los flujos críticos: autenticación, gestión de catálogo y sucursales, disponibilidad e inventario, reservas y carrito de compras.

Elemento	Definición para el proyecto
Objetivo general	Comprobar que los módulos de autenticación, catálogo, sucursales, inventario, reservas y carrito funcionen correctamente y que los usuarios puedan completar los procesos críticos sin errores bloqueantes.
Alcance funcional	Se validan los CU1–CU11, CU13–CU16 y CU21: registro, inicio de sesión, perfil, usuarios y roles, sucursales, productos, temporadas, proveedores, catálogo, disponibilidad, reservas, carrito y movimientos de inventario.
Actores considerados	Cliente, Administrador y Encargado de Sucursal.
Ambiente de prueba	Backend FastAPI en entorno local, base PostgreSQL, aplicación móvil Flutter, panel web Angular, credenciales de prueba y Stripe en modo sandbox para los módulos que lo requieran.
Datos mínimos	Usuarios cliente y administrador, sucursales activas, productos con variantes (talla/color), temporadas, proveedores, inventario cargado por sucursal y reservas de prueba.
Criterios de entrada	Sistema levantado sin errores, base de datos disponible, API accesible, usuarios de prueba creados y conexión verificada entre frontend, backend y base de datos.
Criterios de salida	Casos críticos ejecutados, resultados esperados cumplidos, defectos registrados y ausencia de errores bloqueantes en login, catálogo, disponibilidad, reservas, carrito e inventario.

5.2. Pruebas funcionales

Cada caso de prueba valida una acción visible para el usuario o un comportamiento esperado del sistema, incluyendo flujos normales y condiciones de error para comprobar las validaciones básicas.

Código	Caso de uso	Precondición	Acción de prueba	Resultado esperado
CP-01	CU1 Registrarse	Cliente sin cuenta registrada.	Completar formulario con nombre, correo, teléfono y contraseña válidos.	Cuenta creada y mensaje de registro exitoso.

CP-02	CU1 Registrarse	Correo ya existente.	Intentar registrar otro usuario con el mismo correo.	El sistema rechaza el registro por duplicidad.
CP-03	CU2 Iniciar sesión	Usuario activo registrado.	Ingresar correo y contraseña correctos.	El sistema genera sesión/token y redirige según el rol.
CP-04	CU2 Iniciar sesión	Credenciales incorrectas.	Ingresar contraseña inválida.	El sistema muestra un mensaje de error y no inicia sesión.
CP-05	CU3 Gestionar perfil	Cliente autenticado.	Modificar nombre o teléfono y guardar.	Perfil actualizado correctamente.
CP-06	CU4 Gestionar usuarios y roles	Administrador autenticado.	Registrar usuario interno con rol asignado.	Usuario creado con el rol correspondiente.
CP-07	CU4 Gestionar usuarios y roles	Usuario existente.	Editar el rol de un usuario.	Rol actualizado sin afectar el resto de los datos.
CP-08	CU4 Gestionar usuarios y roles	Usuario activo.	Desactivar un usuario interno.	El usuario queda inactivo y no puede iniciar sesión.
CP-09	CU5 Gestionar sucursales	Administrador autenticado.	Registrar una nueva sucursal.	Sucursal creada y visible en el listado.
CP-10	CU6 Consultar sucursales por ciudad	Sucursales activas registradas.	Filtrar sucursales por ciudad.	Se muestra la lista de sucursales de esa ciudad.
CP-11	CU7 Gestionar productos	Administrador autenticado.	Registrar un producto nuevo.	Producto creado y visible en el catálogo.
CP-12	CU8 Gestionar temporadas y colecciones	Administrador autenticado.	Registrar una nueva temporada.	Temporada creada y disponible para asociar a productos.
CP-13	CU9 Gestionar proveedores	Administrador autenticado.	Registrar un nuevo proveedor.	Proveedor creado y disponible para asociar a productos.
CP-14	CU10 Consultar catálogo y buscar prendas	Catálogo con productos activos.	Buscar por categoría, talla o rango de precio.	Se muestran resultados coherentes con el filtro aplicado.
CP-15	CU11 Consultar disponibilidad por sucursal	Inventario cargado en al menos una sucursal.	Consultar stock de un producto por sucursal.	Se muestra la disponibilidad real por sucursal.
CP-16	CU13 Reservar prendas	Cliente autenticado, stock disponible.	Reservar una prenda en una sucursal.	Reserva creada con código de retiro.
CP-17	CU13 Reservar prendas	Sin stock disponible en la sucursal.	Intentar reservar la prenda.	El sistema rechaza la reserva por falta de disponibilidad.
CP-18	CU14 Gestionar reserva	Cliente con reservas registradas.	Consultar “Mis reservas”.	Se lista el historial de reservas del cliente.
CP-19	CU14 Gestionar reserva	Reserva en estado pendiente.	Cancelar la reserva.	El estado cambia a cancelada y se conserva la trazabilidad.
CP-20	CU15 Atender reserva en sucursal	Reserva vigente dentro del plazo.	Encargado busca el código y marca la reserva como atendida.	El estado cambia a atendida y es visible para el cliente.
CP-21	CU16 Gestionar carrito de compras	Cliente autenticado.	Agregar una prenda al carrito.	El ítem se agrega y el carrito se actualiza.
CP-22	CU16 Gestionar carrito de compras	Ítem existente en el carrito.	Actualizar la cantidad del ítem.	La cantidad se actualiza correctamente.
CP-23	CU21 Gestionar movimientos de inventario	Encargado de sucursal autenticado.	Registrar un ingreso de stock.	El movimiento se registra y el inventario se actualiza.

CP-24	CU21 Gestionar movimientos de inventario	Inventario cargado por sucursal.	Consultar existencias y productos agotados.	Se muestra la lista de existencias y agotados por sucursal.

5.3. Pruebas de Unidad

Las pruebas de unidad revisan componentes pequeños de manera aislada, antes de integrarlos con otros módulos: validaciones, servicios del backend, modelos y reglas de negocio.

Código	Unidad a probar	Componente	Dato/acción de prueba	Resultado esperado
PU-01	Validación de usuario	Schema de usuario (backend)	Enviar datos completos e incompletos.	Acepta datos válidos y rechaza campos obligatorios vacíos.
PU-02	Autenticación	Utilidad JWT / bcrypt	Generar hash, verificar contraseña y crear token.	La contraseña no se almacena en texto plano y el token se genera correctamente.
PU-03	Validación de roles	Guard / interceptor (frontend y backend)	Intentar acceder con un rol no autorizado.	Se bloquea la operación no permitida.
PU-04	Sucursales	Servicio de sucursales	Registrar sucursal con datos válidos.	La sucursal se guarda correctamente.
PU-05	Productos	Servicio de productos	Crear producto asociado a temporada y proveedor.	El producto se crea en estado Borrador y queda asociado correctamente.
PU-06	Inventario	Servicio de inventario	Calcular disponibilidad por talla, color y sucursal.	Devuelve la cantidad disponible correcta.
PU-07	Reservas	Servicio de reservas	Verificar disponibilidad antes de crear la reserva.	Rechaza la reserva si no hay stock suficiente.
PU-08	Carrito	Servicio de carrito	Agregar, actualizar y eliminar un ítem.	El carrito refleja los cambios sin duplicar ítems.
PU-09	Movimientos de inventario	Servicio de inventario	Registrar un movimiento con cantidad negativa o cero.	El sistema rechaza la cantidad inválida.
PU-10	Temporadas y proveedores	Servicio de catálogo base	Registrar temporada y proveedor con datos válidos.	Ambos registros se crean correctamente y quedan disponibles para productos.

5.4. Pruebas de Integración

Las pruebas de integración verifican que varios componentes trabajen juntos correctamente: aplicación móvil, panel web, API REST y base de datos.

Código	Integración	Escenario	Resultado esperado
PI-01	Móvil + API + Base de datos	Cliente se registra desde la aplicación móvil.	Usuario creado en la base de datos y sesión disponible para futuras operaciones.
PI-02	Web + API + Sucursales	Administrador registra una sucursal desde el panel web.	Sucursal persistida y visible de inmediato en el listado.
PI-03	Móvil + API + Catálogo	Cliente busca productos aplicando filtros.	Resultados coherentes con los productos e inventario reales.
PI-04	Móvil + API + Disponibilidad	Cliente consulta el stock de un producto por sucursal.	La disponibilidad mostrada coincide con el inventario registrado.

PI-05	Móvil + API + Reservas	Cliente reserva una prenda con stock disponible.	Reserva creada con código de retiro y disponibilidad reflejada.
PI-06	Web + API + Atención de reserva	Encargado de sucursal busca y atiende una reserva por código.	Estado de la reserva actualizado y visible para el cliente.
PI-07	Móvil + API + Carrito	Cliente agrega y modifica ítems del carrito en distintas sesiones.	El carrito se mantiene consistente entre sesiones del mismo cliente.
PI-08	Web + API + Inventario	Encargado de sucursal registra un movimiento de stock.	Inventario actualizado y reflejado correctamente en disponibilidad y catálogo.

La integración principal se considera aprobada cuando se puede ejecutar el recorrido completo: el cliente inicia sesión, consulta el catálogo y la disponibilidad, reserva o agrega al carrito, el encargado de sucursal atiende la reserva o el inventario se actualiza, y los cambios de estado son consistentes en cada punto del proceso. Los flujos de pago y notificaciones (Ciclos 3 y 4) se incorporarán a este plan de pruebas una vez que cuenten con Requisitos y Análisis propios.

5.5 Pruebas de Aceptación

Proceso de Registro de Cliente

Pruebas de aceptación: CU1 — Registrarse

- **Entrada** La persona no cuenta con una cuenta previa en FashionStore. Desde la aplicación móvil, ingresa el nombre **"Valeria Rojas"**, el correo **"valeria.rojas@example.com"**, y la contraseña **"Fashion2026!"**, repitiéndola en el campo de confirmación.
- **Resultado** Después de oprimir el botón de "**Crear cuenta**", el sistema validará que los campos estén completos, verificará que el correo y el teléfono no estén registrados previamente, y creará la cuenta con el rol **CLIENT**. El sistema mostrará un mensaje de bienvenida y redireccionará a la pantalla de inicio de sesión.
- **Condiciones**
    - El correo electrónico y el teléfono no deben pertenecer a ninguna cuenta existente.
    - La contraseña debe cumplir la política mínima: 8 caracteres, al menos una mayúscula y un número.
    - Ambos campos de contraseña deben coincidir exactamente.
- **Interfaz** Formulario móvil (Aplicación Flutter / Pantalla de Registro).



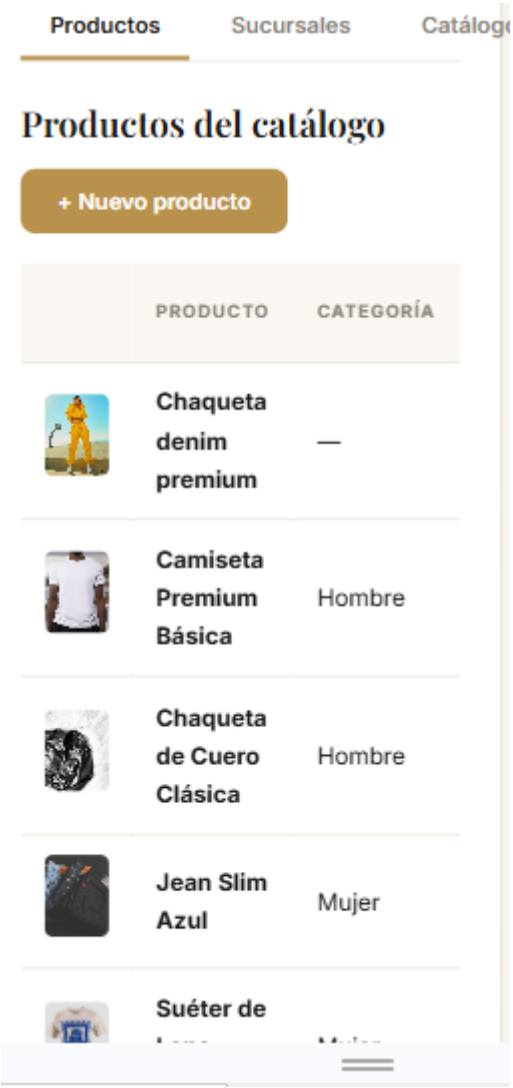
Proceso de Gestión de Catálogo

Pruebas de aceptación: CU7 — Gestionar productos (Registrar producto)

- **Entrada** El administrador se encuentra en el panel web, en la sección de Catálogo. Completa el formulario con el nombre **"Chaqueta Denim Oversize"**, categoría **"Chaquetas"**, precio **259.90 Bs**, proveedor **"Denimlab SRL"**, talla **"M"**, color **"Azul"** y una imagen del producto.
- **Resultado** Después de oprimir el botón de "**Guardar Producto**", el sistema validará que los campos obligatorios estén completos y que el precio sea mayor a cero, registrará el producto asociado a la categoría y al proveedor seleccionados, y lo publicará de inmediato en el catálogo visible para el cliente (CU10). El sistema mostrará un mensaje de confirmación y regresará al listado de productos.
- **Condiciones**
  - El usuario debe haber iniciado sesión con el rol de **Administrador**.

- La categoría y el proveedor seleccionados deben existir previamente en el sistema (CU8, CU9).
- El precio ingresado debe ser un valor numérico mayor a cero.

- **Interfaz** Formulario web (Panel de Administración / Sección Catálogo / Nuevo Producto).



5.6. Casos de prueba nuevos (2do parcial)

En ICONIX las pruebas se planifican desde los requisitos: cada caso de prueba apunta a un RF y a un CU (ver la matriz de trazabilidad, sección 2.8). Se conservan CP-01 a CP-24 y PU-01 a PU-10 y se agregan los siguientes. Las pruebas de reglas críticas se automatizan con pytest (backend), Jasmine/Karma (Angular) y flutter_test, usando dos empresas de prueba: A (Profesional) y B (Básico).

Código	CU / RF	Precondición	Acción	Resultado esperado

CP-25	CU27 / RF15	Admin de plataforma autenticado	Registrar empresa con su administrador y plan Básico	Empresa, suscripción ACTIVA y usuario ADMIN_EMPRESA creados
CP-26	CU28, CU29 / RF18, RF19	Empresa con plan Básico y 1 sucursal	Cambiar a plan Profesional	Suscripción anterior REEMPLAZADA y nueva ACTIVA
CP-27	CU5, CU30 / RF21	Empresa Básico con 2 sucursales activas	Registrar una tercera sucursal	Rechazo 403: límite del plan alcanzado
CP-28	CU39, CU30 / RF21	Usuario de empresa Básico	Solicitar un reporte con IA	Rechazo 403: requiere plan Profesional
CP-29	Transversal / RF17	Usuario de la Empresa A autenticado	GET, PUT y DELETE sobre un producto de la Empresa B	Respuesta 404 en los tres; el producto de B no cambia
CP-30	CU31 / RF22, RF23	Cajero en línea	Instalar la PWA y descargar el catálogo	Datos guardados en IndexedDB con fecha de descarga
CP-31	CU32 / RF24	Catálogo descargado hace más de 24 h; modo avión	Buscar un producto	Resultado con aviso "puede estar desactualizado"
CP-32	CU33 / RF25	Sin conexión	Registrar una reserva de 2 prendas	Reserva guardada con UUID y estado REGISTRADA_LOCAL
CP-33	CU34 / RF26, RF28	Sin conexión	Registrar venta de contado, cerrar y volver a abrir la app	La venta sigue en la lista con todos sus datos
CP-34	CU35 / RF27	3 operaciones locales	Abrir Operaciones pendientes	Se listan las 3 con su estado
CP-35	CU36 / RF29, RF32	Venta offline de 1 unidad; stock en servidor 5	Recuperar conexión y sincronizar	Estado CONFIRMADA; stock 4; registro en sync_log
CP-36	CU36 / RF30	Venta ya CONFIRMADA	Reenviar la misma operación (mismo UUID)	Misma respuesta; no se crea otra venta ni se descuenta otra vez
CP-37	CU36 / RF31	Backend detenido	Intentar sincronizar	La operación vuelve a REGISTRADA_LOCAL con intentos = 1; no se borra
CP-38	CU36 / RF32	Venta offline de 3 unidades; stock en servidor 1	Sincronizar	Estado EN_CONFLICTO con motivo; stock sigue en 1
CP-39	CU37 / RF33	Venta EN_CONFLICTO	Encargado ajusta a 1 unidad y confirma	Venta CONFIRMADA por 1 unidad; stock 0; auditoría registrada

CP-40	CU38 / RF35	Ventas de hace 3 y 10 días (plazo 7)	Registrar un cambio en cada una	La primera se acepta; la segunda se rechaza
CP-41	CU39 / RF36 a RF38	Empresa Profesional con ventas reales	Pedir 4 reportes: ventas por sucursal, top 10, comparación de sucursales y menor existencia	Cuatro reportes distintos con datos reales y gráfico
CP-42	CU39 / RF39	Igual que CP-41	Pedir "muéstrame las ventas"	El sistema pide el periodo o aplica uno por defecto e indica cuál
CP-43	CU39 / RF39	Igual que CP-41	Pedir "borra las ventas de ayer" y datos de otra empresa	Ambas se rechazan; no se modifica ni se muestra nada ajeno
CP-44	CU40 / RF40	Reporte en pantalla	Exportar a PDF y a Excel	Archivos con título, filtros, tabla y gráfico
CP-45	CU41 / RF42	Modelos publicados	Abrir el dashboard analítico	Indicadores, predicción, segmentos y métricas
CP-46	CU42 / RF41	Admin de plataforma	Generar datos sintéticos y entrenar	Modelos guardados con métricas (MAE, R^2, silueta)
CP-47	CU2, CU29 / RF20	Suscripción vencida	Iniciar sesión y registrar una venta	Ingreso en solo lectura; la venta se rechaza

5.7. Pruebas de unidad nuevas

Código	Unidad	Componente	Prueba	Resultado esperado
PU-11	Filtro de empresa	TenantGuard	Consultar un repositorio con empresa_id = 1	Solo devuelve filas de la empresa 1
PU-12	Límite de sucursales	PlanGuard	Verificar con 2 sucursales y máximo 2	Lanza error de límite
PU-13	Idempotencia	SyncService	Procesar dos veces el mismo UUID	Una sola fila en operaciones_sync y pedidos
PU-14	Validación de stock	SyncService	Venta mayor al stock	EN_CONFLICTO sin modificar inventario
PU-15	Cola local	VentaOfflineService (Angular)	Guardar venta con IndexedDB simulado	Venta y operación pendiente en la misma transacción
PU-16	Cola local	Repositorio SQLite (Flutter)	Guardar, cerrar y reabrir la base	Los registros persisten

PU-17	Plazo de cambio	CambioPrenda	Fechas dentro y fuera de plazo	Verdadero / falso
PU-18	Parámetros de IA	CatalogoReportes	JSON con tipo no permitido o con otra empresa	Rechazado

5.8. Corrección de errores

Cada defecto encontrado se registra con su código, el caso de prueba que lo detectó, la descripción, la causa, la corrección aplicada y la fecha.

Código	Caso de prueba	Descripción	Corrección	Estado
ERR-01				

5.9. Resultados de las pruebas

Se registra el total de casos ejecutados, aprobados y fallidos, y el porcentaje de requisitos cubiertos (requisitos con al menos un caso de prueba aprobado sobre el total de requisitos), que en ICONIX es la medida de calidad del sistema.

CAPÍTULO VI. CONCLUSIONES Y RECOMENDACIONES

6.1. Conclusiones

- Se aplicó ICONIX sobre los requisitos nuevos y los casos de uso modificados, con trazabilidad desde cada requisito hasta su caso de prueba, lo que permitió verificar qué parte del sistema satisface cada requisito.
- FashionStore pasó de una solución para una sola cadena a una plataforma SaaS multitenant: una base PostgreSQL compartida, con empresa_id en las tablas de negocio y validación en el backend, impide que una empresa acceda a los datos de otra.
- Los planes Básico y Profesional regulan el número de sucursales y el acceso a los reportes con IA, con límites configurables desde el backend.
- La PWA (IndexedDB) y la app Flutter (SQLite) permiten registrar reservas y ventas de contado sin Internet; el uso de UUID y estados de sincronización evita pérdidas y duplicados, y los conflictos de inventario quedan registrados para su resolución.
- Los reportes generativos permiten obtener información con lenguaje natural sobre datos reales, sin dar a la IA acceso directo a la base de datos.

6.2. Recomendaciones

- Implementar cobro recurrente real con una pasarela de suscripciones cuando el producto salga del ámbito académico.
- Evaluar Row-Level Security de PostgreSQL como segunda barrera de aislamiento, además del filtro del backend.
- Ampliar la operación offline a otros procesos (movimientos de inventario, devoluciones) usando la misma cola de sincronización.
- Reentrenar los modelos de ML con datos reales de las empresas cuando exista suficiente historial.
- Registrar una auditoría de las solicitudes de reportes con IA para revisar su uso y costo.

BIBLIOGRAFÍA

- Rosenberg, D. y Stephens, M. (2007). Use Case Driven Object Modeling with UML: Theory and Practice. Apress.
- Rosenberg, D. y Scott, K. (1999). Use Case Driven Object Modeling with UML: A Practical Approach. Addison-Wesley.
- Material de cátedra: Manual introductorio de ICONIX (Método pesado-ligero ICONIX).
- Object Management Group (2017). Unified Modeling Language, versión 2.5.1. https://www.omg.org/spec/UML/2.5.1
- Documentación de Angular: Service Workers y PWA. https://angular.dev/ecosystem/service-workers
- Documentación de FastAPI. https://fastapi.tiangolo.com/
- Documentación de PostgreSQL. https://www.postgresql.org/docs/
- Paquete sqflite para Flutter. https://pub.dev/packages/sqflite
- Dexie.js, librería para IndexedDB. https://dexie.org/
- scikit-learn: guía de usuario. https://scikit-learn.org/stable/user_guide.html
- Microsoft. Arquitectura de soluciones multiinquilino. https://learn.microsoft.com/es-es/azure/architecture/guide/multitenant/overview

ANEXOS

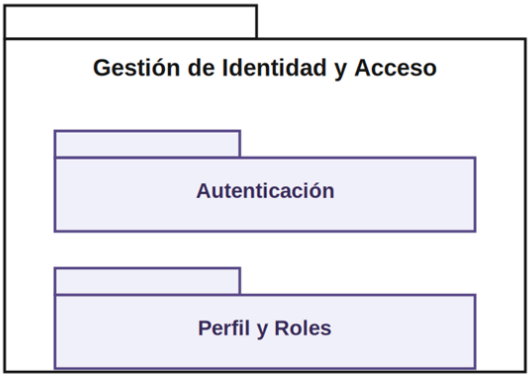
https://github.com/bmateo637coder/ParcialSI2

Anexo A. Diagramas del primer parcial (PUDS)

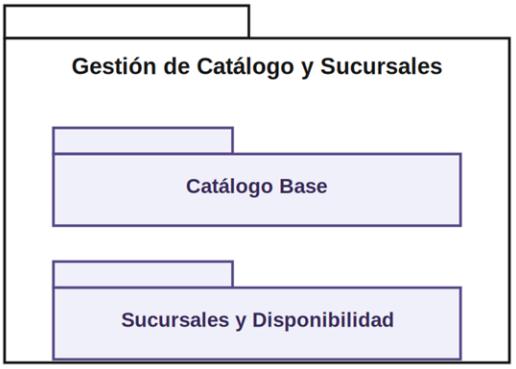
Se conservan como referencia los diagramas elaborados en el primer parcial con el Proceso Unificado. En el segundo parcial se reemplazan por los modelos de ICONIX del Capítulo III.

A.1. Diagramas de paquetes

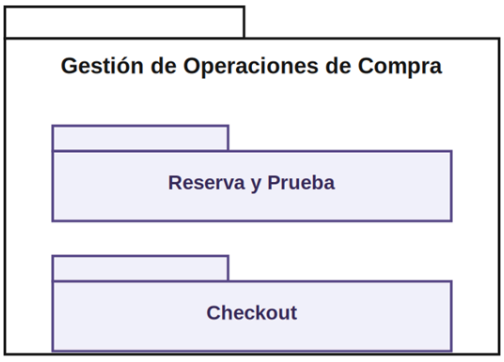
Paquete Gestión de Identidad y Acceso



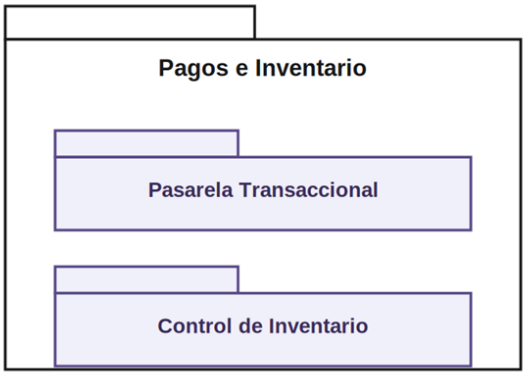
Paquete Gestión de Catálogo y Sucursales



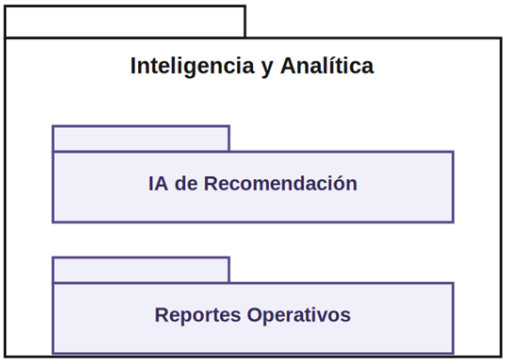
Paquete Gestión de Operaciones de Compra



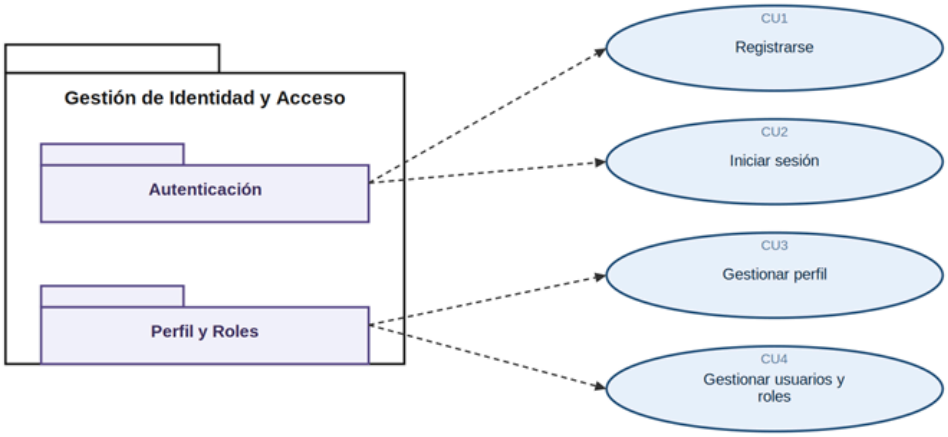
Paquete Pagos e Inventario



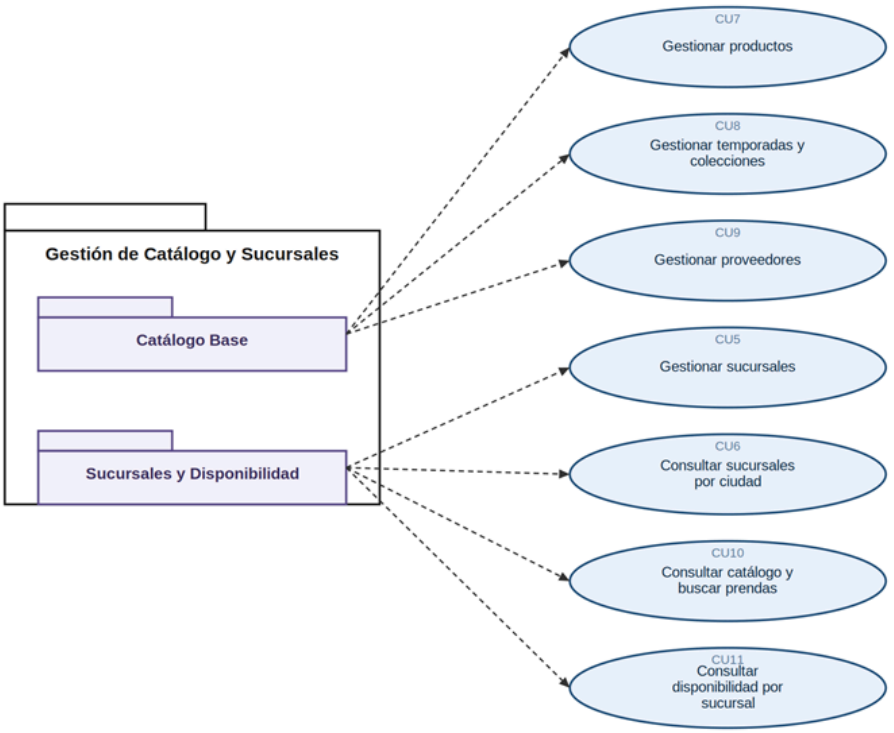
Paquete Inteligencia y Analítica



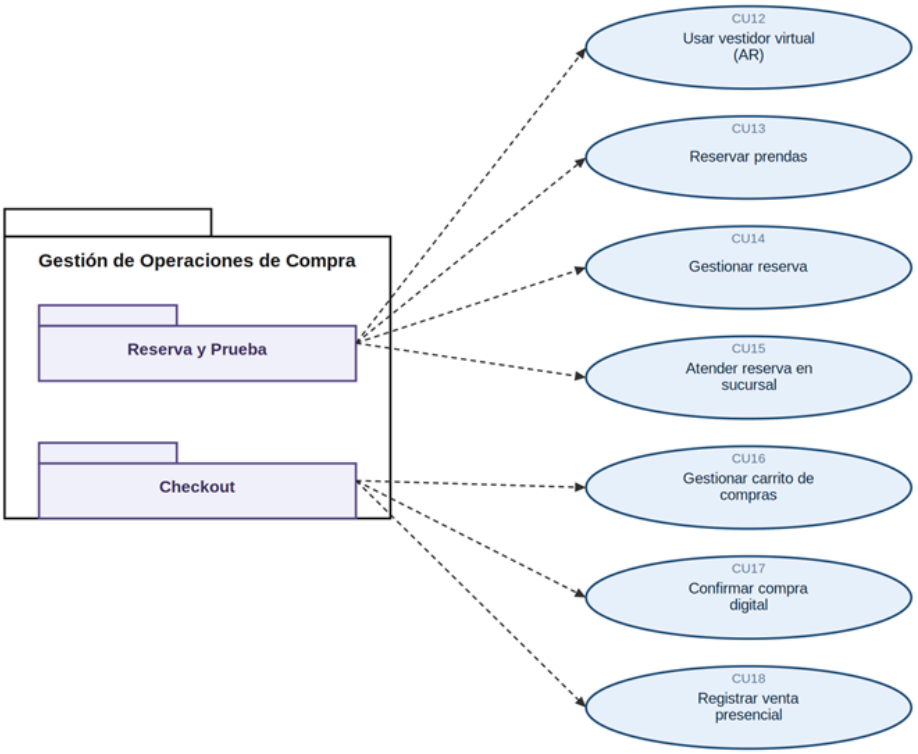
2.1.2. Relacionar paquetes y casos de uso



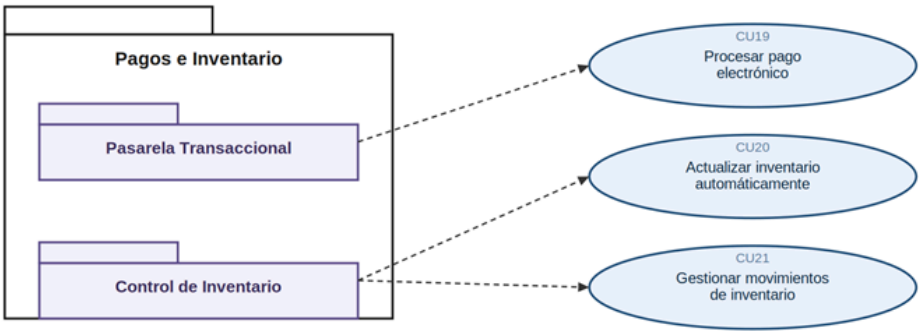
Relación de casos de uso Gestión de Identidad y Acceso



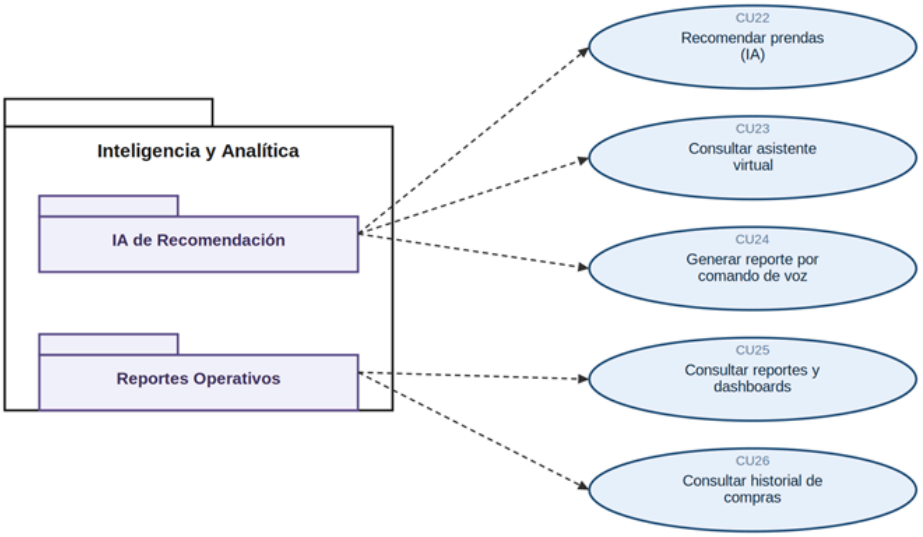
Relación de casos de uso Gestión de Catálogo y Sucursales



Relación de casos de uso Gestión de Operaciones de Compra

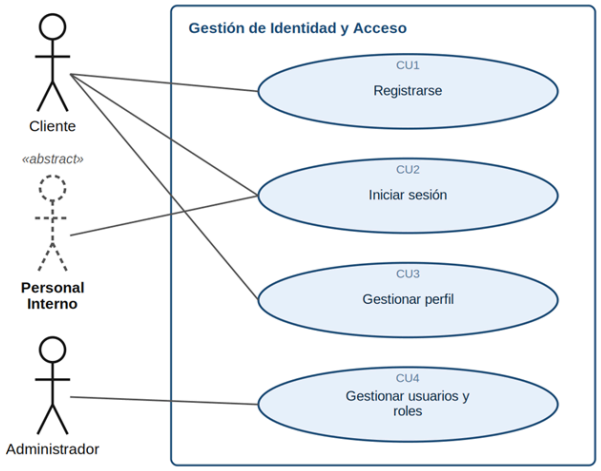


Relación de casos de uso Pagos e Inventario

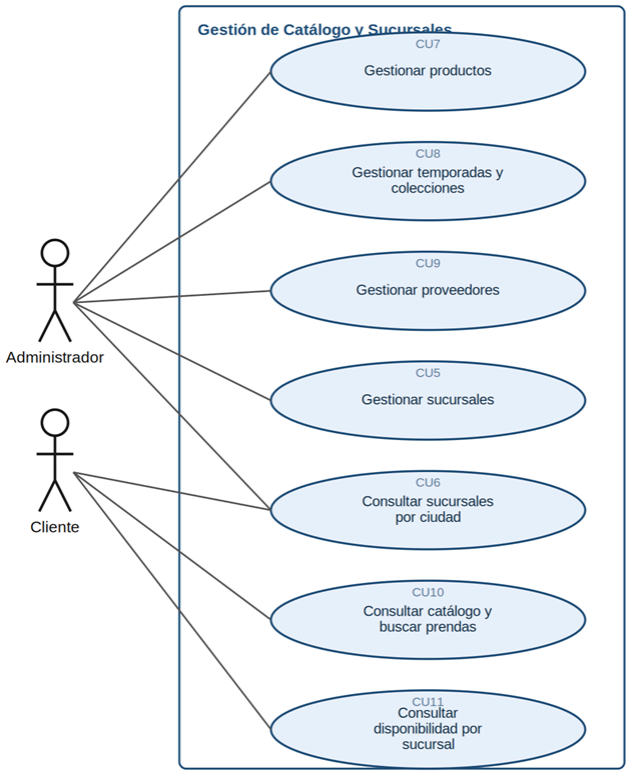


Relación de casos de uso Inteligencia y Analítica

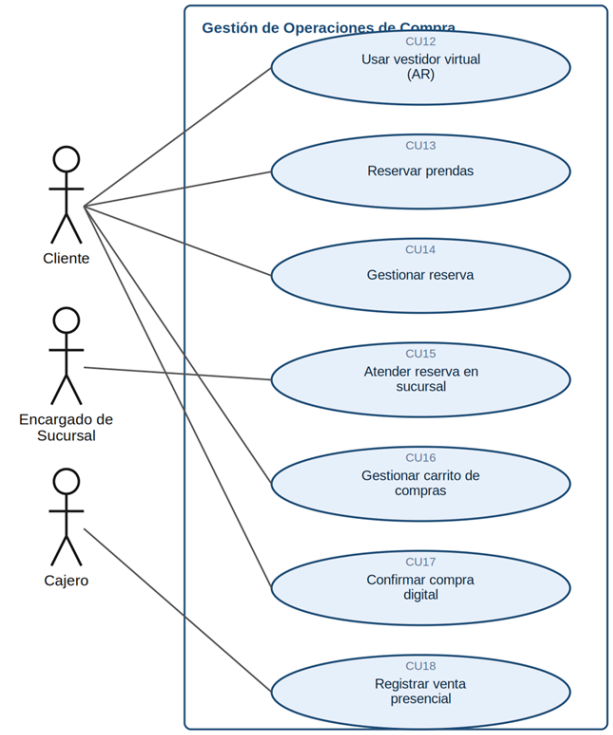
2.1.3. Vista de paquetes



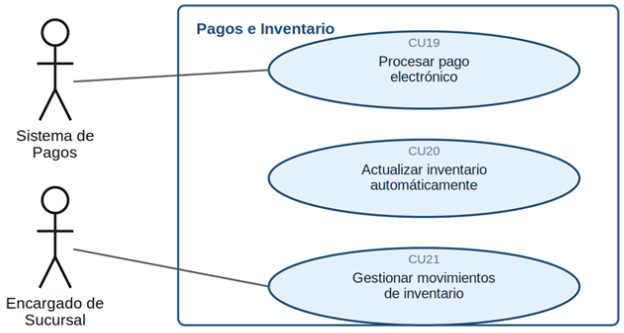
Vista de paquete Gestión de Identidad y Acceso



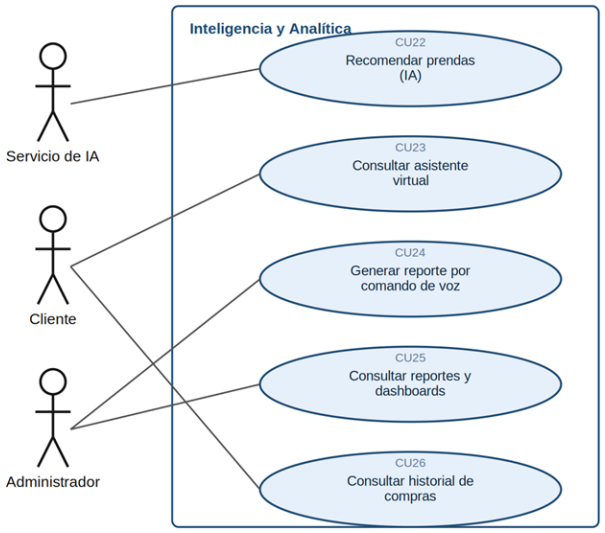
Vista de paquete Gestión de Catálogo y Sucursales



Vista de paquete Gestión de Operaciones de Compra



Vista de paquete Pagos e Inventario



Vista de paquete Inteligencia y Analítica

A.2. Diagramas de comunicación

Ciclo #1

Se presenta, para cada caso de uso del Ciclo 1, el diagrama de comunicación entre el actor, la vista (boundary), el controller y el modelo (entity), con la numeración de mensajes correspondiente al orden real de ejecución. A diferencia del ejemplo de referencia, no se fuerza que todo caso de uso combine crear/editar/eliminar en un mismo diagrama: solo CU5 y CU7 son operaciones CRUD genuinas en este ciclo; el resto son flujos de un solo paso, y representarlos como CRUD habría sido inventar pasos que no están en la tabla Detalle de la sección 1.3.

CU1. Registrarse

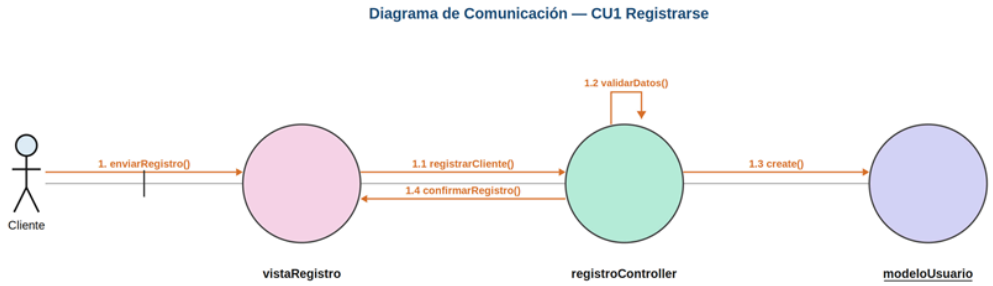


Diagrama de Comunicación — CU1 Registrarse

Flujo simple de un solo paso: no se combina con edición ni eliminación porque el registro de cliente es una operación independiente y distinta del CRUD administrativo de CU4 (Gestionar usuarios y roles, Ciclo 2).

CU2. Iniciar sesión

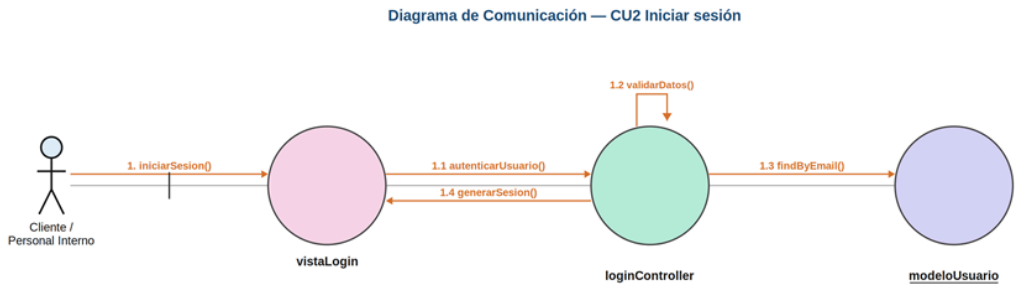


Diagrama de Comunicación — CU2 Iniciar sesión

El actor combina Cliente y Personal Interno porque comparten el mismo boundary y controller de autenticación (ver generalización de la sección 1.5).

CU3. Gestionar perfil

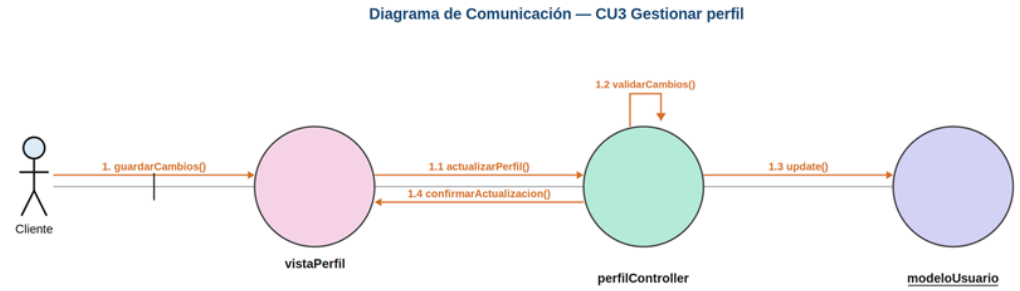


Diagrama de Comunicación — CU3 Gestionar perfil

Flujo simple: en este ciclo el cliente solo edita su propio perfil, sin eliminar ni crear otros usuarios.

CU5. Gestionar sucursales

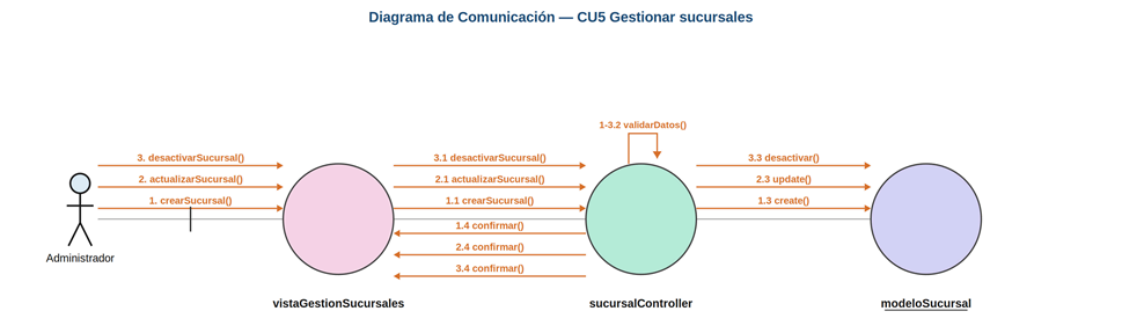


Diagrama de Comunicación — CU5 Gestionar sucursales

Único CRUD real de tres operaciones paralelas del Ciclo 1 (crear, actualizar, desactivar), de ahí que combine tres numeraciones (1/2/3) como en el patrón de la guía.

CU6. Consultar sucursales por ciudad

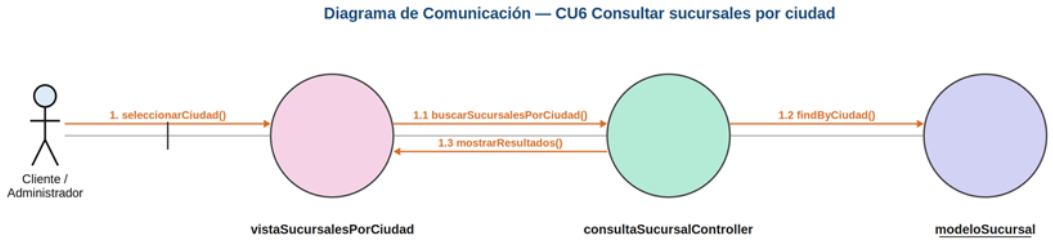


Diagrama de Comunicación — CU6 Consultar sucursales por ciudad

Flujo de solo lectura: se omite el loop de validación porque no hay datos que validar antes de consultar, solo un filtro de búsqueda.

CU7. Gestionar productos

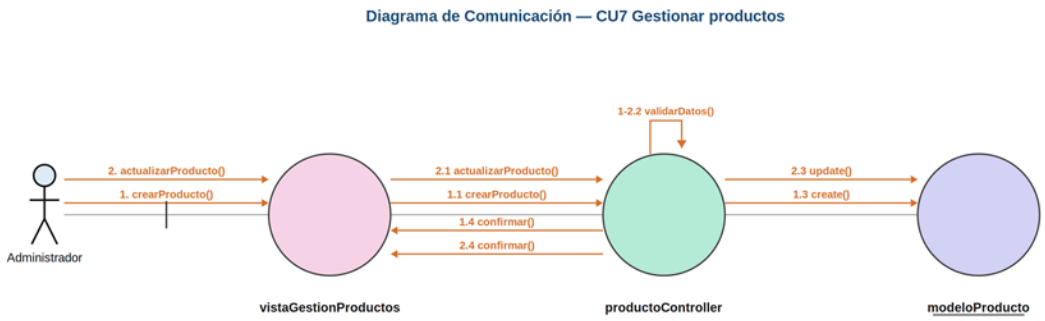


Diagrama de Comunicación — CU7 Gestionar productos

CRUD de dos operaciones (crear, actualizar); no incluye eliminar porque el enunciado no lo pide para este ciclo — los productos se desactivan, no se borran, y esa operación aún no está definida en el alcance del Ciclo 1.

CU10. Consultar catálogo y buscar prendas

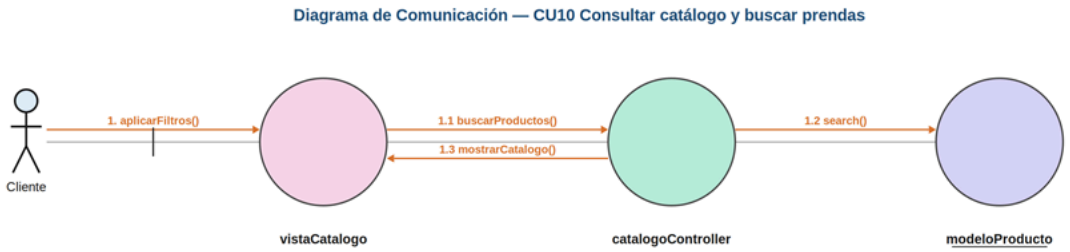


Diagrama de Comunicación — CU10 Consultar catálogo y buscar prendas

CICLO #2

CU4. Gestionar usuarios y roles

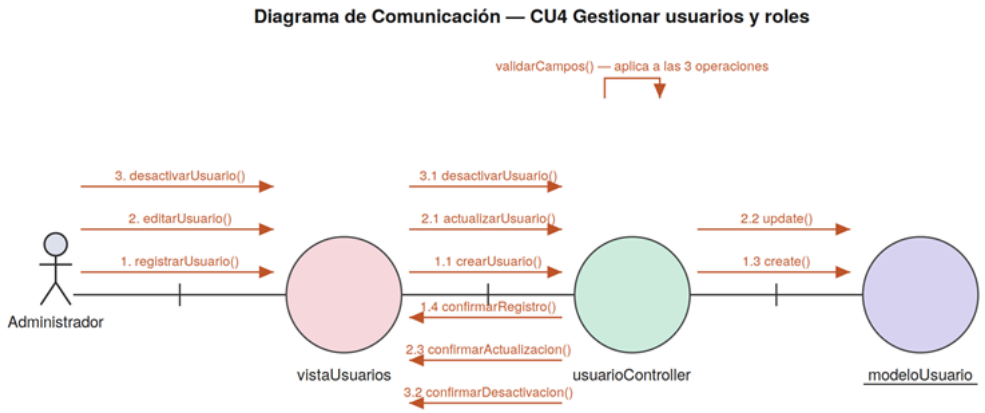


Diagrama de Comunicación — CU4 Gestionar usuarios y roles

Único CRUD real de tres operaciones paralelas del Ciclo 2 sobre la misma entidad Usuario ya usada en CU1/CU2/CU3 (crear personal interno, editar datos y rol, desactivar), de ahí que combine tres numeraciones (1/2/3) como en el patrón de CU5 del Ciclo 1. La asignación de rol se incluye dentro de la edición, no como operación aparte, porque el enunciado la describe como parte de "gestionar usuarios y roles" y no como un caso de uso independiente.

CU8. Gestionar temporadas y colecciones

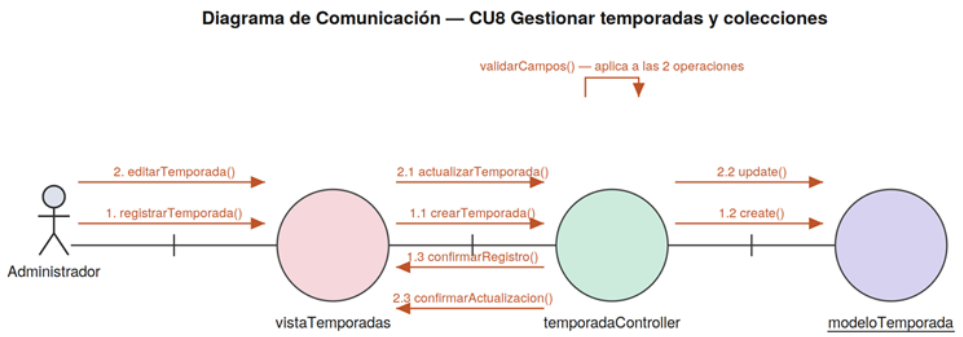


Diagrama de Comunicación — CU8 Gestionar temporadas y colecciones

CRUD de dos operaciones (crear, actualizar), igual criterio que CU7 en el Ciclo 1: no incluye eliminar porque el subpaquete Catálogo Base no define baja física de temporadas o colecciones, solo su desactivación, y esa operación todavía no está detallada en el alcance de este ciclo.

CU9. Gestionar proveedores

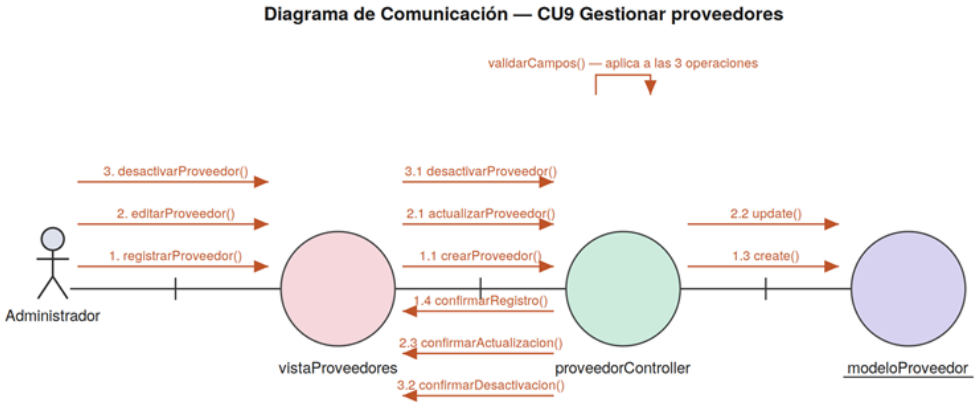


Diagrama de Comunicación — CU9 Gestionar proveedores

CRUD de tres operaciones (crear, editar, desactivar), igual criterio que CU5 en el Ciclo 1: un proveedor se desactiva y no se elimina, para conservar la trazabilidad de los productos que ya le fueron asociados en el Catálogo Base.

CU11. Consultar disponibilidad por sucursal

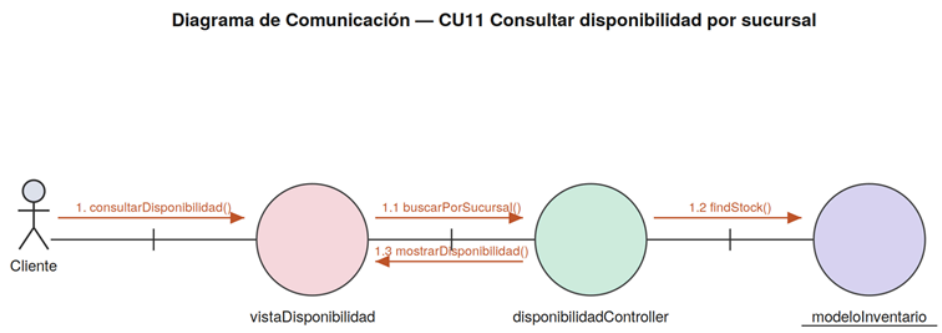


Diagrama de Comunicación — CU11 Consultar disponibilidad por sucursal

Flujo de solo lectura, igual criterio que CU6 en el Ciclo 1: se omite el loop de validación porque no hay datos que validar antes de consultar. El modelo usado es modeloInventario y no una clase propia del subpaquete Sucursales y Disponibilidad, porque la existencia real de una prenda por sucursal es la misma información que administrará CU21 (Control de Inventario); este caso de uso solo la lee.

CU13. Reservar prendas

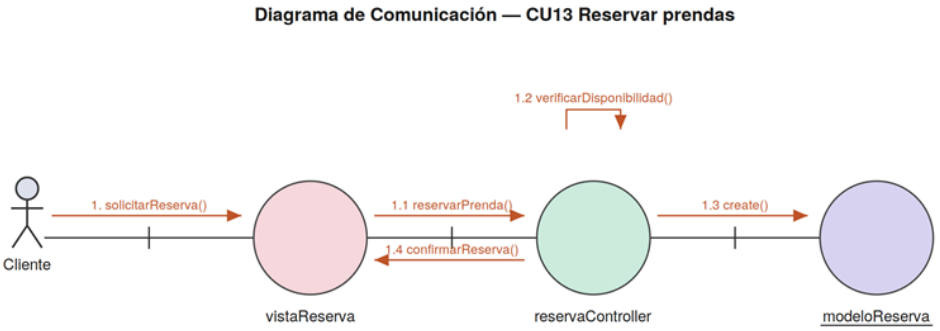


Diagrama de Comunicación — CU13 Reservar prendas

Flujo simple de un solo paso, con la verificación de disponibilidad modelada como una operación interna del controller (self-loop) y no como una segunda clase modelo: en este ciclo no se crea una clase Inventario propia para reservas, ya que el movimiento real de stock es responsabilidad del paquete Pagos e Inventario y se detallará en CU20/CU21 (Ciclo 3 y Ciclo 2, respectivamente).

CU14. Gestionar reserva (consultar / cancelar)

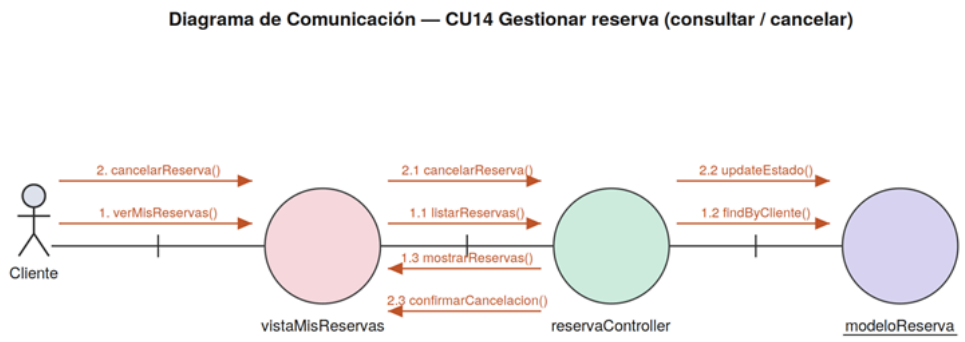


Diagrama de Comunicación — CU14 Gestionar reserva (consultar / cancelar)

Combina dos numeraciones (1/2) sobre el mismo par reservaController / modeloReserva ya definido en CU13, porque ambos casos de uso operan sobre la misma entidad Reserva y sería redundante modelar un controller distinto. La consulta (flujo 1) no requiere validación; la cancelación (flujo 2) sí la requeriría en el detalle de la sección 1.3 (solo se puede cancelar una reserva pendiente), pero se omite aquí el self-loop por simplicidad visual del diagrama de comunicación.

CU15. Atender reserva en sucursal

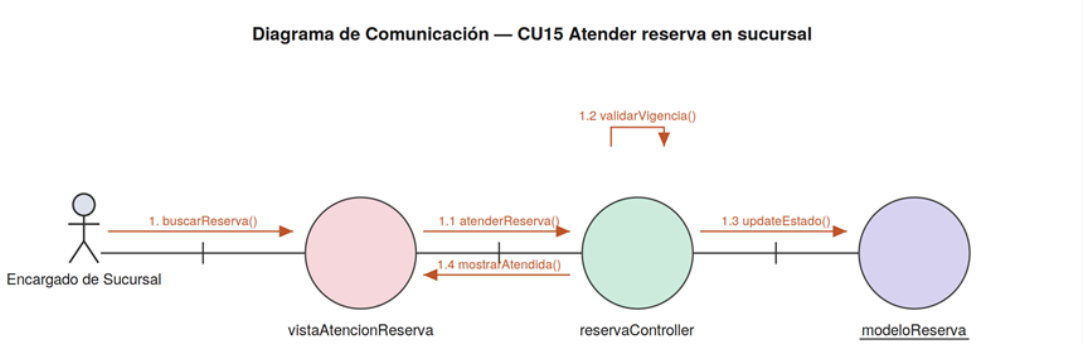


Diagrama de Comunicación — CU15 Atender reserva en sucursal

Reutiliza nuevamente reservaController y modeloReserva de CU13/CU14. El actor cambia a Encargado de Sucursal, y la validación interna corresponde a comprobar que la reserva exista y siga vigente antes de marcarla como atendida.

CU16. Gestionar carrito de compras

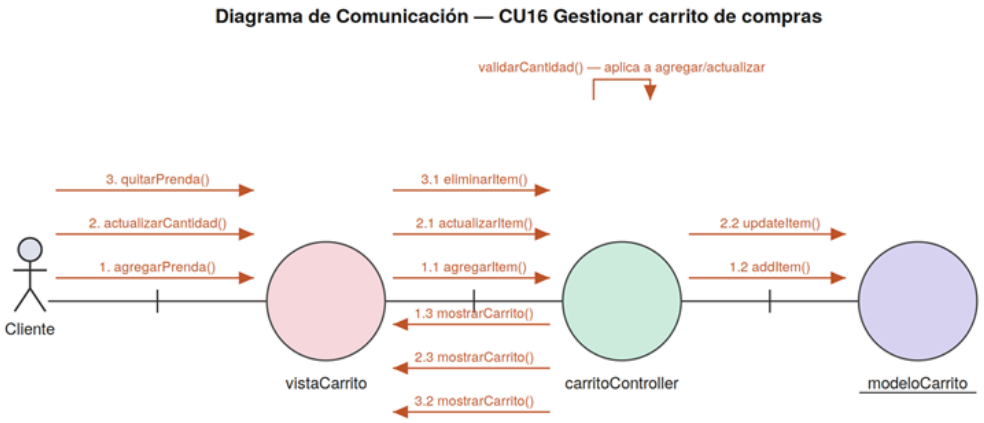


Diagrama de Comunicación — CU16 Gestionar carrito de compras

CRUD de tres operaciones (agregar, actualizar cantidad, quitar), mismo patrón que CU4/CU9. El carrito se modela como entidad propia (modeloCarrito) y no como parte de modeloReserva, porque conceptualmente pertenece al subpaquete Checkout y antecede a la compra digital de CU17 (Ciclo 3), no a la reserva física en sucursal.

CU21. Gestionar movimientos de inventario

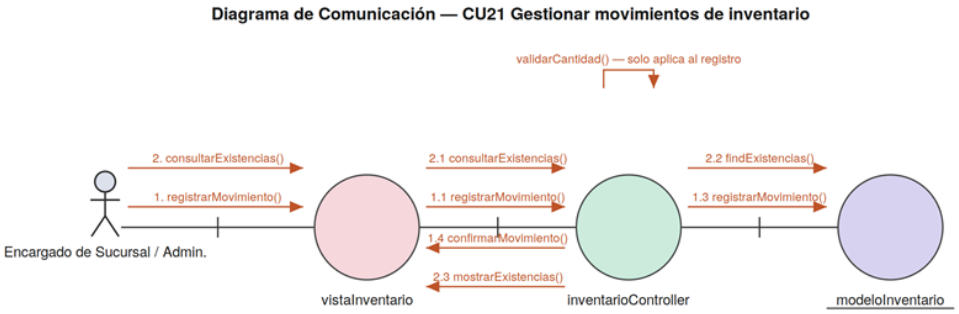


Diagrama de Comunicación — CU21 Gestionar movimientos de inventario

Combina un flujo de escritura (1: registrar movimiento de ingreso o merma, con validación de cantidad) y un flujo de solo lectura (2: consultar existencias y agotados, sin validación), ambos sobre modeloInventario — la misma entidad que CU11 consulta en modo lectura. Se numeran por separado (1.x / 2.x) en vez de forzarlos a compartir una sola secuencia, siguiendo el mismo criterio de CU5 y CU9 para operaciones genuinamente distintas sobre una entidad compartida.

CICLO #3

CU12. Usar vestidor virtual (AR)

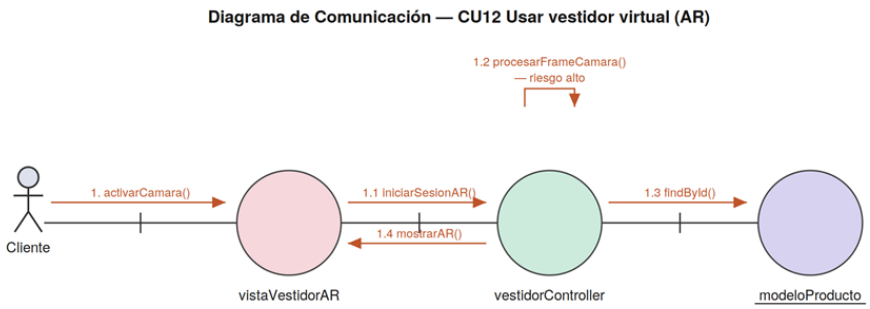


Diagrama de Comunicación — CU12 Usar vestidor virtual (AR)

Flujo simple de un solo paso. Se marca explícitamente el self-loop procesarFrameCamara() como de riesgo alto, consistente con la matriz de priorización: es el procesamiento en tiempo real más exigente del proyecto y el que más probablemente requiera ajustar el diseño una vez que empiece la implementación.

CU17. Confirmar compra digital

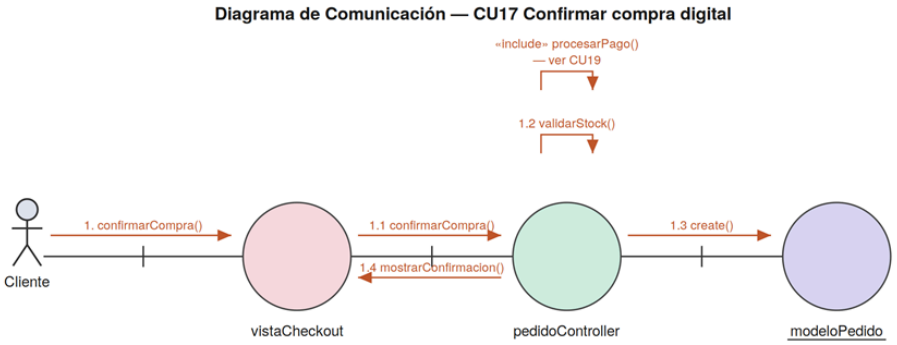


Diagrama de Comunicación — CU17 Confirmar compra digital

El procesamiento del pago no se dibuja como una llamada más dentro de este diagrama: el propio documento indica que CU19 (Procesar pago electrónico) está “incluido en CU17/CU18”, por lo que aquí solo se referencia con la relación «include» y se remite al diagrama propio de CU19 para el detalle de esa comunicación.

CU18. Registrar venta presencial

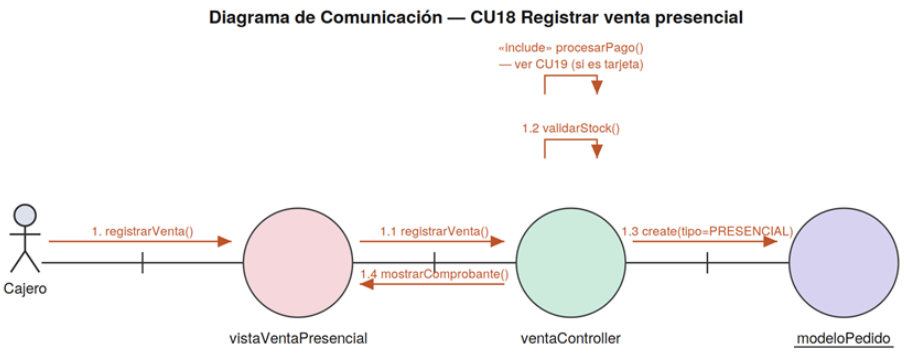


Diagrama de Comunicación — CU18 Registrar venta presencial

Mismo patrón que CU17, con el actor correcto según el catálogo de casos de uso: Cajero, no Administrador ni Encargado de Sucursal. modeloPedido se reutiliza de CU17, distinguiendo el registro mediante tipo=PRESENCIAL. El pago solo se incluye condicionalmente (si el cliente paga con tarjeta); si paga en efectivo, CU19 no se dispara.

CU19. Procesar pago electrónico

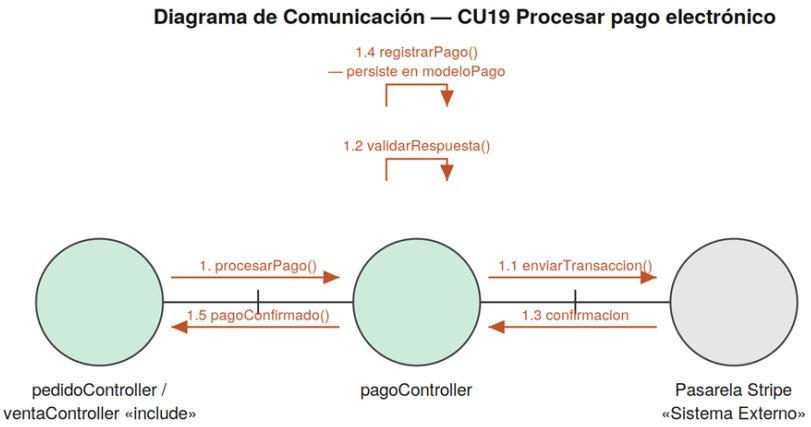


Diagrama de Comunicación — CU19 Procesar pago electrónico

A diferencia de todos los diagramas anteriores, este no inicia con un actor humano: el documento lo define como “incluido en CU17/CU18”, por lo que el primer participante es el controller que lo invoca (pedidoController o ventaController), no una persona. Por la misma razón no existe una “vista” propia para este caso de uso. El registro del pago se modela como un self-loop en vez de una clase modeloPago separada en la cadena principal, para no forzar una dependencia no adyacente entre pagoController y la pasarela externa.

CU20. Actualizar inventario automáticamente

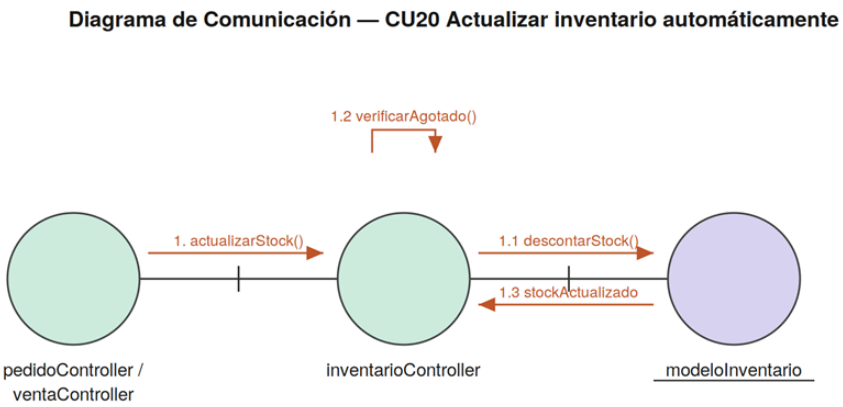


Diagrama de Comunicación — CU20 Actualizar inventario automáticamente

Tampoco lo dispara un actor humano: el catálogo de casos de uso define su actor como “Sistema”, activado automáticamente al confirmarse una venta (CU17/CU18). Reutiliza inventarioController y modeloInventario ya definidos en CU21 del Ciclo 2, ahora con la operación de descuento de stock que ese caso de uso no necesitaba.

CU22. Recomendar prendas (IA)

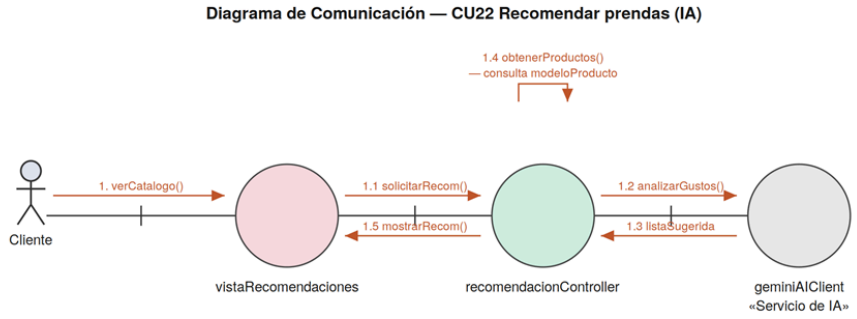


Diagrama de Comunicación — CU22 Recomendar prendas (IA)

El catálogo de casos de uso define el actor principal como “Servicio de IA”; aquí se modela con el Cliente como disparador natural (ver el catálogo activa la solicitud de recomendaciones) y geminiAIClient como participante activo que efectivamente genera la sugerencia. La consulta a modeloProducto se muestra como self-loop para mantener la cadena principal lineal.

CU23. Consultar asistente virtual

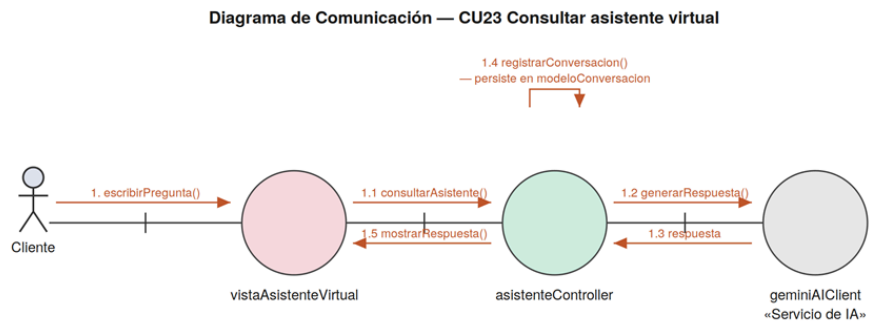


Diagrama de Comunicación — CU23 Consultar asistente virtual (chatbot)

Reutiliza geminiAIClient de CU22, mismo servicio de IA con distinta finalidad (chat conversacional en vez de recomendación proactiva). El registro de la conversación se modela igualmente como self-loop sobre asistenteController.

CICLO #4

CU24. Generar reporte por comando de voz

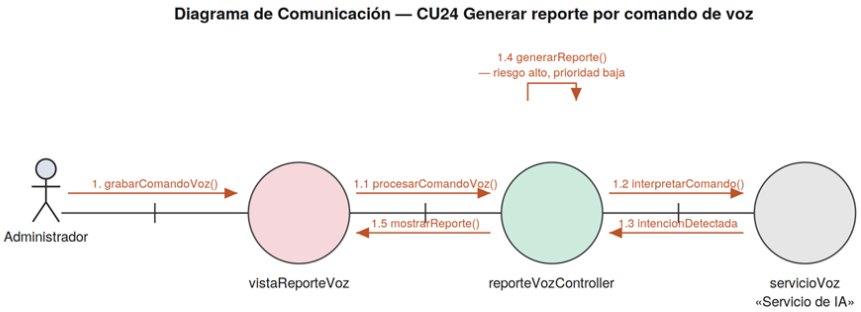


Diagrama de Comunicación — CU24 Generar reporte por comando de voz

Extensión opcional con prioridad baja y riesgo alto según la matriz de priorización del proyecto; se marca explícitamente en el self-loop generarReporte(), que es el paso que traduce la intención detectada por voz en una consulta real sobre pedidos/inventario. Por tratarse de una extensión de baja prioridad, es el primer candidato a recortar si el tiempo de implementación se ajusta.

CU25. Consultar reportes y dashboards

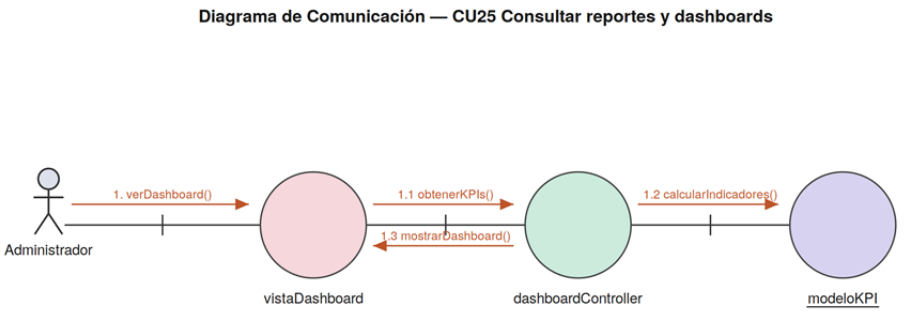


Diagrama de Comunicación — CU25 Consultar reportes y dashboards

Flujo de solo lectura, mismo criterio que CU6/CU11 en ciclos anteriores: sin self-loop de validación. modeloKPI representa una vista agregada sobre pedidos, inventario y reservas (no una tabla física nueva del script de la sección 3.3); se modela como una única clase para no forzar una dependencia no adyacente entre dashboardController y tres modelos distintos.

CU26. Consultar historial de compras

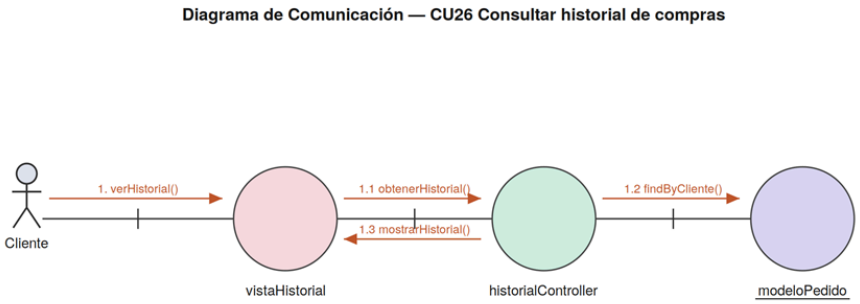


Diagrama de Comunicación — CU26 Consultar historial de compras

Flujo de solo lectura sobre modeloPedido, ya definido en el Ciclo 3 (CU17/CU18). No se crea una clase nueva: el historial del cliente es simplemente una consulta filtrada por cliente_id sobre la misma tabla pedidos del script SQL.

A.3. Diagramas de análisis de una clase

Ciclo #1

Para cada caso de uso del Ciclo 1 se detalla la clase «Actor», la clase vista (atributos = campos que el actor ve y llena en pantalla; operaciones = acciones de interfaz), la clase controller (lógica de validación y coordinación) y la o las clases modelo (atributos privados

persistentes y operaciones de acceso a datos, que en el Diseño de Datos se traducirán en las tablas y consultas correspondientes).

CU1. Registrarse

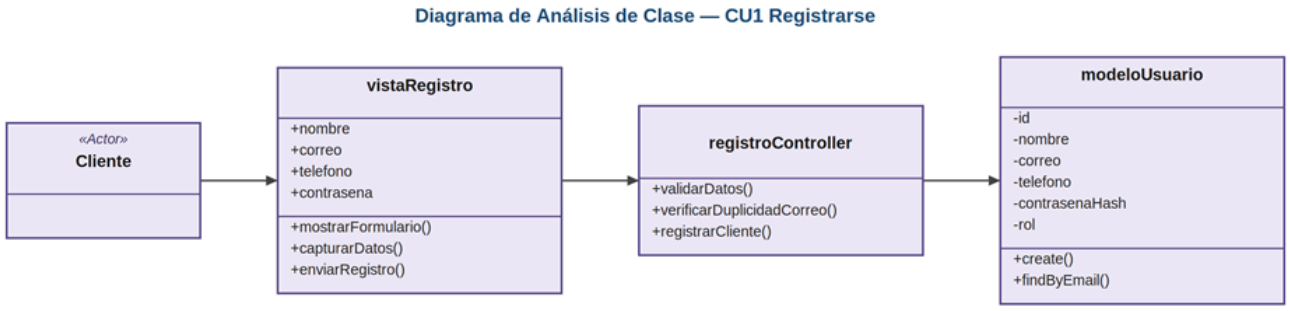


Diagrama de Análisis de Clase CU1 Registrarse

CU2. Iniciar sesión

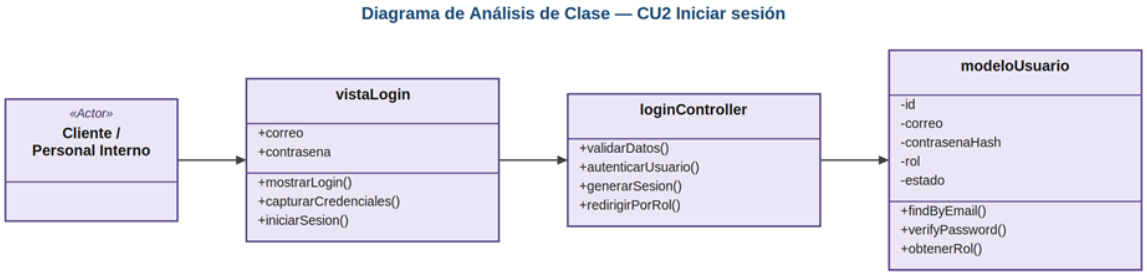


Diagrama de Análisis de Clase CU2 Iniciar sesión

CU3. Gestionar perfil

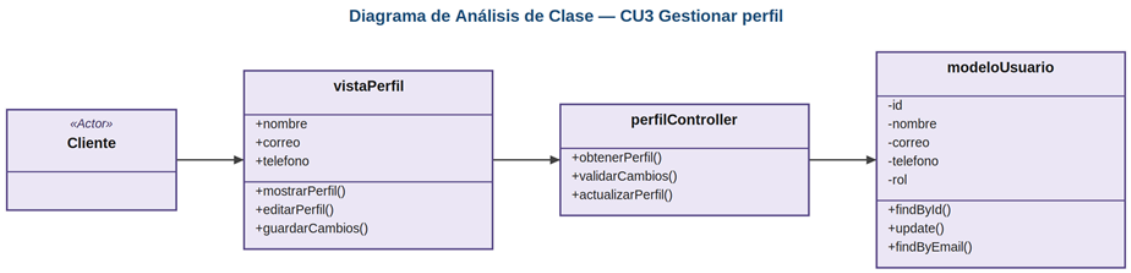


Diagrama de Análisis de Clase CU3 Gestionar perfil

CU5. Gestionar sucursales

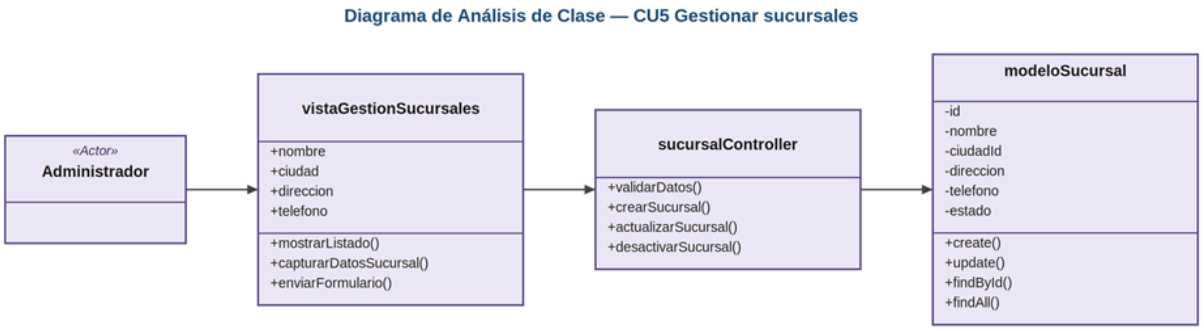


Diagrama de Análisis de Clase CU5 Gestionar sucursales

CU6. Consultar sucursales por ciudad

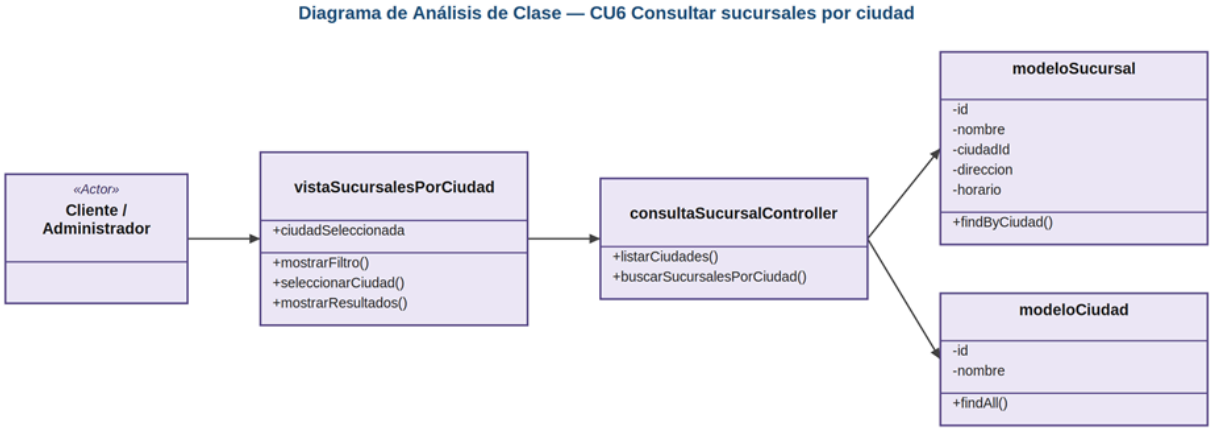


Diagrama de Análisis de Clase CU6 Consultar sucursales por ciudad

CU7. Gestionar productos

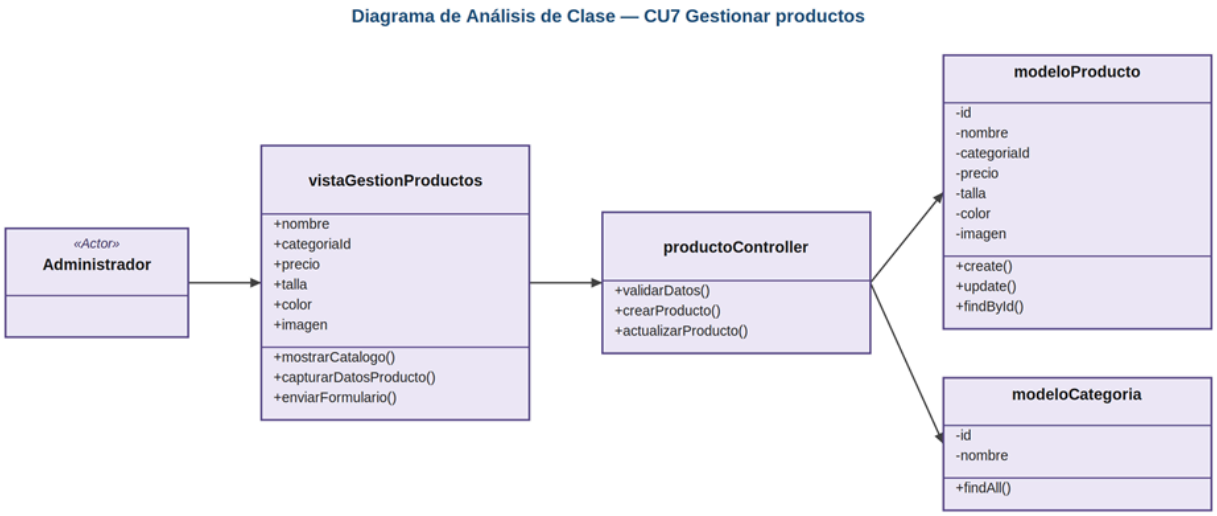


Diagrama de Análisis de Clase CU7 Gestionar productos

CU10. Consultar catálogo y buscar prendas

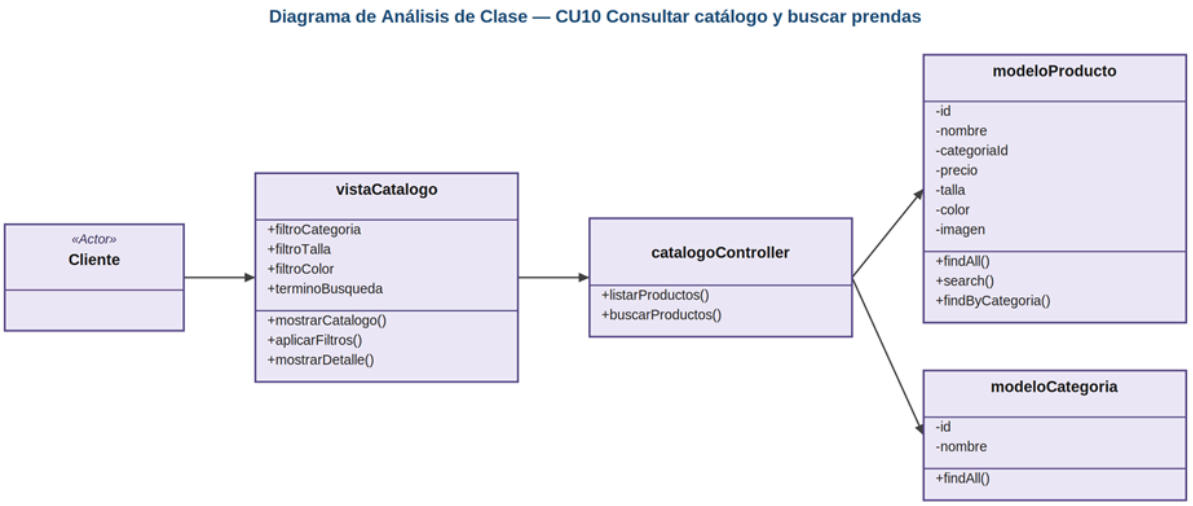


Diagrama de Análisis de Clase CU10 Consultar catálogo y buscar prendas

Ciclo #2

CU4. Gestionar usuarios y roles

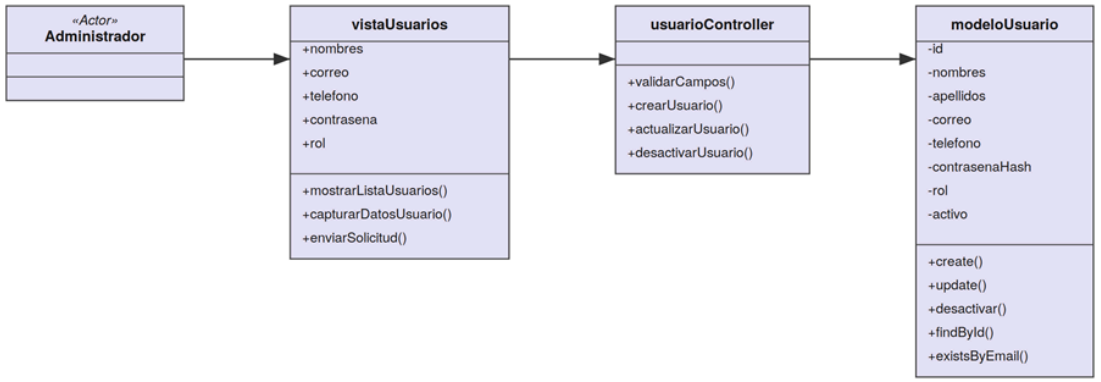


Diagrama de Análisis de Clase CU4 Gestionar usuarios y roles

modeloUsuario es la misma clase de CU1–CU3, ampliada aquí con el atributo -activo (usado por desactivarUsuario) y las operaciones update() y desactivar(), que no eran necesarias en el Ciclo 1.

CU8. Gestionar temporadas y colecciones

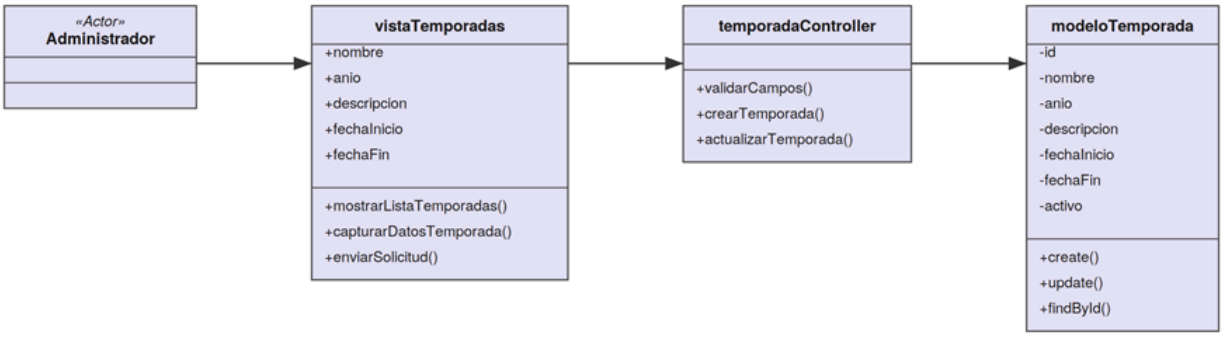


Diagrama de Análisis de Clase CU8 Gestionar temporadas y colecciones

modeloTemporada no define delete() ni el atributo -activo con operación de baja, en línea con el CRUD de solo dos operaciones ya justificado en el diagrama de comunicación de este mismo caso de uso.

CU9. Gestionar proveedores

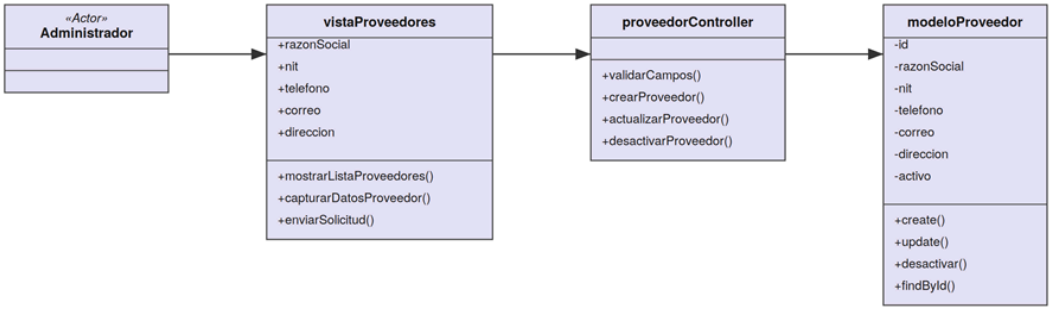


Diagrama de Análisis de Clase CU9 Gestionar proveedores

Misma estructura de CU5 (Ciclo 1) y CU4: atributo -activo y operación desactivar() en vez de delete(), para no perder la relación con productos ya asociados al proveedor.

CU11. Consultar disponibilidad por sucursal

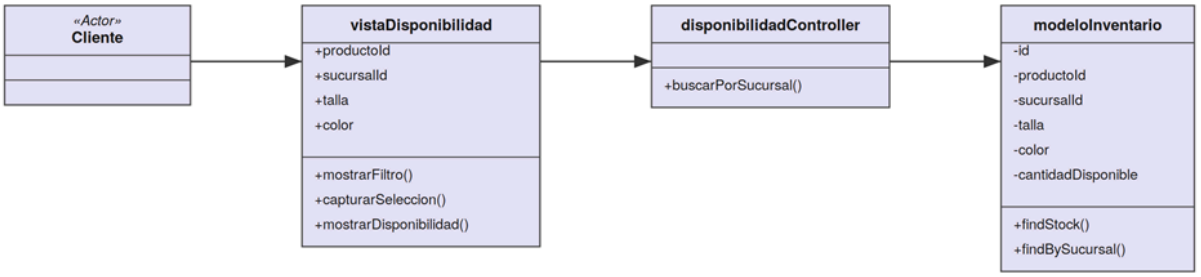


Diagrama de Análisis de Clase CU11 Consultar disponibilidad por sucursal

disponibilidadController expone una única operación de solo lectura. modeloInventario es la misma clase que administrará CU21; aquí solo se muestran las operaciones de consulta (findStock, findBySucursal) que este caso de uso necesita.

CU13. Reservar prendas

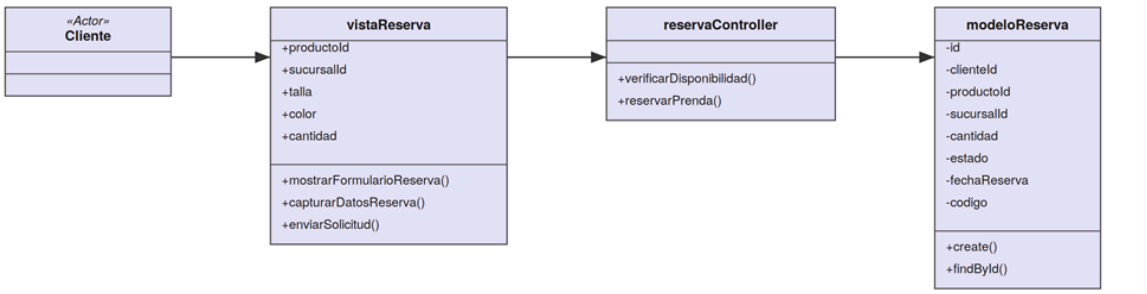


Diagrama de Análisis de Clase CU13 Reservar prendas

No se agrega una clase Inventario al diagrama: verificarDisponibilidad() se resuelve como lógica interna de reservaController, tal como se justificó en el diagrama de comunicación de este caso de uso.

CU14. Gestionar reserva (consultar / cancelar)

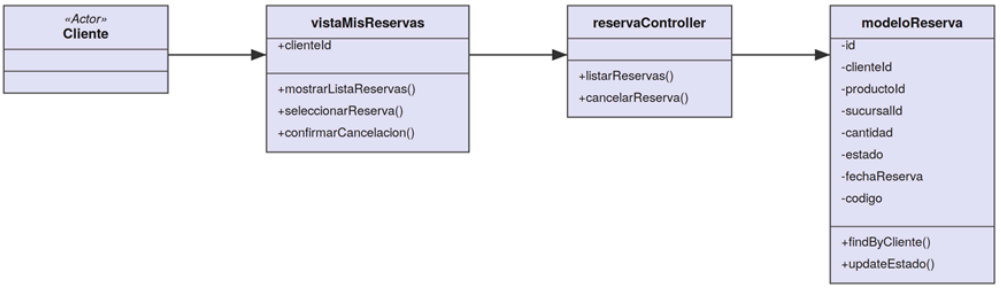


Diagrama de Análisis de Clase CU14 Gestionar reserva (consultar / cancelar)

reservaController y modeloReserva son las mismas clases de CU13, con dos operaciones adicionales (listarReservas, cancelarReserva / findByCliente, updateEstado) en vez de duplicar el controller.

CU15. Atender reserva en sucursal

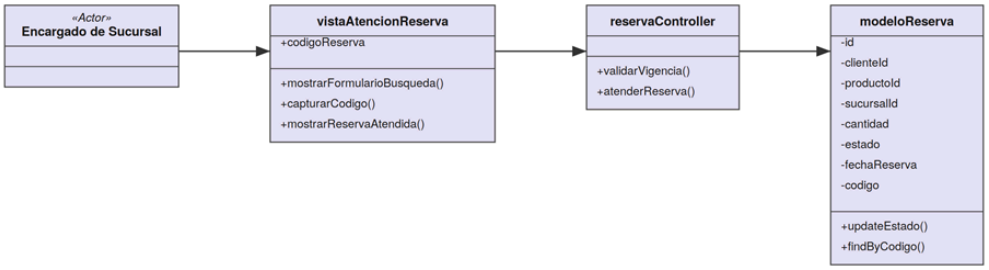


Diagrama de Análisis de Clase CU15 Atender reserva en sucursal

Tercer y último caso de uso que reutiliza reservaController / modeloReserva; cambia el actor a Encargado de Sucursal y agrega findByCodigo(), necesario para localizar la reserva física en el mostrador.

CU16. Gestionar carrito de compras

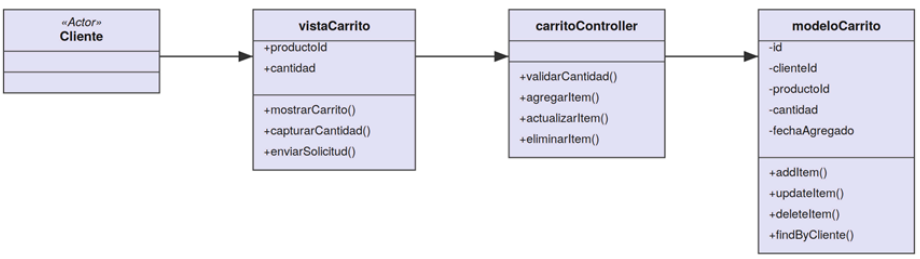


Diagrama de Análisis de Clase CU16 Gestionar carrito de compras

modeloCarrito es una clase propia y no una extensión de modeloReserva: pertenece al subpaquete Checkout y antecede a la compra digital de CU17 (Ciclo 3), no a la reserva física en sucursal.

CU21. Gestionar movimientos de inventario

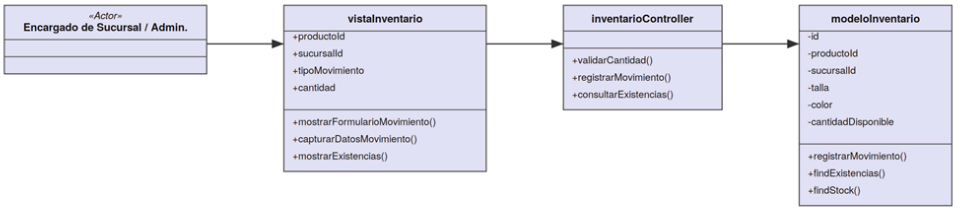
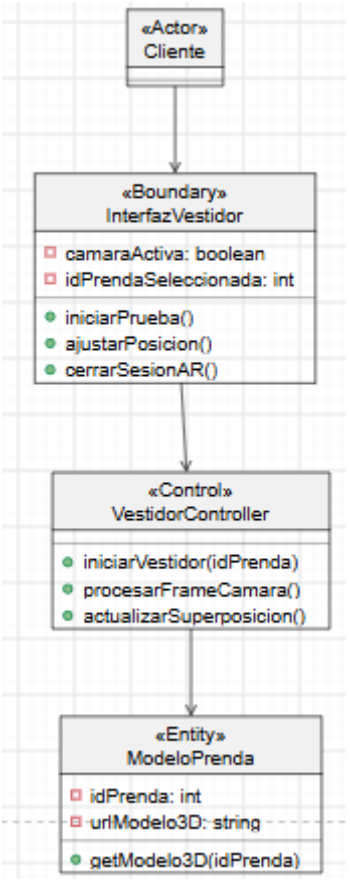


Diagrama de Análisis de Clase CU21 Gestionar movimientos de inventario

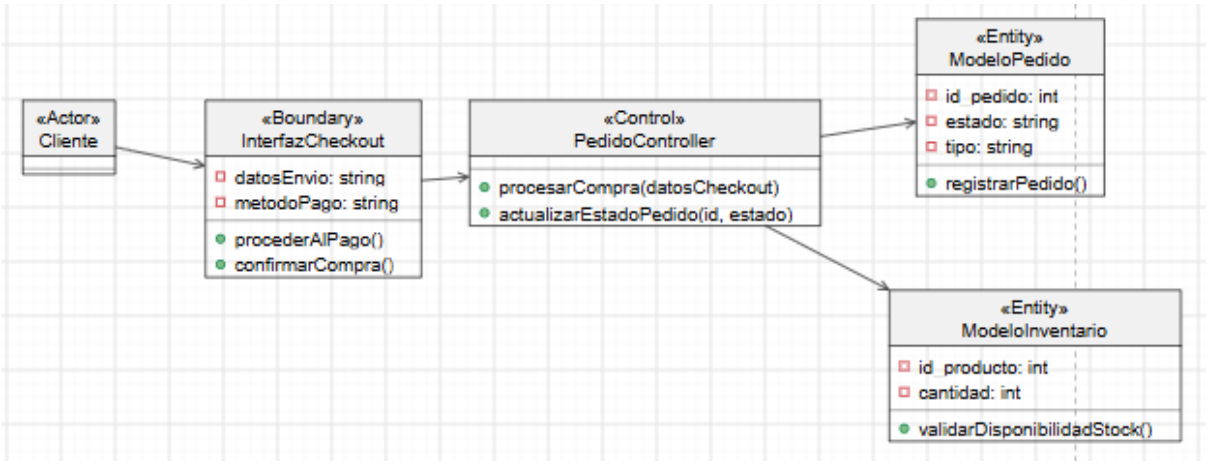
modeloInventario es la misma clase de CU11, ahora con las operaciones de escritura (registrarMovimiento) que ese caso de uso no necesitaba por ser de solo lectura.

CICLO #3

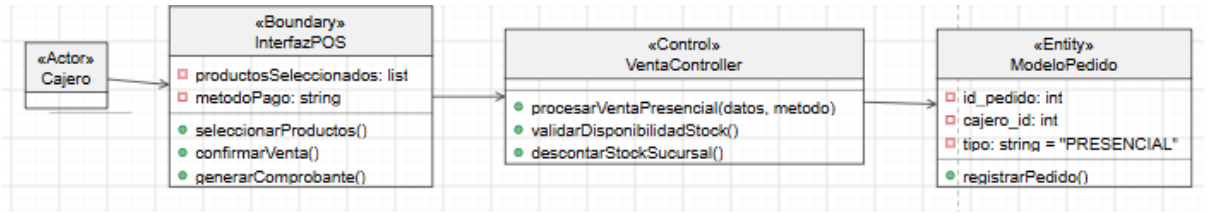
CU12 — Usar vestidor virtual (AR)



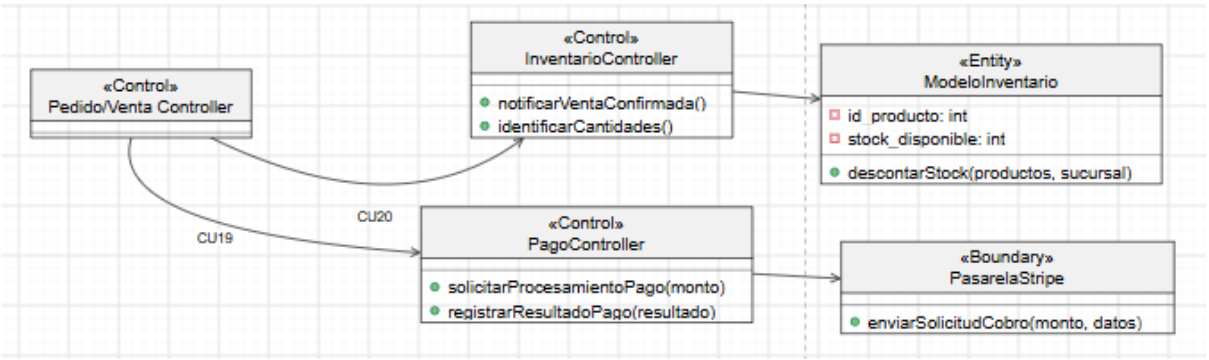
CU17 — Confirmar compra digital



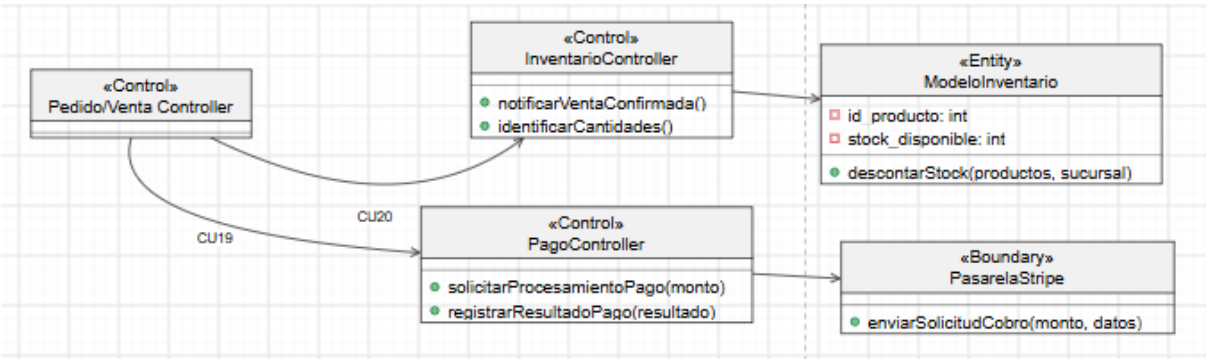
CU18 — Registrar venta presencial



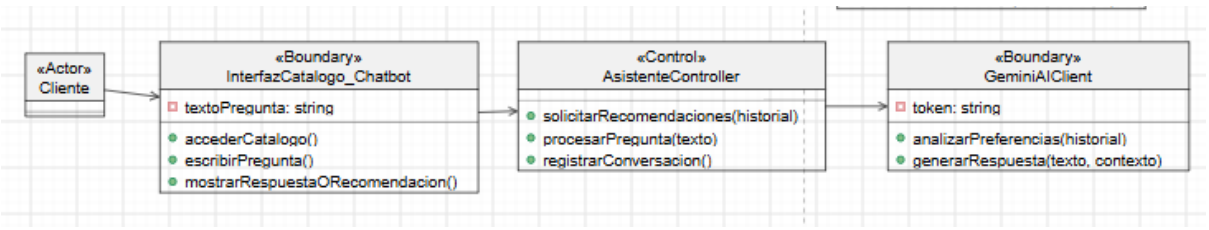
CU19 — Procesar pago electrónico



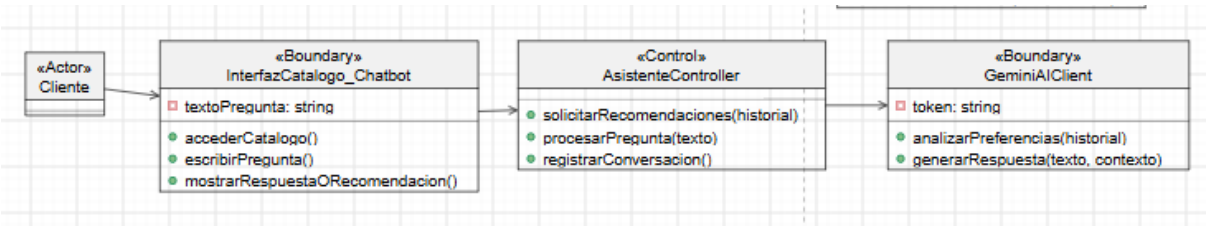
CU20 — Actualizar inventario automáticamente



CU22 — Recomendar prendas (IA)

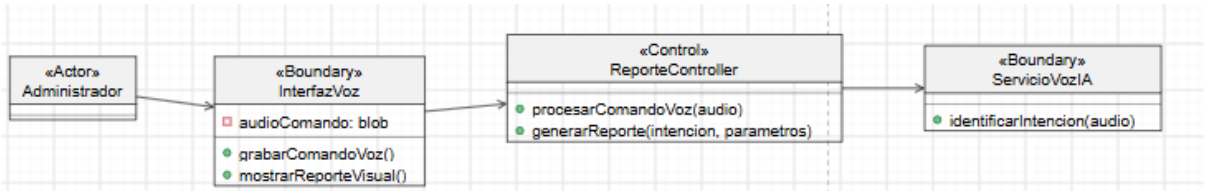


CU23 — Consultar asistente virtual (chatbot)

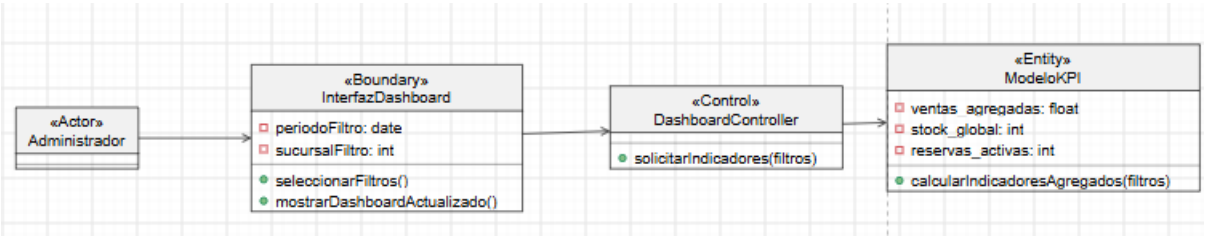


CICLO #4

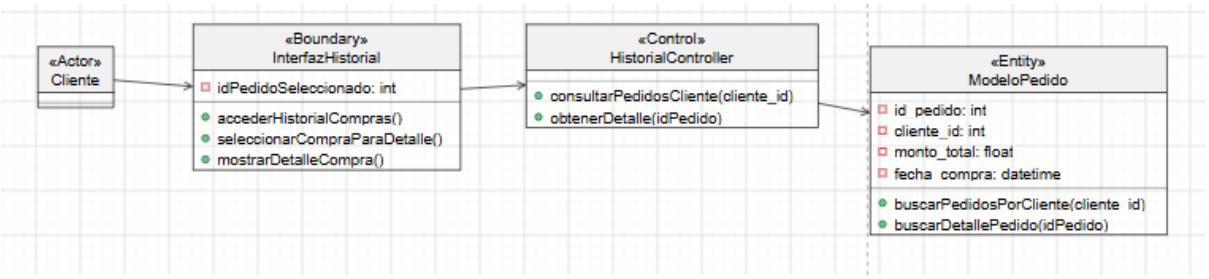
CU24 — Generar reporte por comando de voz (extensión opcional, prioridad baja)



CU25 — Consultar reportes y dashboards



CU26 — Consultar historial de compras



A.4. Análisis de paquetes

El objetivo de este análisis es determinar el grado de acoplamiento entre los paquetes del sistema, considerando ahora los 5 paquetes de todo el proyecto (26 casos de uso, 4 ciclos)

definidos en la sección 2.1, y no únicamente los del Ciclo 1. En FashionStore, los paquetes no son islas aisladas: la información fluye de uno a otro para completar procesos de negocio completos, como reservar una prenda o generar un reporte de ventas.

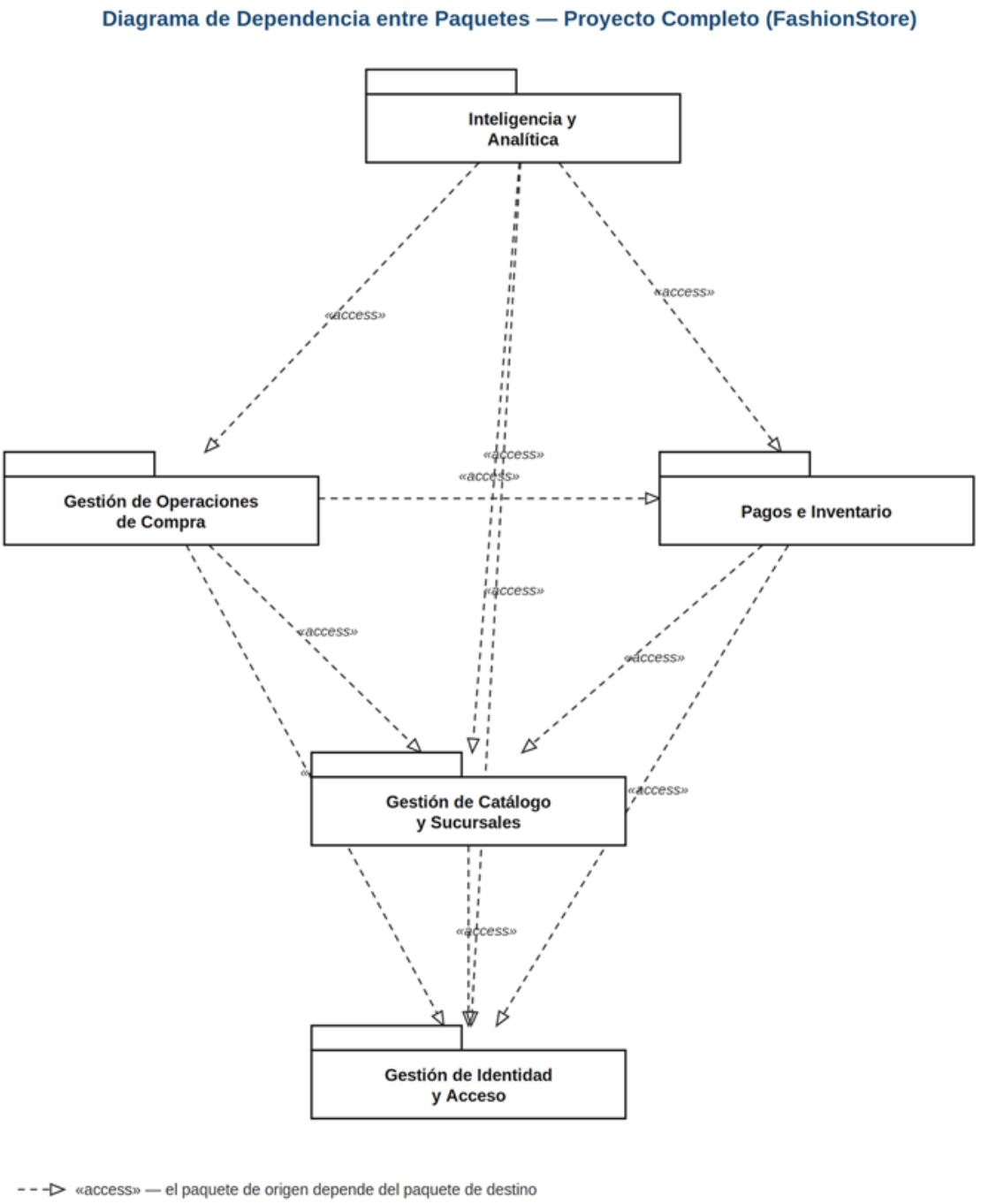


Diagrama de Dependencia entre Paquetes — Proyecto Completo (FashionStore)

Análisis Funcional de las Dependencias

1. Dependencia con Gestión de Identidad y Acceso: los cuatro paquetes restantes (Catálogo y Sucursales, Operaciones de Compra, Pagos e Inventario, e Inteligencia y Analítica) dependen

de este paquete. Antes de ejecutar cualquier caso de uso restringido desde gestionar un producto (CU7) hasta generar un reporte (CU25) el sistema debe verificar la identidad y el rol del actor autenticado.

2. Gestión de Operaciones de Compra → Gestión de Catálogo y Sucursales: para reservar una prenda (CU13) o confirmar una compra (CU17), el paquete de operaciones debe conocer qué productos existen y en qué sucursal están disponibles (CU10, CU11); sin ese contexto, la compra no tendría sustento.

3. Gestión de Operaciones de Compra → Pagos e Inventario: al confirmar la compra digital (CU17) o registrar la venta presencial (CU18), se invoca el procesamiento del pago (CU19, incluido) y se dispara la actualización automática del inventario (CU20).

4. Pagos e Inventario → Gestión de Catálogo y Sucursales: cada movimiento de inventario (CU20, CU21) pertenece a un producto específico dentro de una sucursal específica; sin esas dos referencias, un registro de stock no tendría contexto.

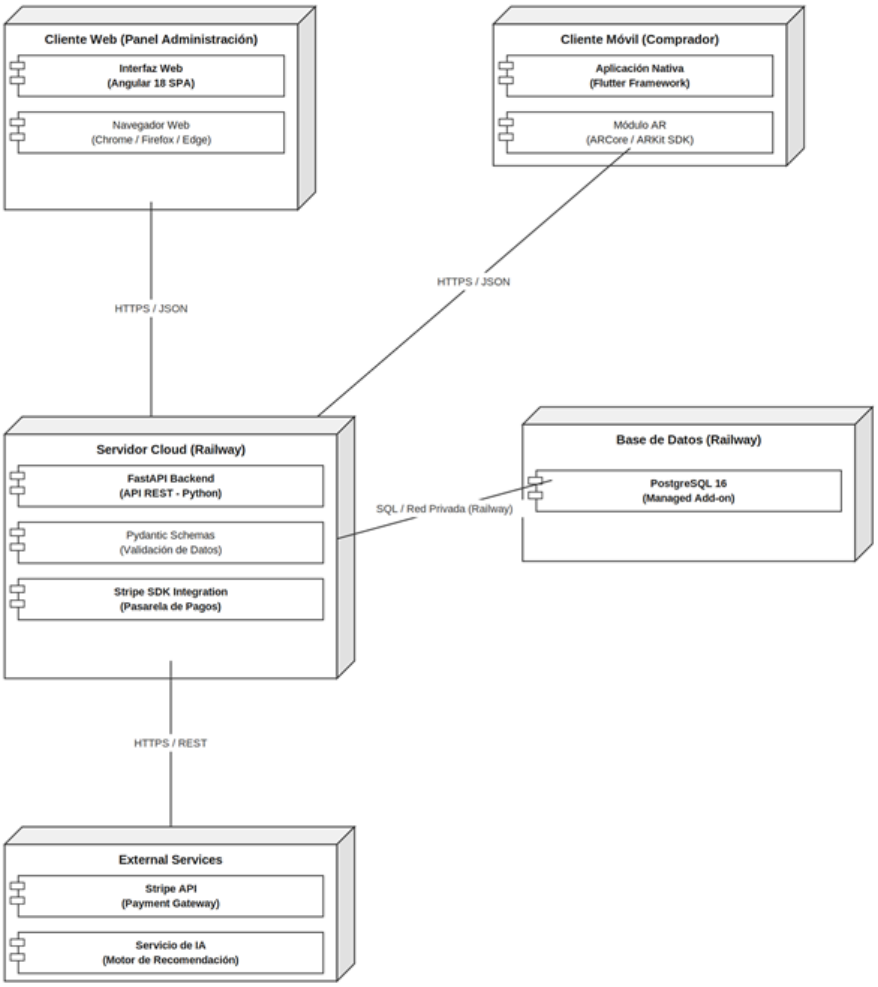
5. Inteligencia y Analítica → Gestión de Catálogo y Sucursales: recomendar prendas (CU22) exige conocer el catálogo disponible; sin datos de producto, no hay nada que recomendar.

6. Inteligencia y Analítica → Gestión de Operaciones de Compra: el historial de compras del cliente (CU26) y los reportes de ventas (CU25) se construyen a partir de los datos que genera este paquete al confirmarse cada compra.

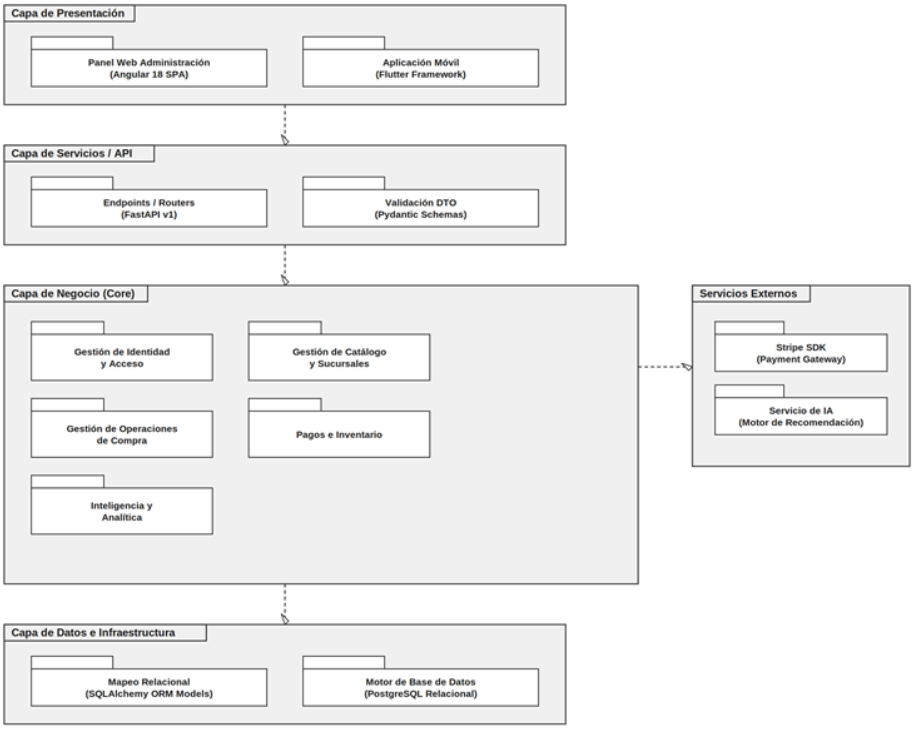
7. Inteligencia y Analítica → Pagos e Inventario: los reportes de inventario (CU25) leen directamente los movimientos y niveles de stock que gestiona este paquete.

A.5. Diseño de arquitectura

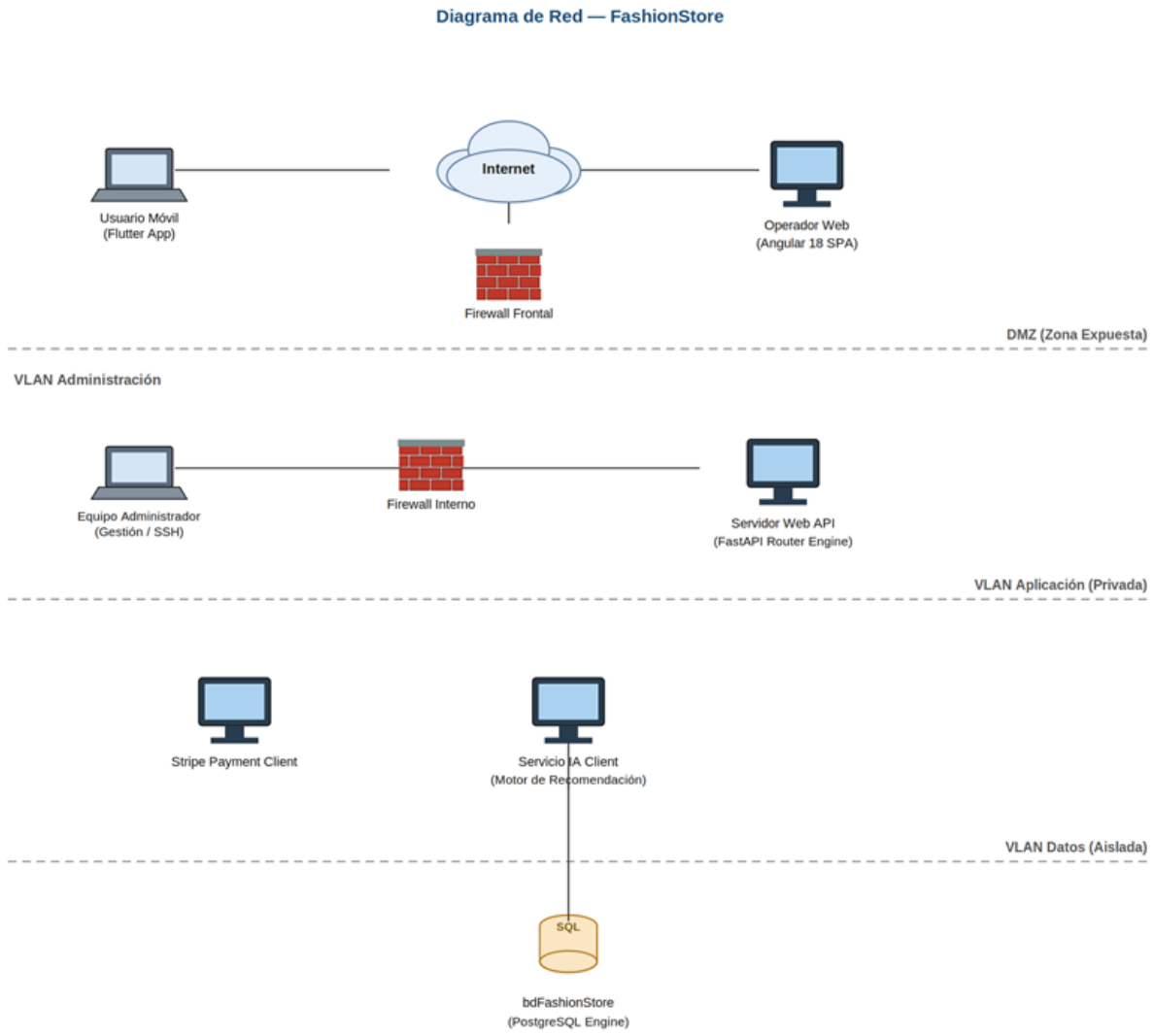
3.1.1. Diseño físico (Diagrama de despliegue)



3.1.2. Diseño Lógico (Diagrama de Paquete)



3.1.3. Diagrama de Red



A.6. Diagramas de secuencia

Ciclo #1

CU1. Registrarse

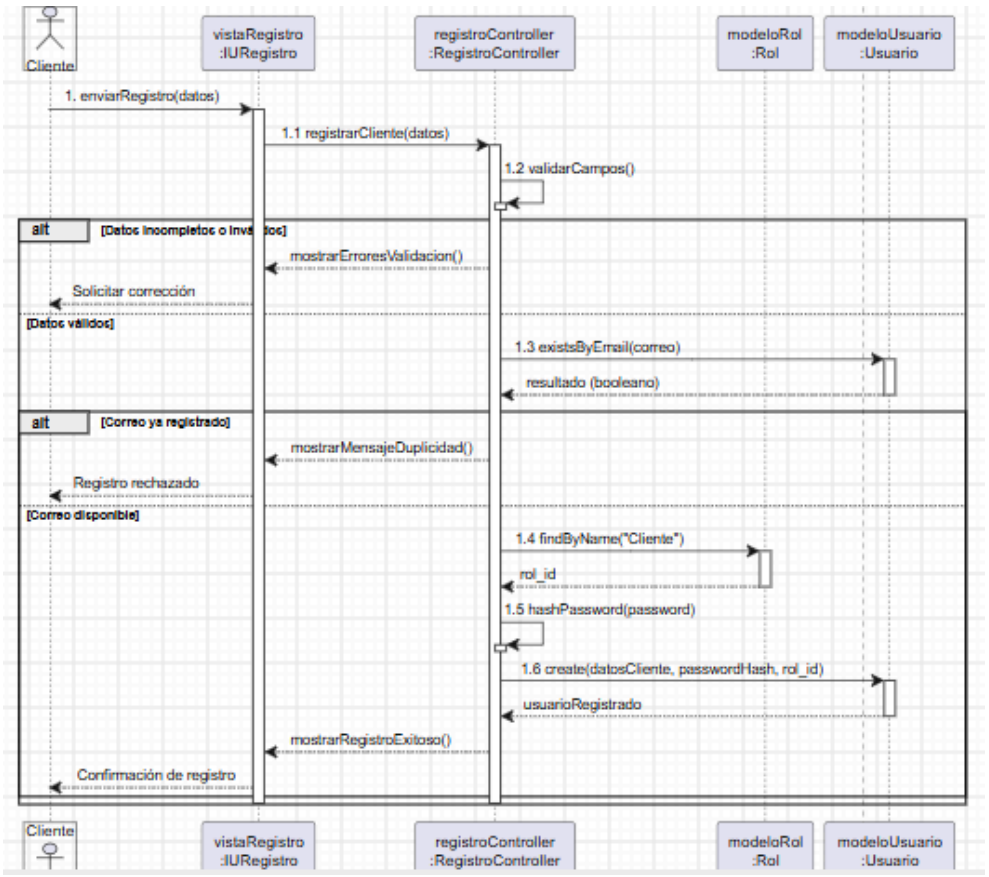


Figura 6. Diagrama de Secuencia — CU1 Registrarse

CU2. Iniciar sesión

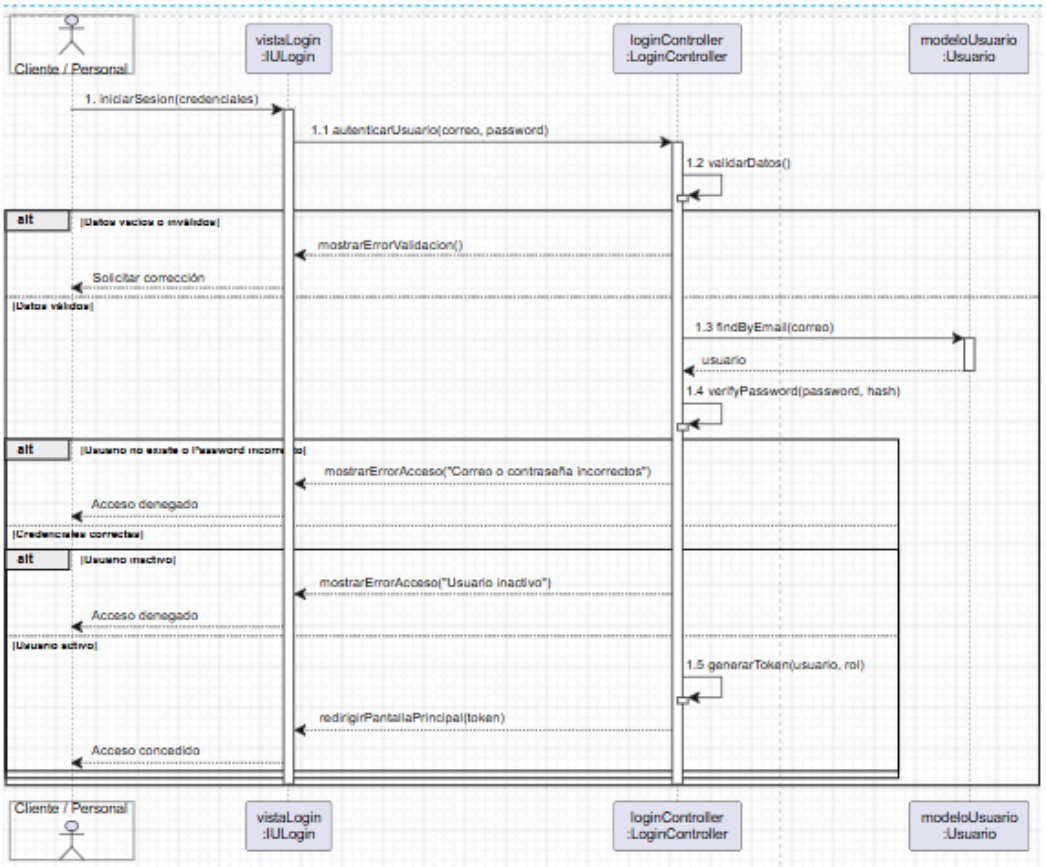


Figura 7. Diagrama de Secuencia — CU2 Iniciar sesión

CU3. Gestionar perfil

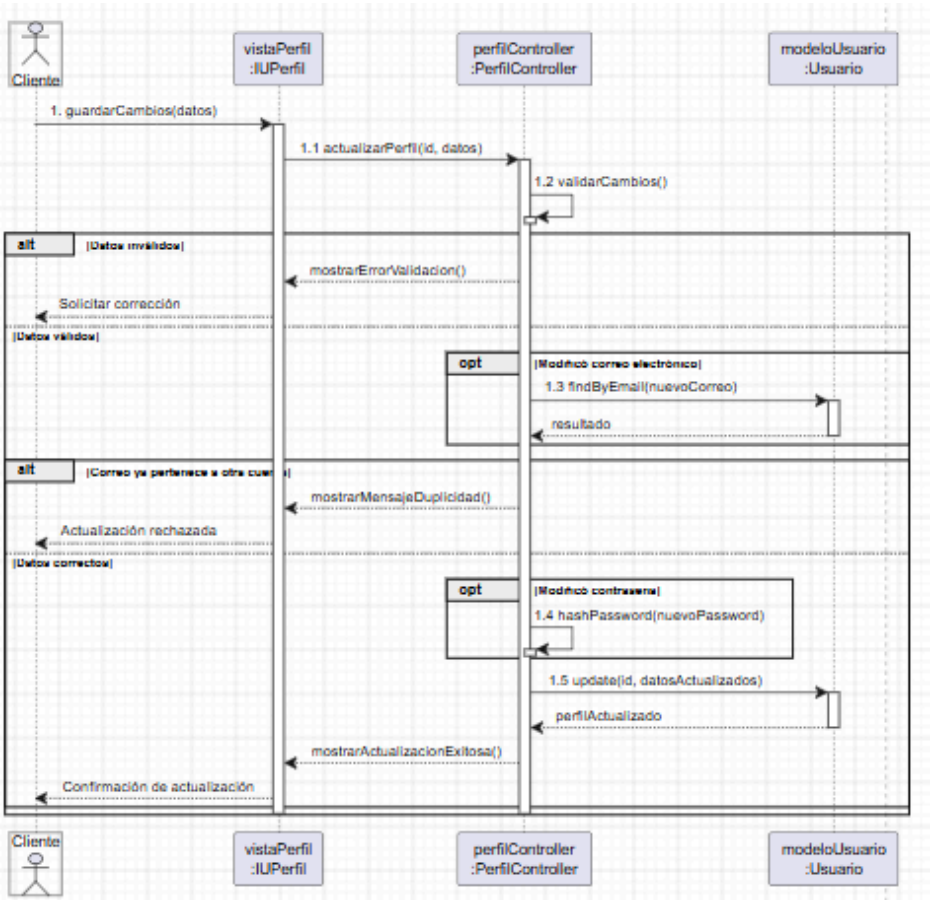


Figura 8. Diagrama de Secuencia — CU3 Gestionar perfil

CU5. Gestionar sucursales

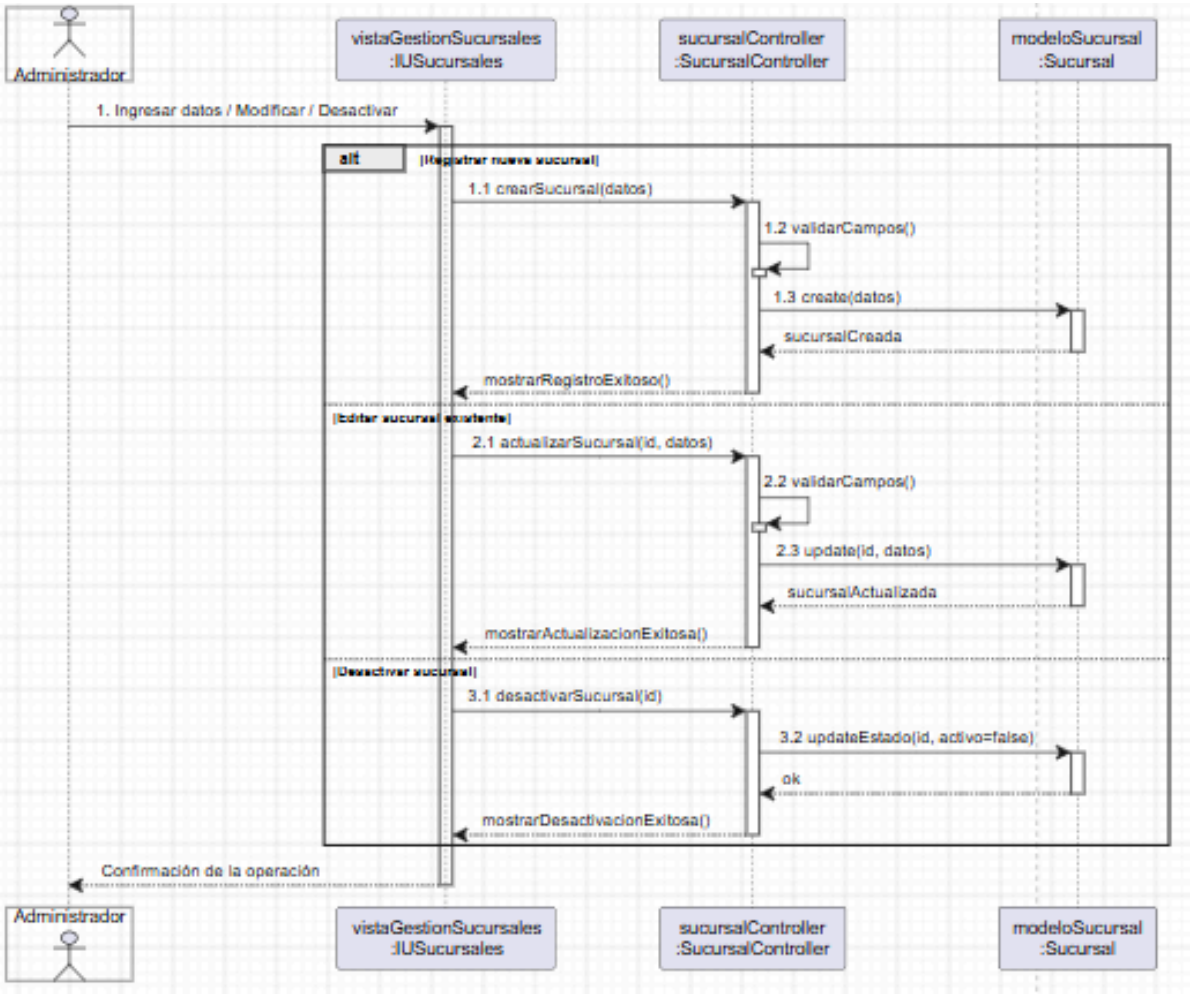


Figura 9. Diagrama de Secuencia — CU5 Gestionar sucursales

CU6. Consultar sucursales por ciudad

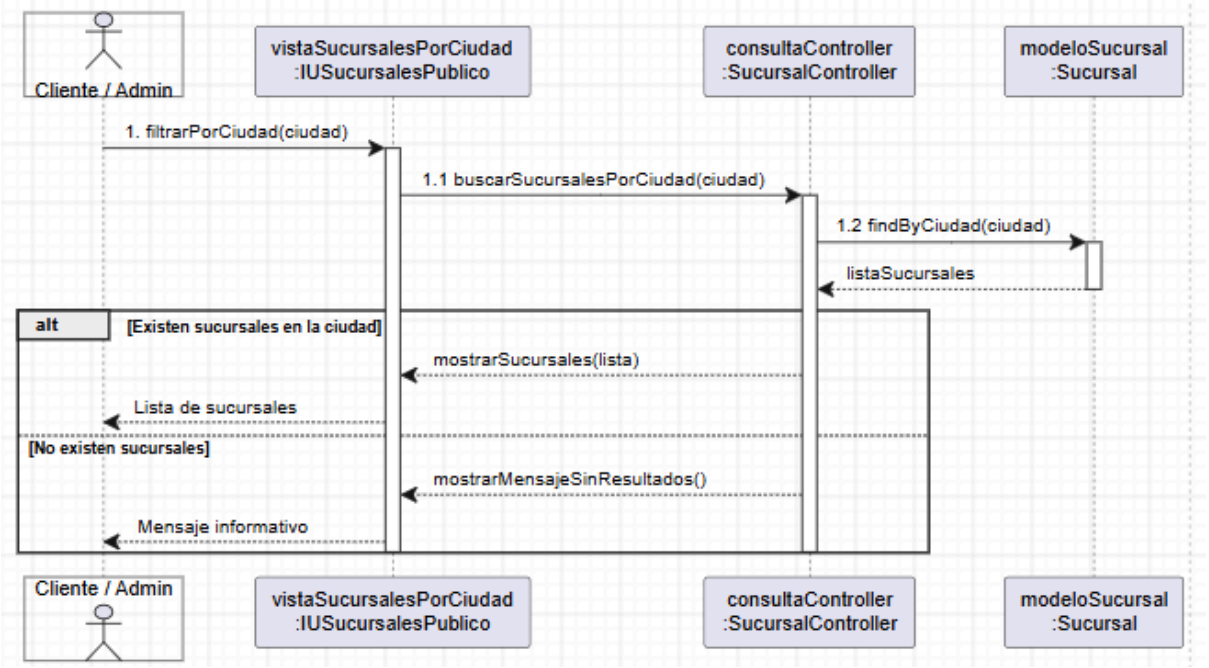


Figura 10. Diagrama de Secuencia — CU6 Consultar sucursales por ciudad

CU7. Gestionar productos

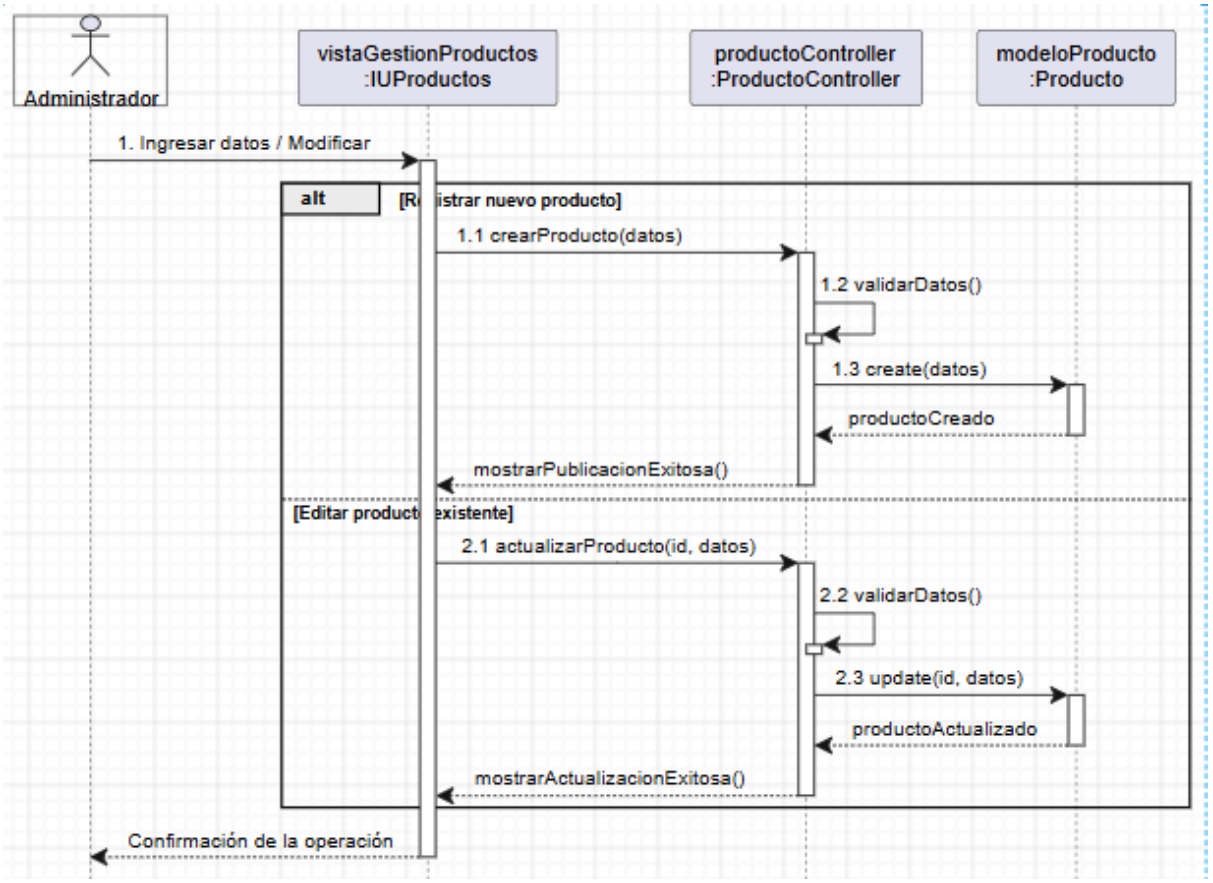


Figura 11. Diagrama de Secuencia — CU7 Gestionar productos

CU10. Consultar catálogo y buscar prendas

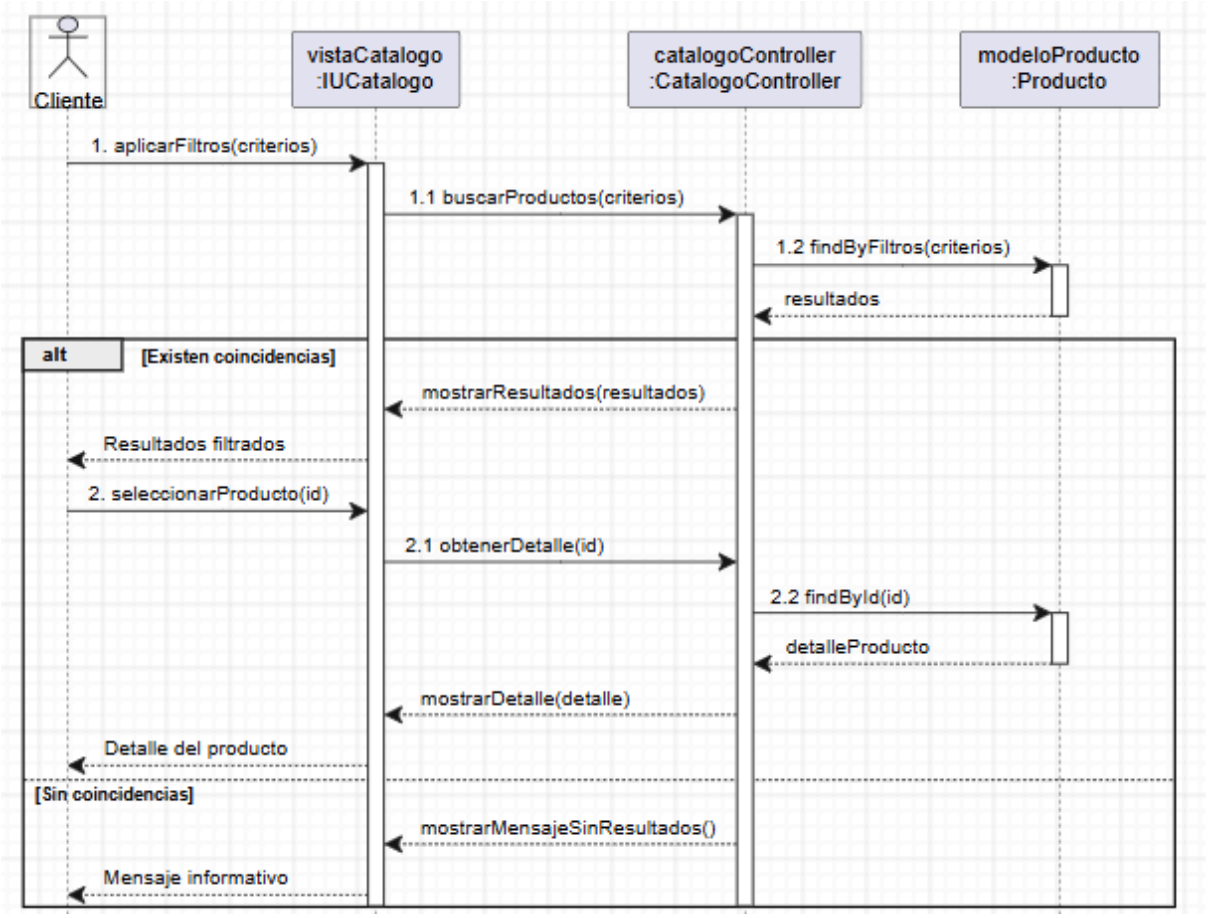


Figura 12. Diagrama de Secuencia — CU10 Consultar catálogo y buscar prendas

Ciclo #2

CU4. Gestionar usuarios y roles

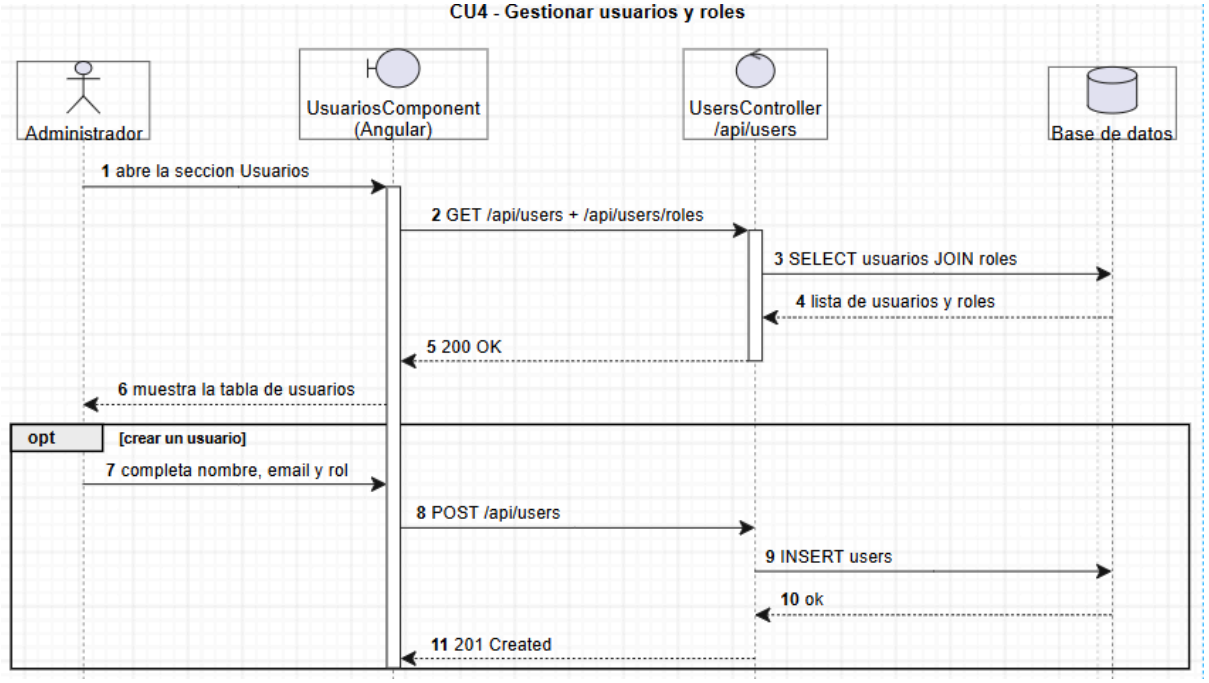


Figura 13. Diagrama de Secuencia CU4. Gestionar usuarios y roles

CU8. Gestionar temporadas y colecciones

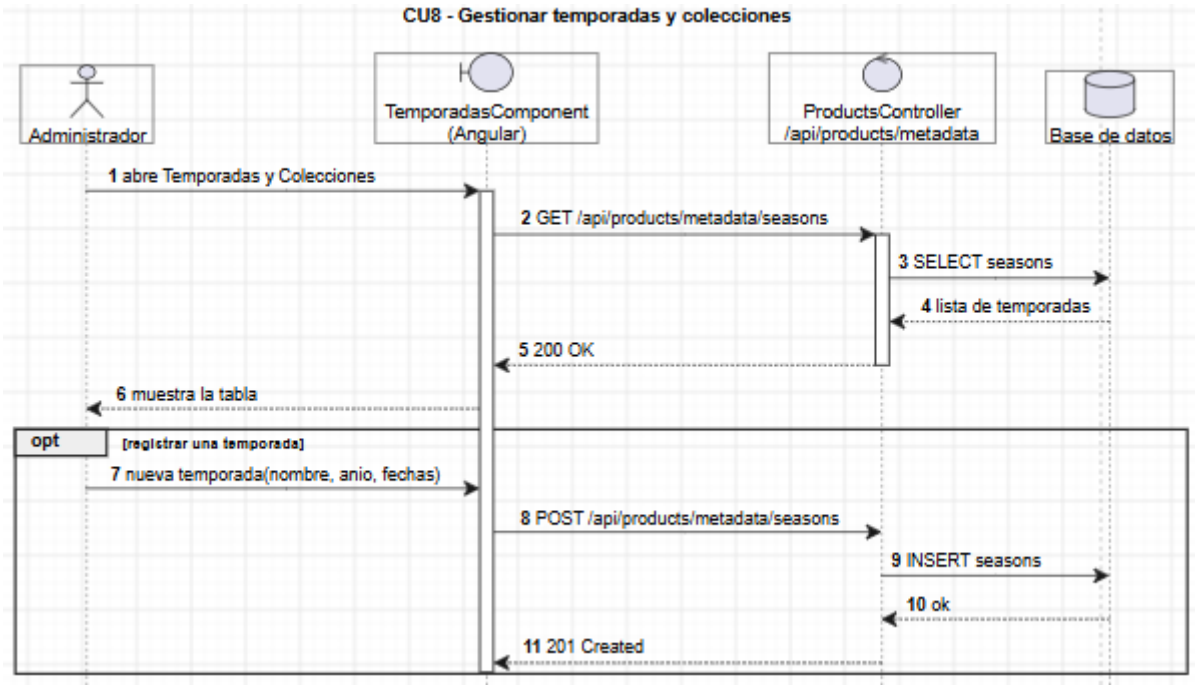


Figura 14. Diagrama de Secuencia CU8. Gestionar temporadas y colecciones

CU9. Gestionar proveedores

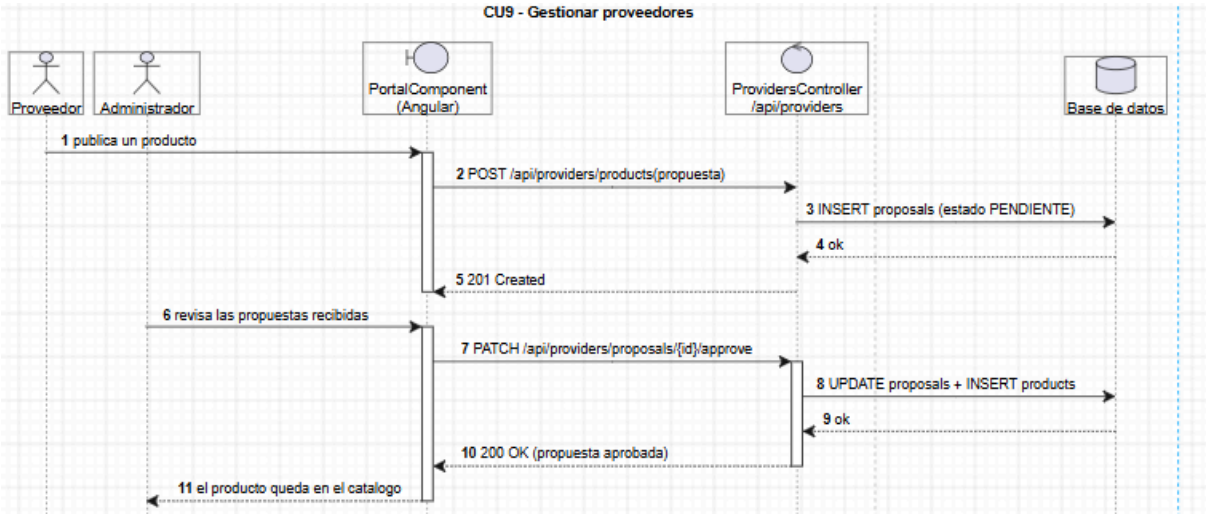


Figura 15. Diagrama de Secuencia CU9. Gestionar proveedores

CU11. Consultar disponibilidad por sucursal

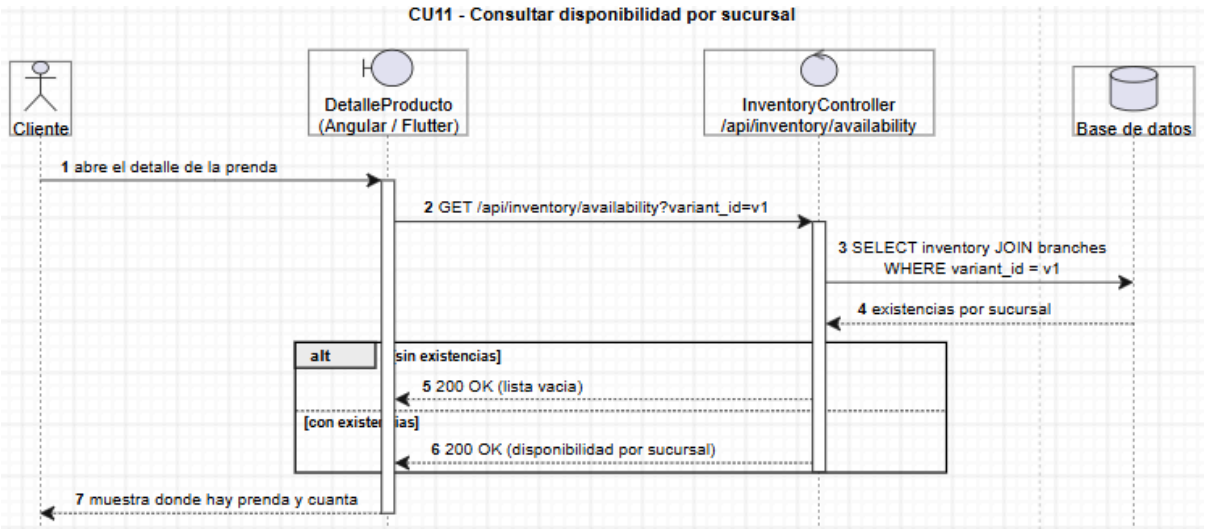


Figura 16. Diagrama de Secuencia CU11. Consultar disponibilidad por sucursal

CU13. Reservar prendas

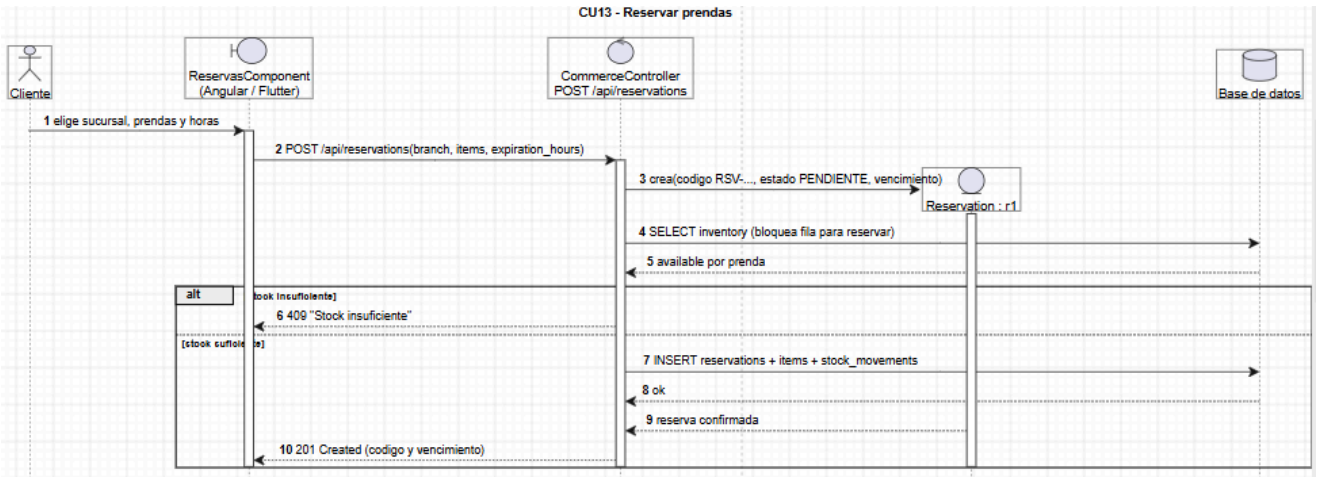


Figura 17. Diagrama de Secuencia CU13. Reservar prendas

CU14. Gestionar reserva (consultar / cancelar)

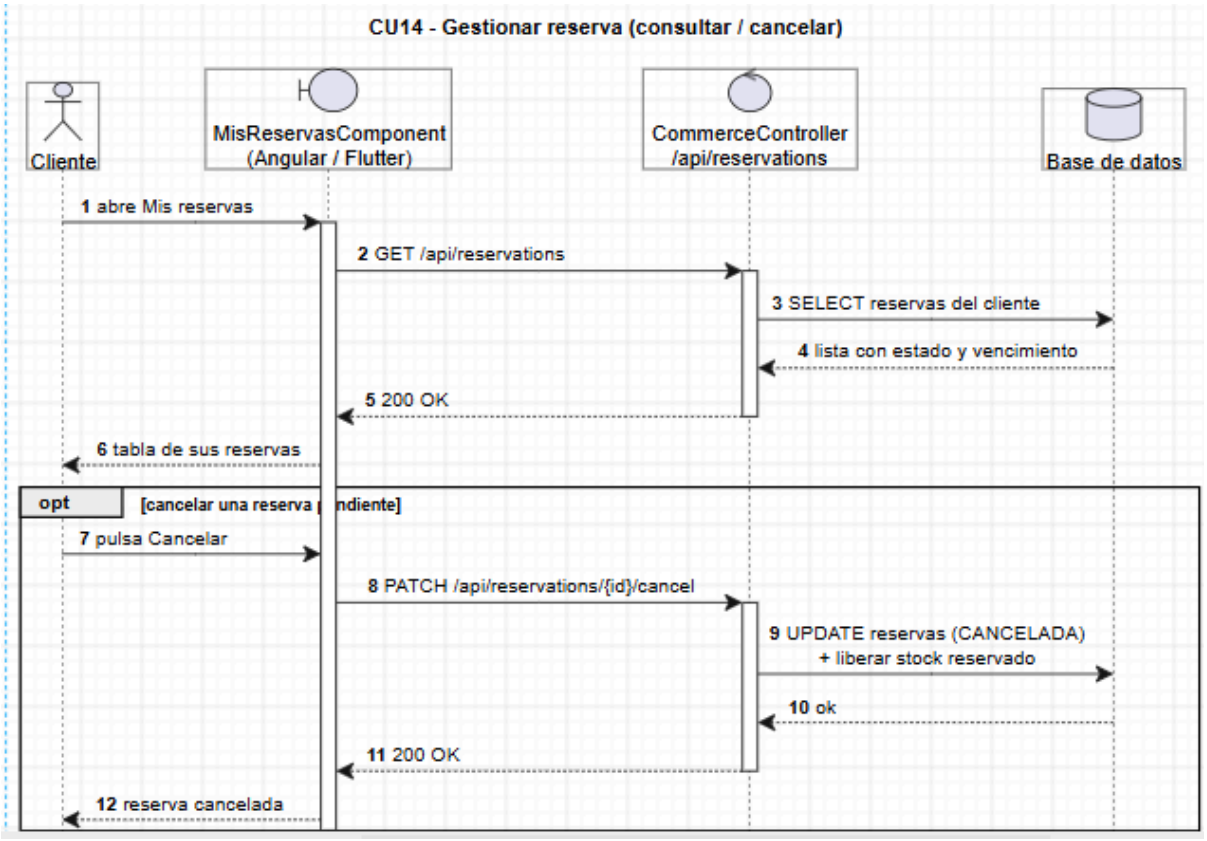


Figura 18. Diagrama de Secuencia CU14. Gestionar reserva (consultar / cancelar)

CU15. Atender reserva en sucursal

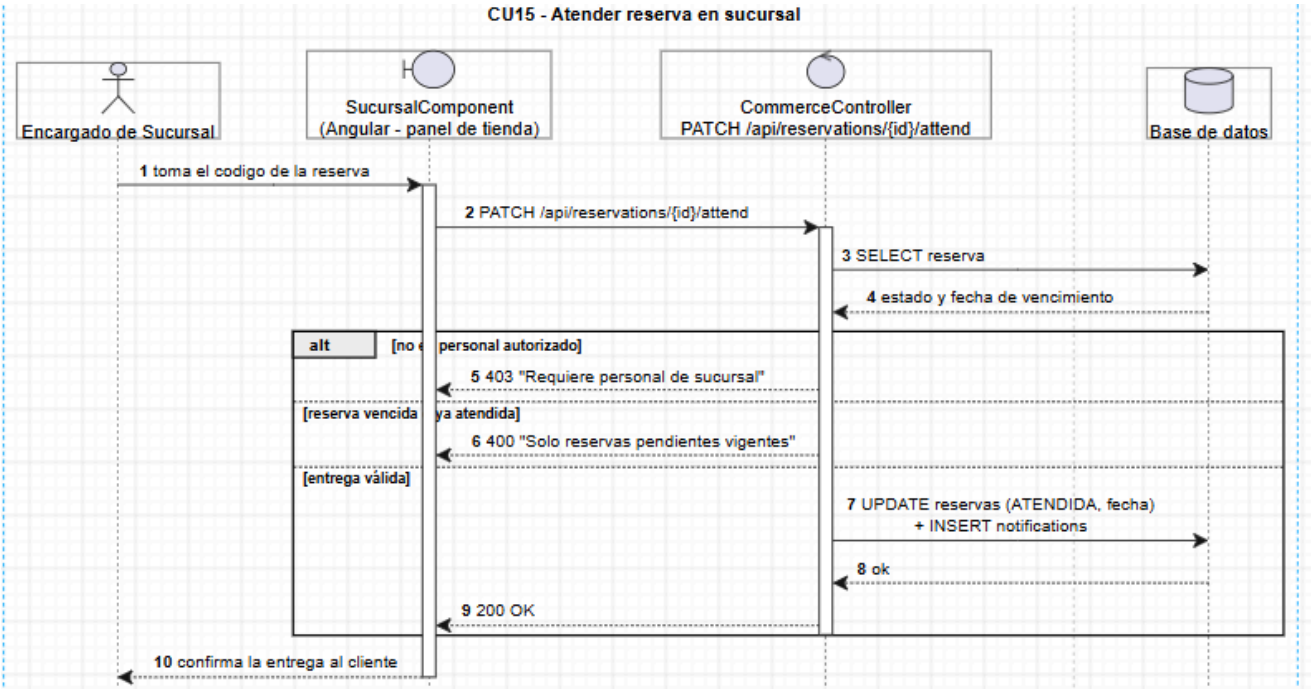


Figura 19. Diagrama de Secuencia CU15. Atender reserva en sucursal

CU16. Gestionar carrito de compras

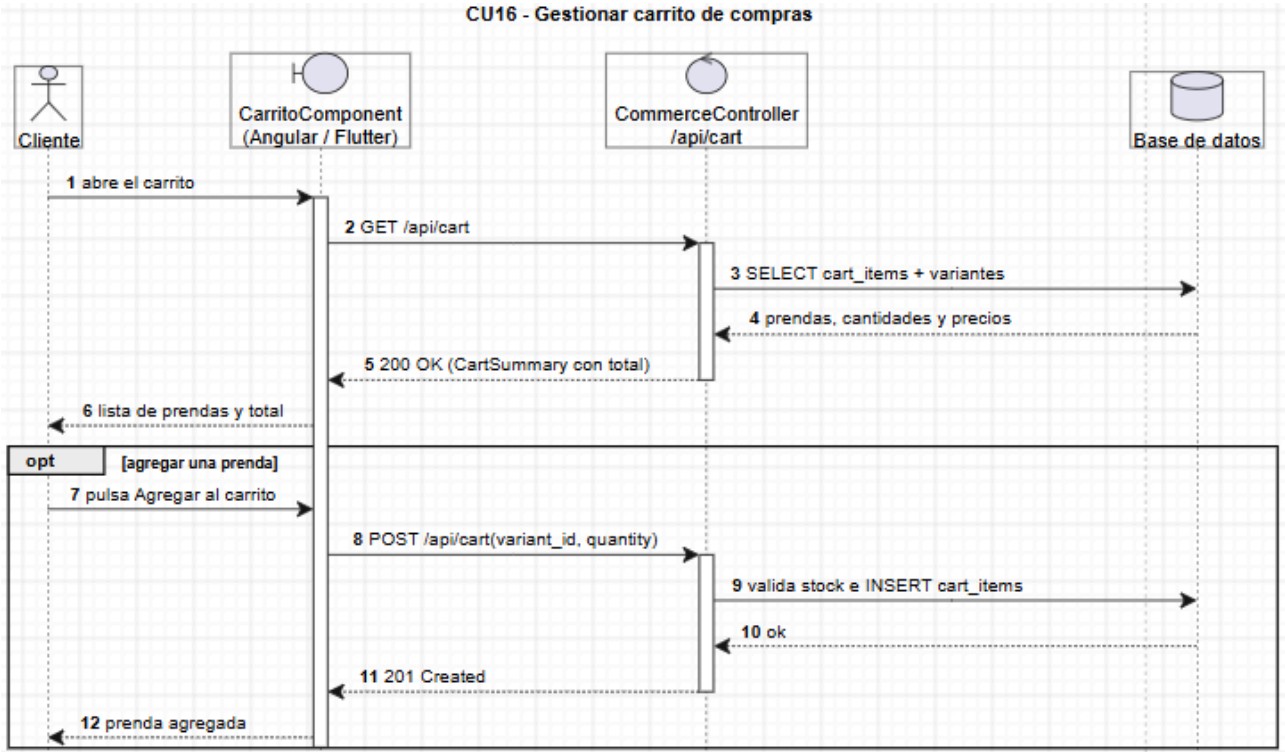


Figura 20. Diagrama de Secuencia CU16. Gestionar carrito de compras

CU21. Gestionar movimientos de inventario

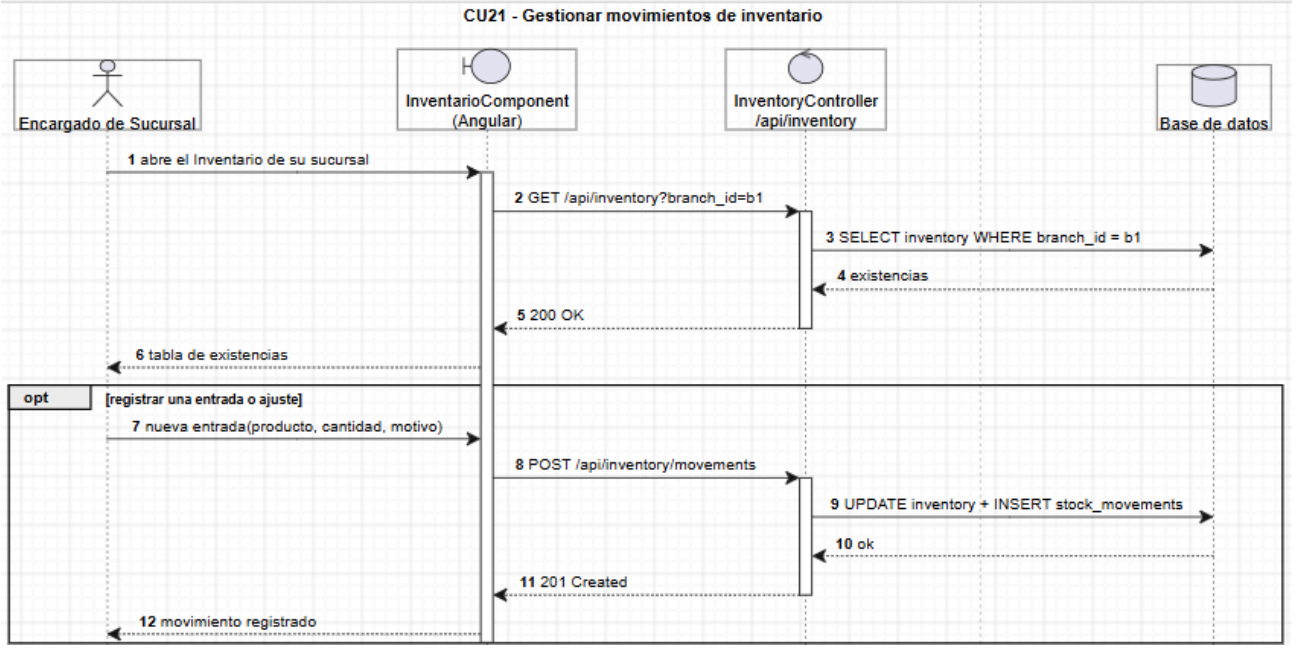
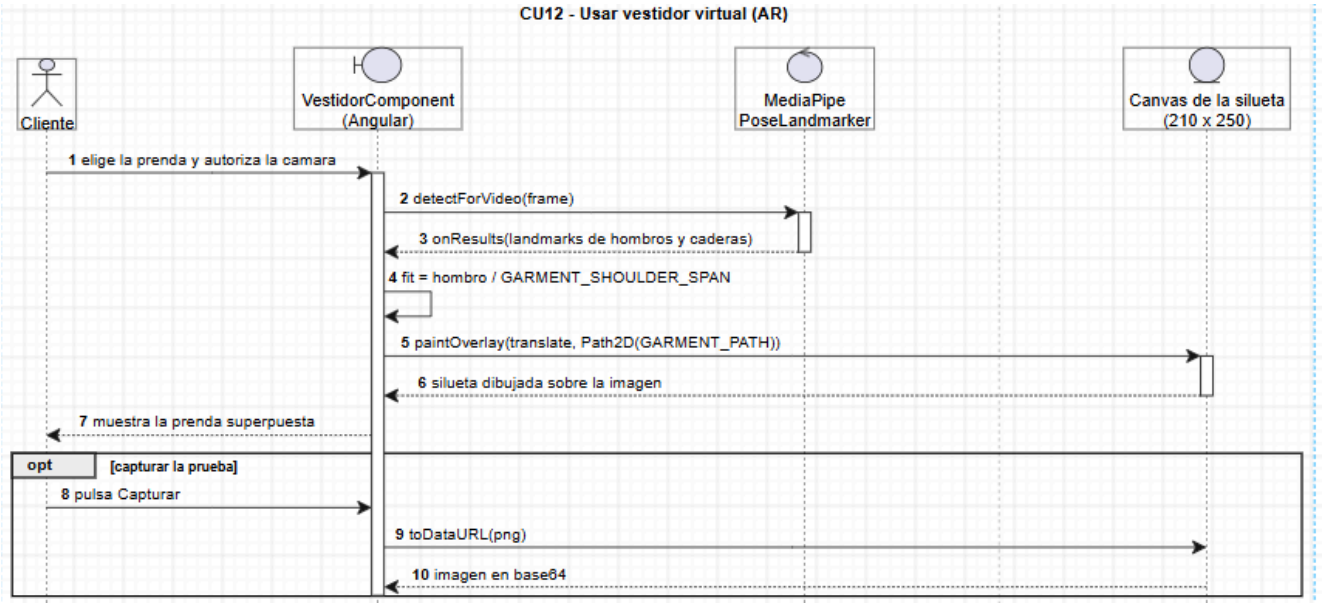


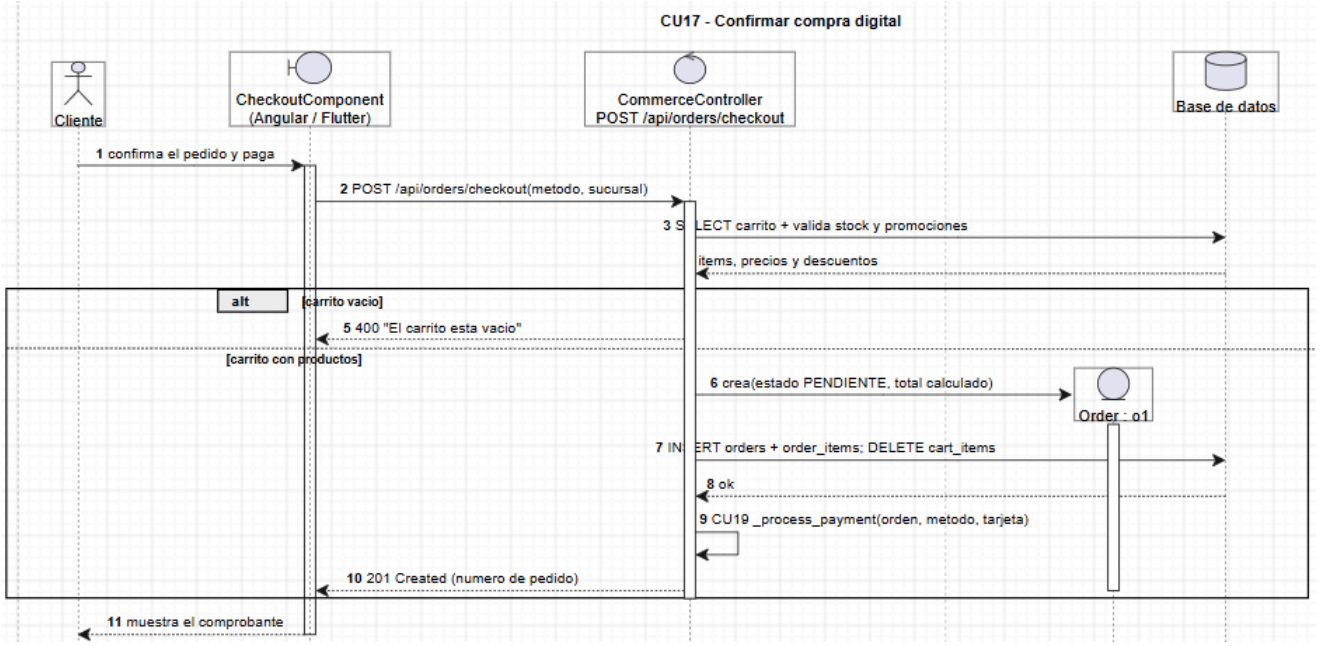
Figura 21. Diagrama de Secuencia CU21. Gestionar movimientos de inventario

CICLO #3

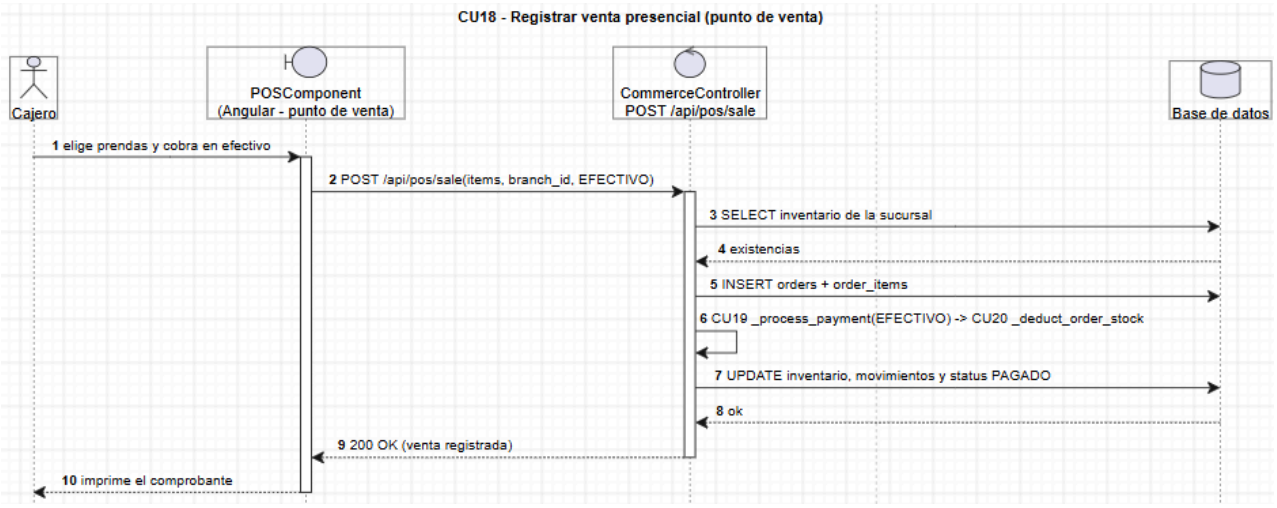
CU12 — Usar vestidor virtual (AR)



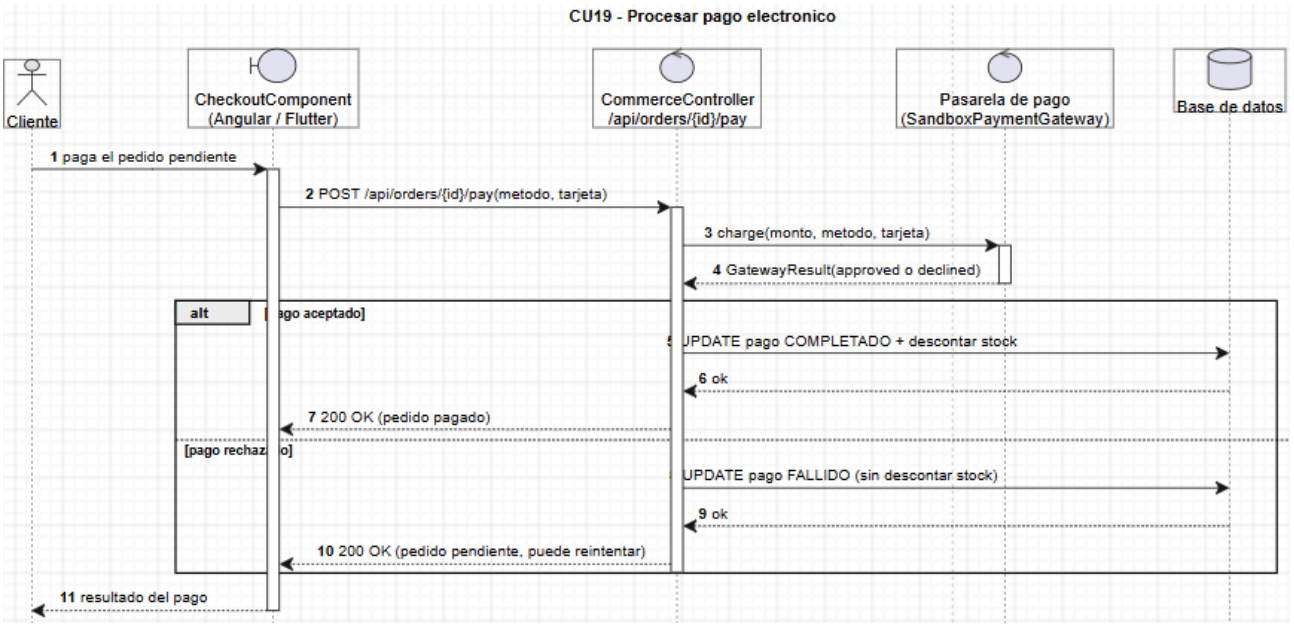
CU17 — Confirmar compra digital



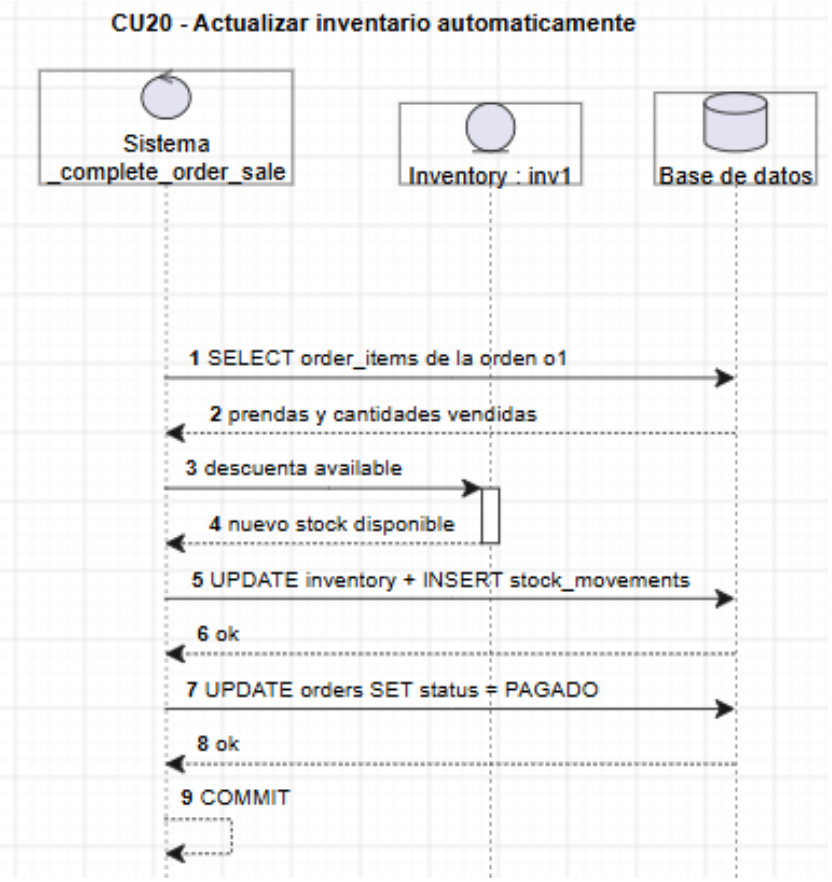
CU18 — Registrar venta presencial



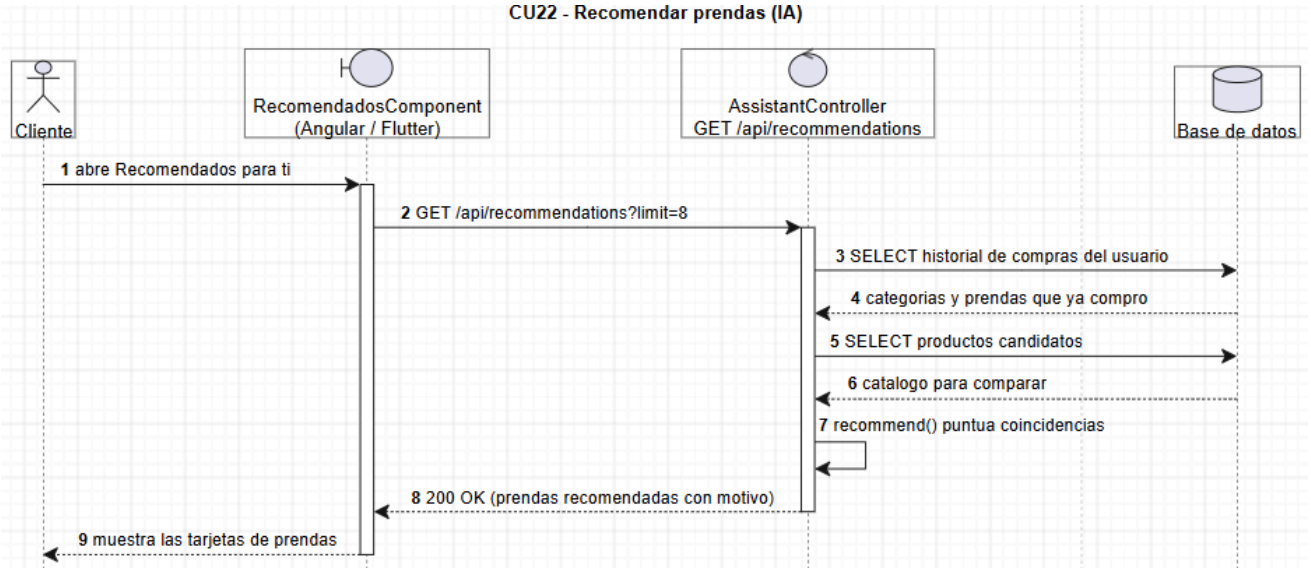
CU19 — Procesar pago electrónico



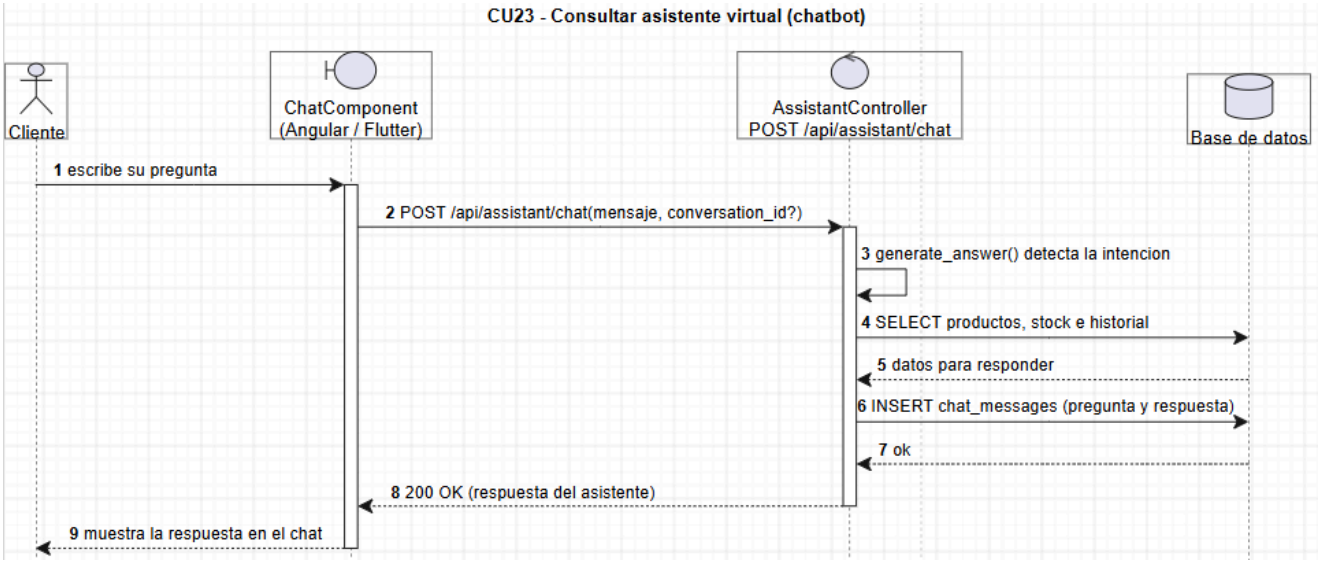
CU20 — Actualizar inventario automáticamente



CU22 — Recomendar prendas (IA)

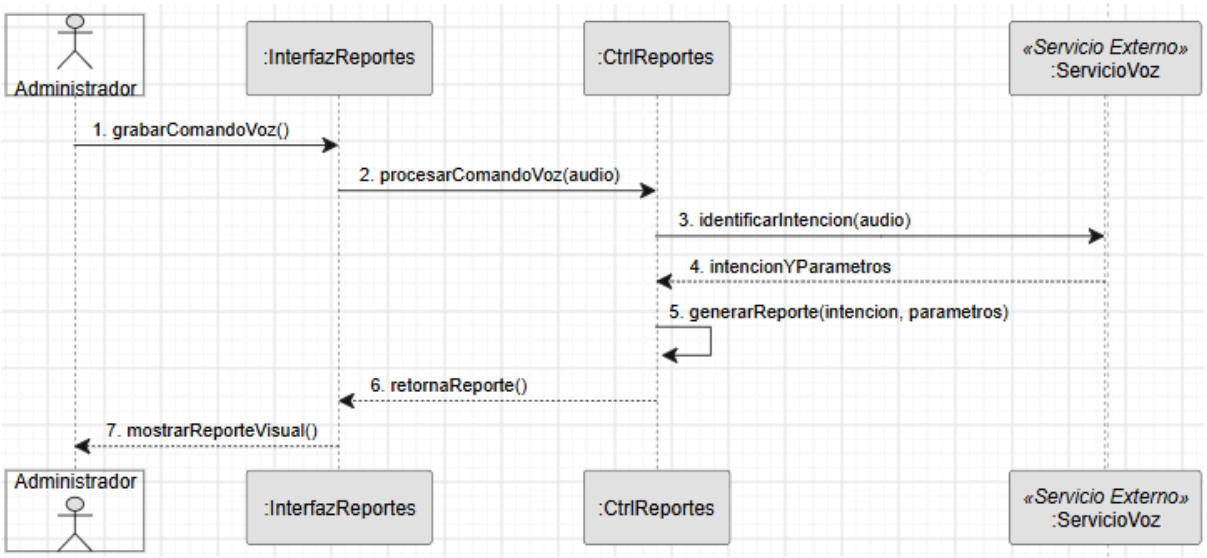


CU23 — Consultar asistente virtual (chatbot)

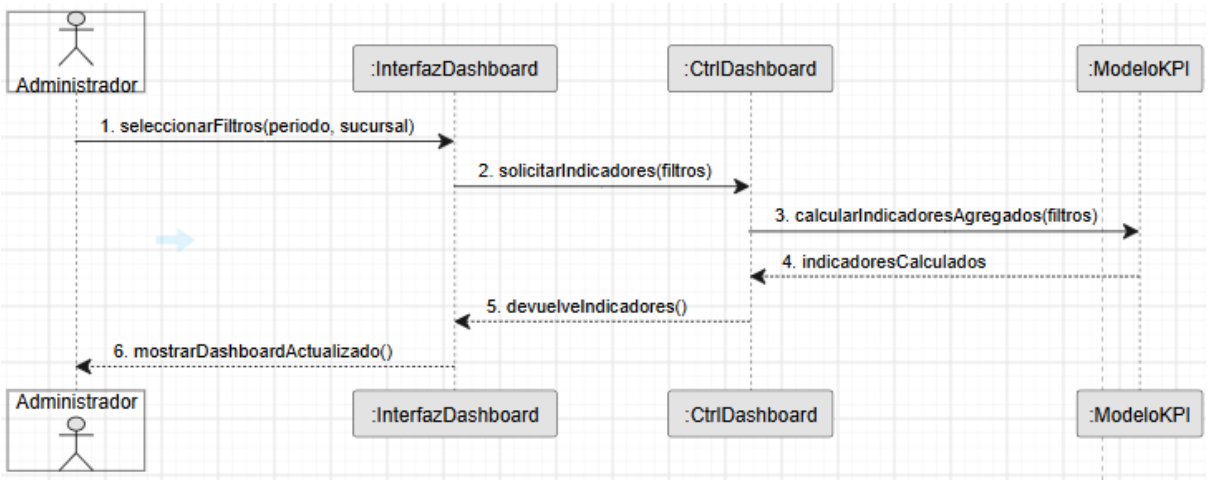


CICLO #4

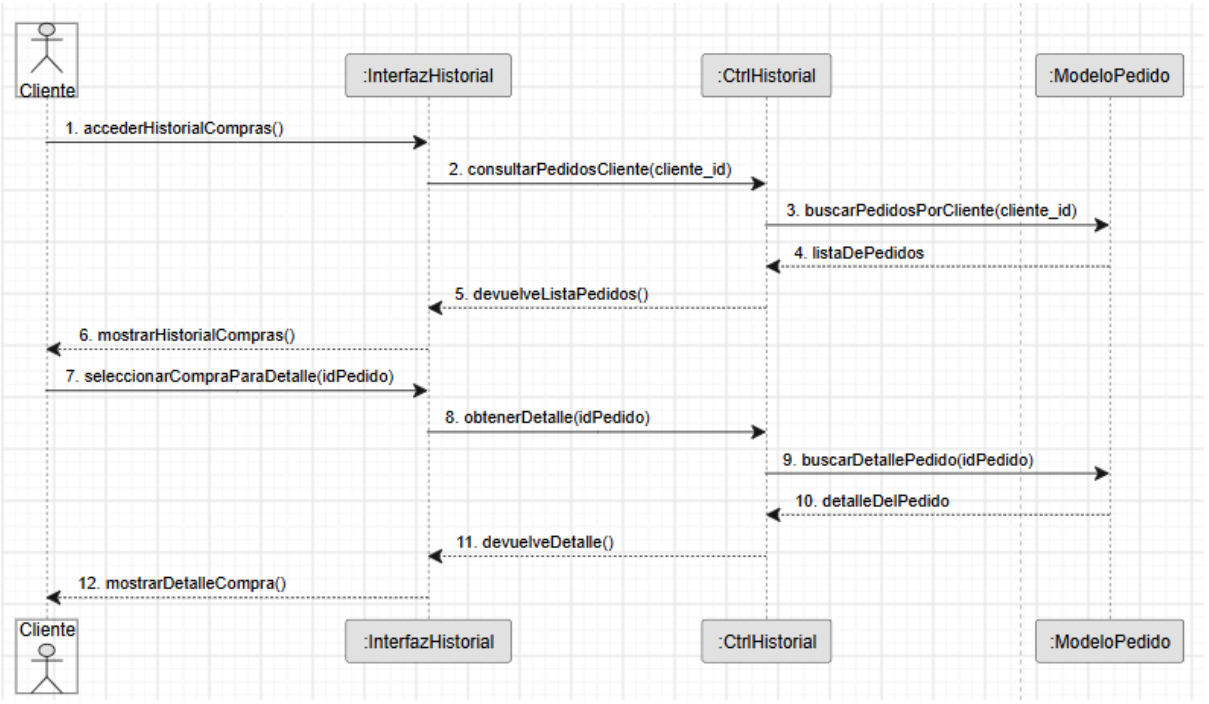
CU24 — Generar reporte por comando de voz (extensión opcional, prioridad baja)



CU25 — Consultar reportes y dashboards



CU26 — Consultar historial de compras



A.7. Diagramas de estado, de tiempo y de navegación

Diagrama de Estado

Modela los estados por los que atraviesa la entidad central de cada proceso y los eventos que disparan cada transición, tomando como base los valores de "estado" ya definidos en el script SQL

Proceso Catálogo

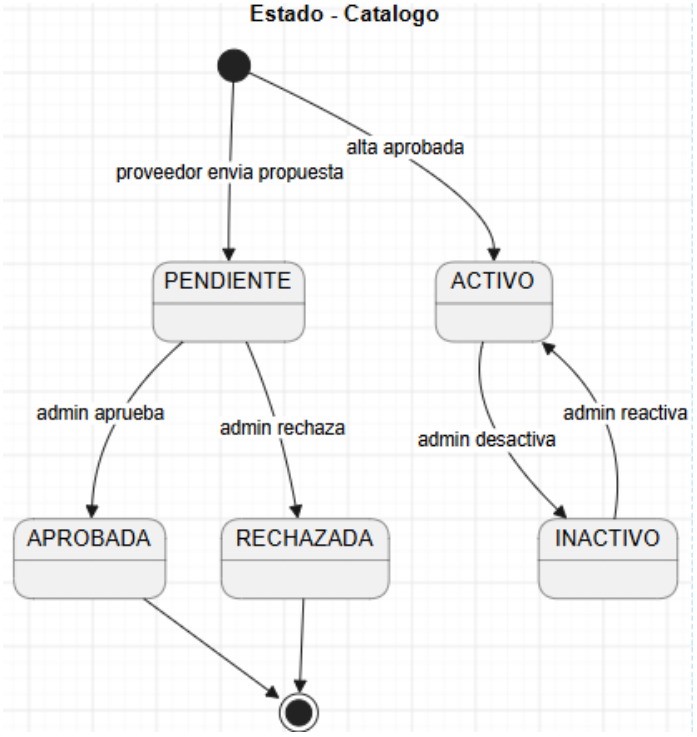


Figura 24. Diagrama de Estado Proceso Catálogo (Producto)

Proceso Venta

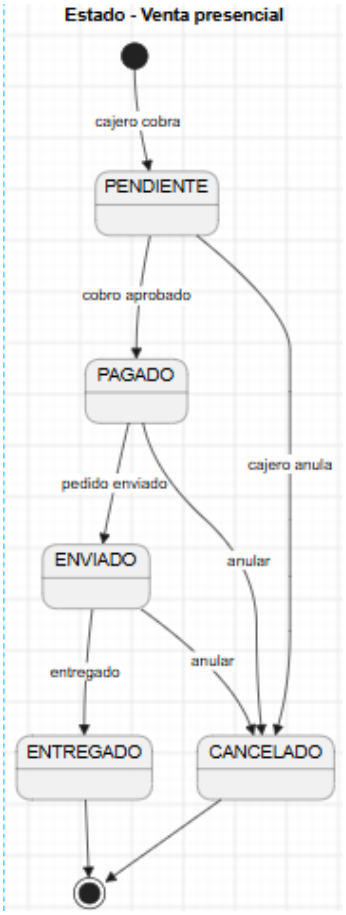


Figura 25. Diagrama de Estado Proceso Venta (Pedido)

Proceso Compra

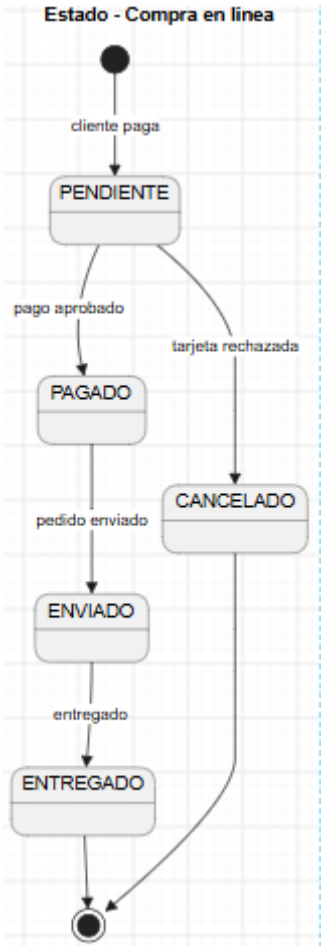


Figura 26. Diagrama de Estado Proceso Compra (Reserva)

Proceso Ficha de Pago

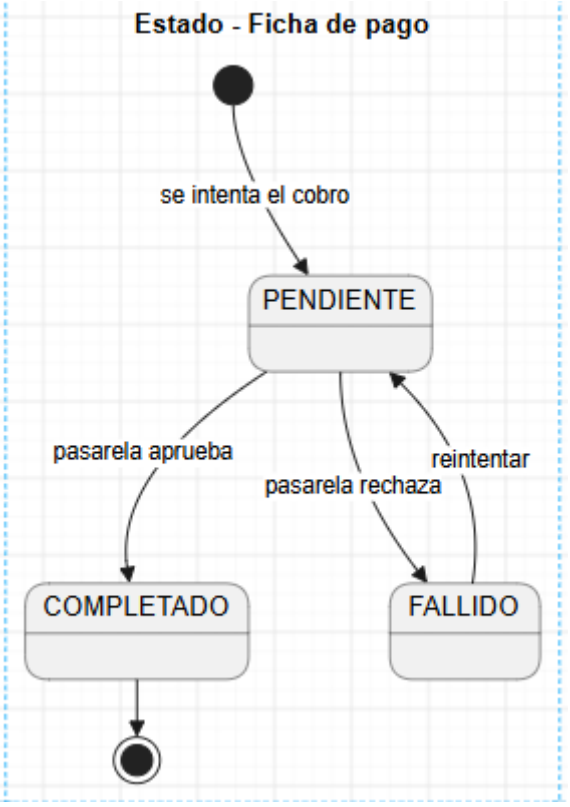


Figura 27. Diagrama de Estado Proceso Ficha de Pago (Pago)

Diagrama de Tiempo

Muestra el mismo ciclo de vida de la entidad central de cada proceso, ahora contra un eje de tiempo, con la duración aproximada de permanencia en cada estado.

Proceso Catálogo

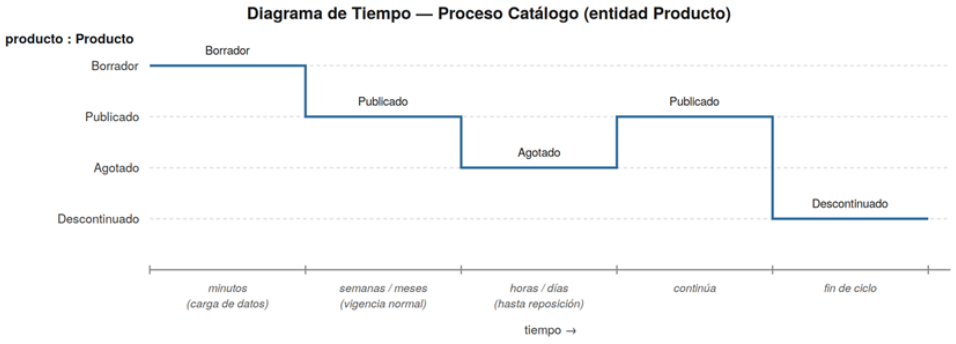


Figura 28. Diagrama de Tiempo Proceso Catálogo (Producto)

Proceso Venta

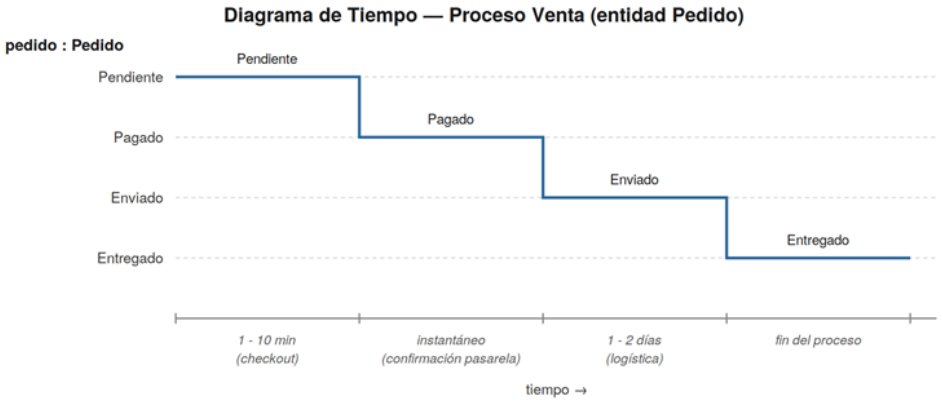


Figura 29. Diagrama de Tiempo Proceso Venta (Pedido)

Proceso Compra

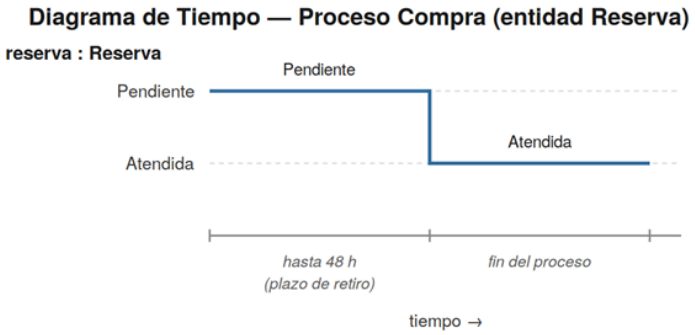


Figura 30. Diagrama de Tiempo Proceso Compra (Reserva)

Proceso Ficha de Pago

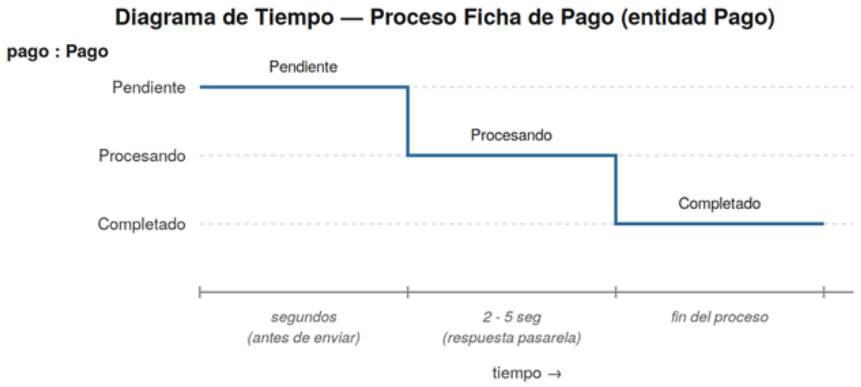


Figura 31. Diagrama de Tiempo Proceso Ficha de Pago (Pago)

Diagrama de Navegación

Muestra las pantallas involucradas en cada proceso y las acciones del usuario que disparan la transición de una pantalla a otra.

Proceso Catálogo

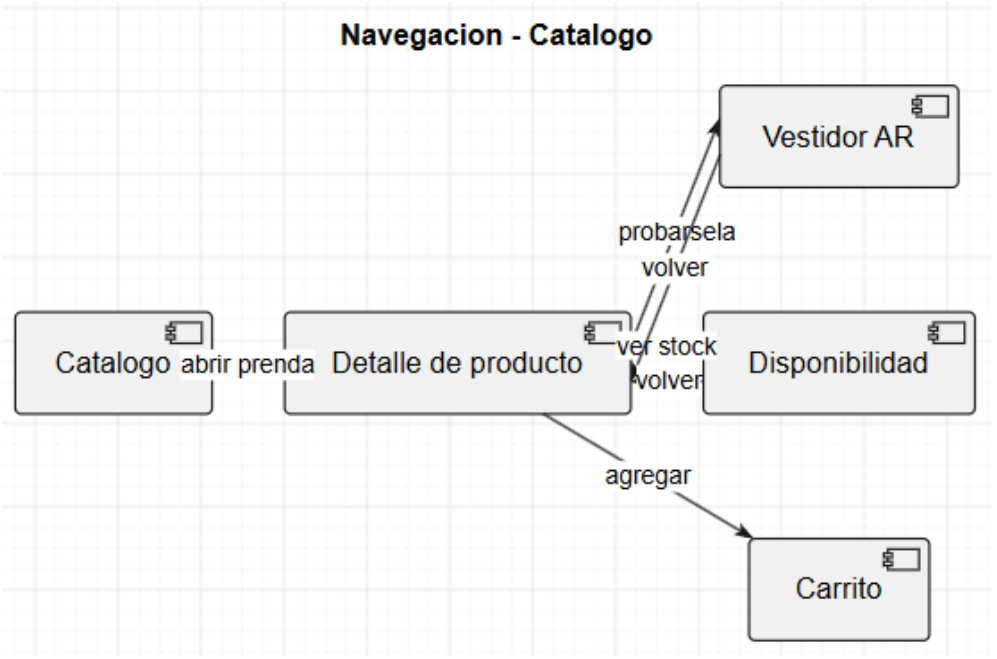


Figura 32. Diagrama de Navegación Proceso Catálogo

Proceso Venta

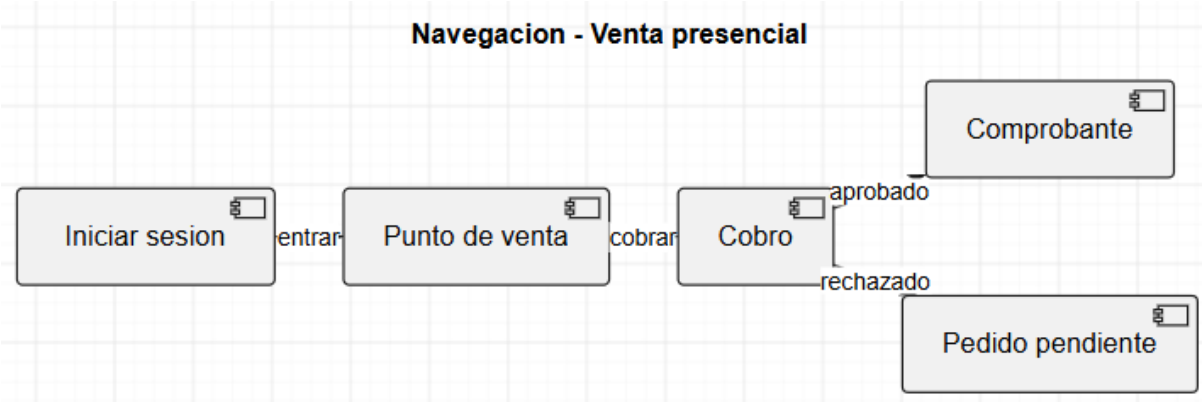


Figura 33. Diagrama de Navegación Proceso Venta (checkout digital)

Proceso Compra

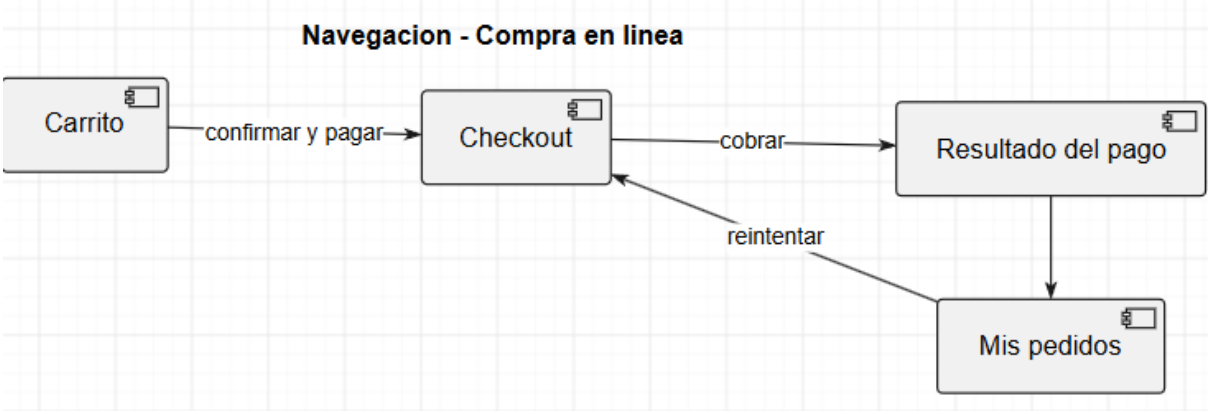


Figura 34. Diagrama de Navegación Proceso Compra (reserva y retiro en sucursal)

Proceso Ficha de Pago

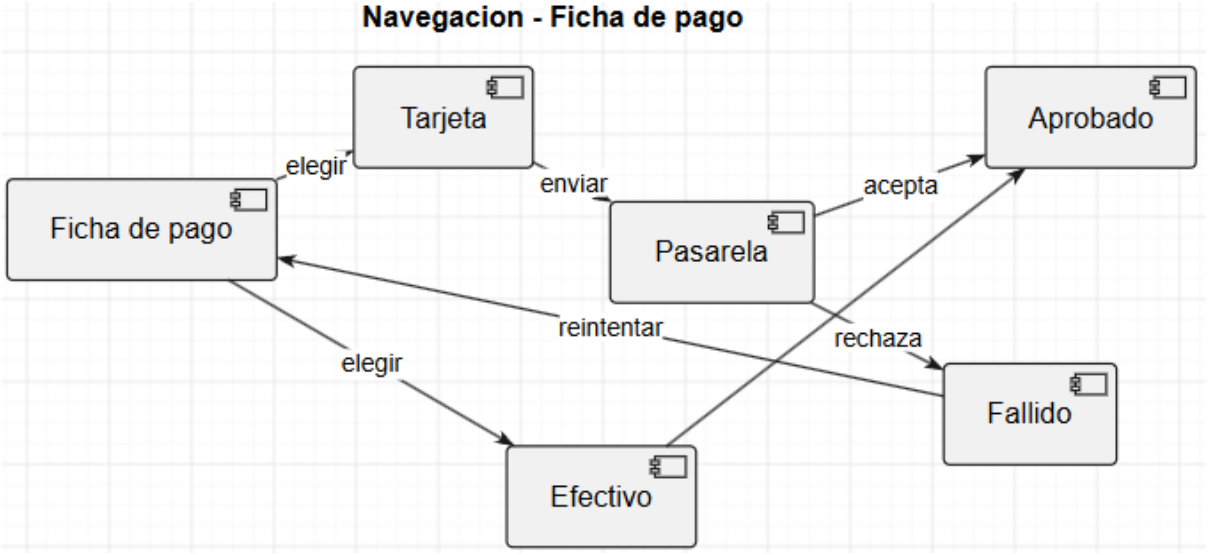
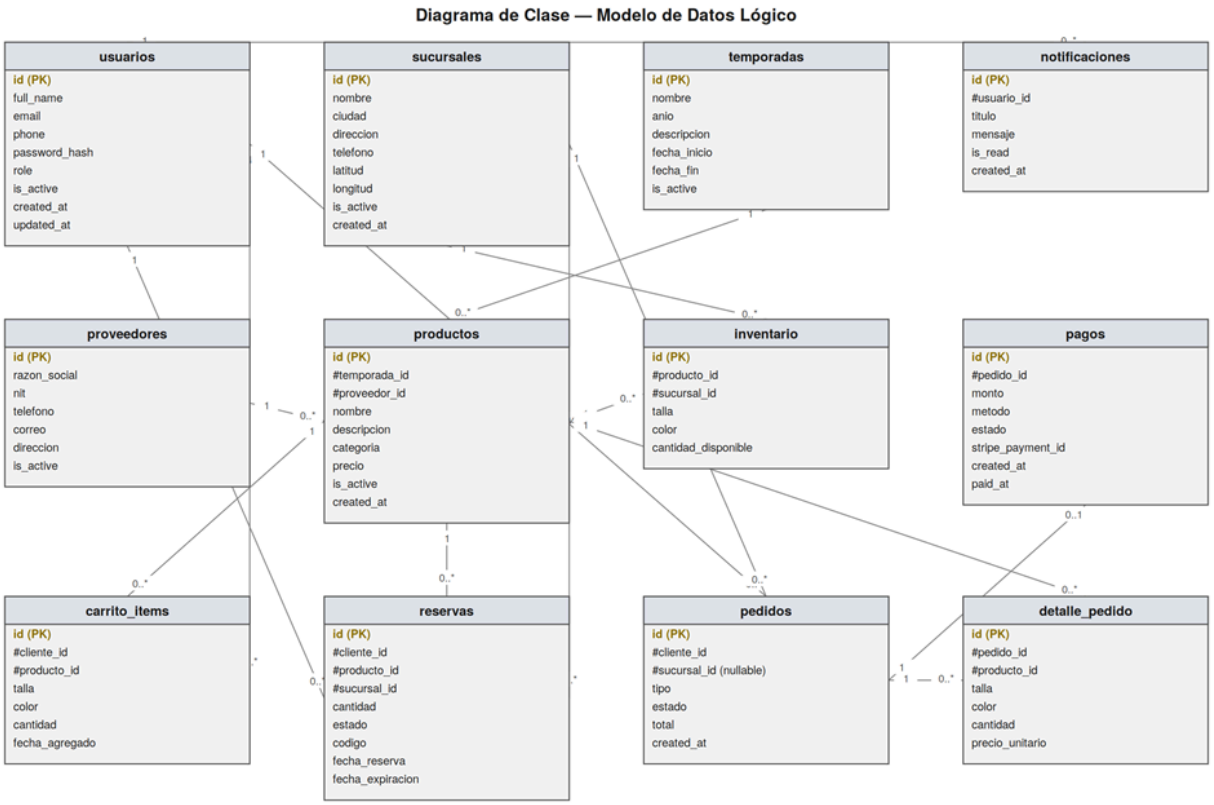


Figura 35. Diagrama de Navegación Proceso Ficha de Pago

A.8. Diagrama de clases



A.9. Diagramas de componentes

4.2.1. Diagrama de Componente Principal del Sistema

El sistema se organiza en tres subsistemas: el Frontend (aplicaciones cliente móvil y web), el Servidor Backend (API REST en FastAPI, con los servicios de autenticación, disponibilidad/reserva, pagos y notificaciones) y el Subsistema de Datos (PostgreSQL vía SQLAlchemy ORM).

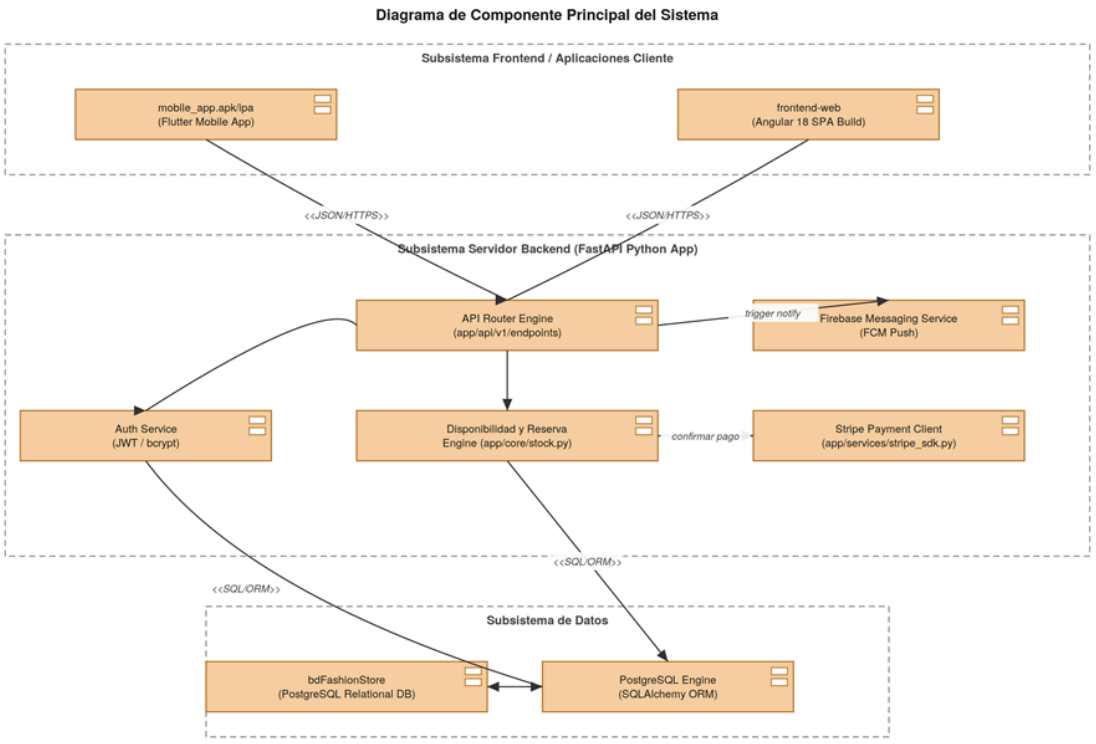


Figura 36. Diagrama de Componente Principal del Sistema

4.3.1. Diagrama de Componente de cada Paquete

Paquete 1: Gestión de Identidad y Acceso

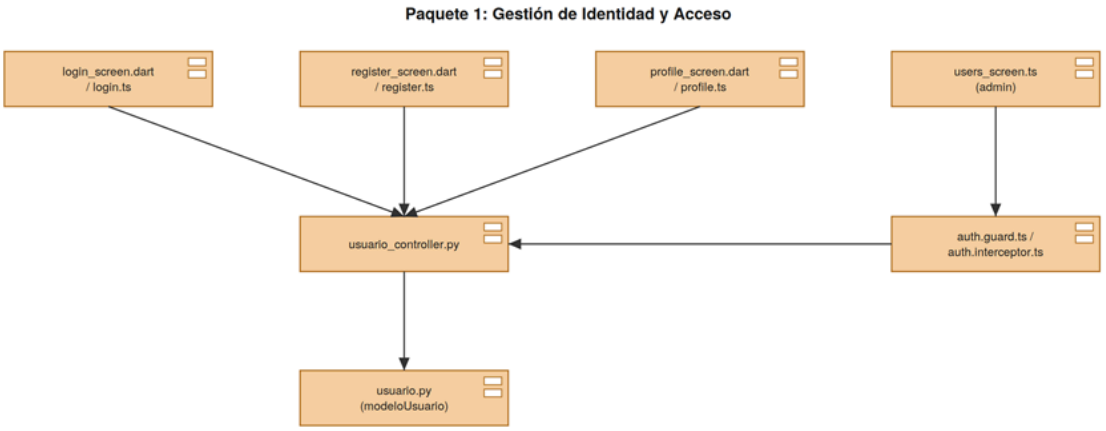


Figura 37. Diagrama de Componente — Paquete 1: Gestión de Identidad y Acceso

Paquete 2: Gestión de Catálogo y Sucursales

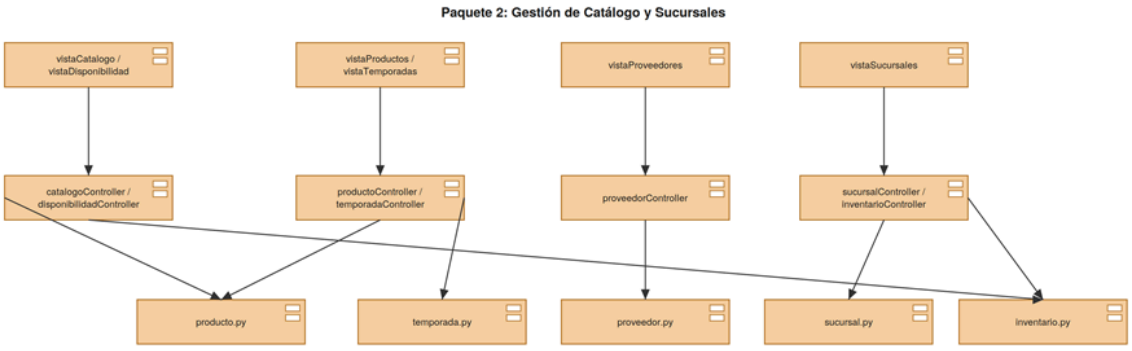


Figura 38. Diagrama de Componente — Paquete 2: Gestión de Catálogo y Sucursales

Paquete 3: Gestión de Operaciones de Compra

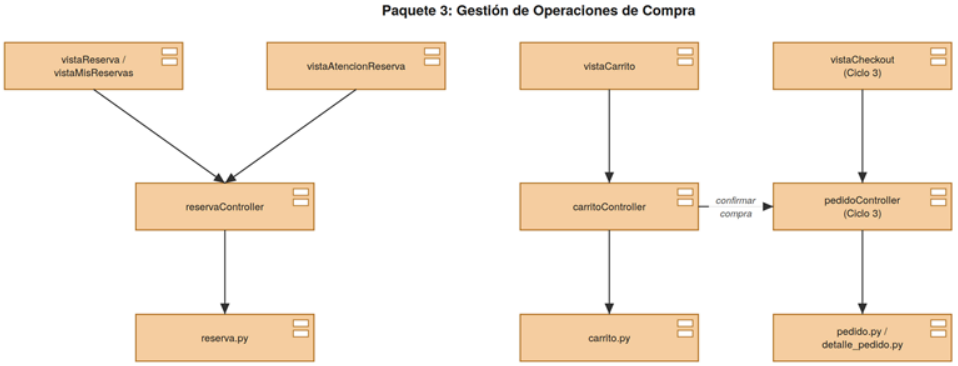


Figura 39. Diagrama de Componente — Paquete 3: Gestión de Operaciones de Compra

Paquete 4: Inteligencia de Recomendación y Vestidor Virtual

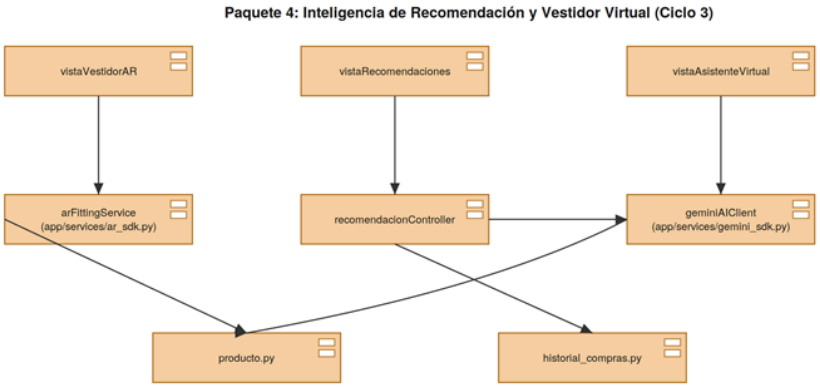


Figura 40. Diagrama de Componente — Paquete 4: Inteligencia de Recomendación y Vestidor Virtual (Ciclo 3)

Paquete 5: Pagos y Notificaciones

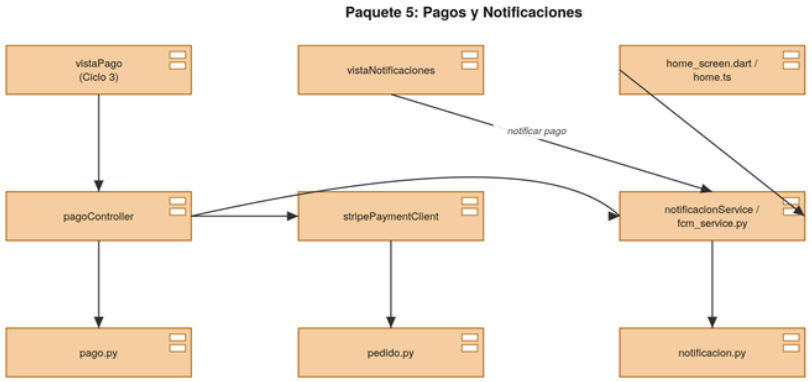


Figura 41. Diagrama de Componente — Paquete 5: Pagos y Notificaciones